เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร

(Computer System Organization)

บทที่ 7 เรื่องหน่วยความจำ (Memory Unit)

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. บอกหน้าที่ของหน่วยความจำ (Memory Unit) ได้อย่างถูกต้อง
2. บอกประเภทและองค์ประกอบของหน่วยความจำ (Memory Unit) ได้อย่างถูกต้อง
3. บอกลำดับขั้นและลำดับชั้นของหน่วยความจำ (Memory Hierarchy) ได้อย่างถูกต้อง
4. อธิบายความจุและความเร็วในการทำงานการเข้าถึงของหน่วยความจำชนิดต่างๆ ได้อย่างถูกต้อง
5. อธิบายการเปรียบเทียบความเร็วการทำงานหน่วยความจำชนิดต่างๆ กับราคาต้นทุนในการผลิตได้อย่างถูกต้อง
6. อธิบายคุณลักษณะของหน่วยความจำ (Memory Characteristics) ในการควบคุมตำแหน่งของหน่วยความจำภายใน (Internal Memory) และ.แบบภายนอก (External Memory) ได้อย่างถูกต้อง
7. อธิบายคุณลักษณะของหน่วยความจำ (Memory Characteristics) ค่าปริมาณความจุของหน่วยความจำขนาดต่างๆ ได้อย่างถูกต้อง
8. อธิบายคุณลักษณะของหน่วยความจำ (Memory Characteristics) การถ่ายโอนข้อมูล (Unit of Transfer) แบบเวิร์ด (Word) และแบบบล็อก (Block) ได้อย่างถูกต้อง
9. อธิบายคุณลักษณะของหน่วยความจำ (Memory Characteristics) การเข้าถึงข้อมูล (Access Method):แบบแบบตามลำดับ (Sequential access) และ แบบการเข้าถึงโดยตรง (Direct access) ได้อย่างถูกต้อง
10. อธิบายคุณลักษณะของหน่วยความจำ (Memory Characteristics) การเข้าถึงข้อมูล (Access Method) แบบสุ่ม (Random Access) และแบบการเข้าถึงแบบที่มีความเกี่ยวข้องกัน (Associative) ได้อย่างถูกต้อง
11. อธิบายการติดต่อระหว่าง CPU กับหน่วยความจำหลัก (Main Memory) ได้อย่างถูกต้อง
12. อธิบายการใช้งานหน่วยความจำหลัก (Main Memory) ประเภท ROM (Read Only Memory) และ Flash Memory ได้อย่างถูกต้อง
13. อธิบายใช้งานของหน่วยความจำหลัก (Main Memory) ประเภท RAM (Random Access Memory) ได้อย่างถูกต้อง
14. อธิบายการทำงานของหน่วยความจำแบบลบเลือน (Volatile) ประเภท SRAM (Static RAM) ได้อย่างถูกต้อง
15. อธิบายการทำงานของหน่วยความจำแบบลบเลือน (Volatile) ประเภท DRAM (Dynamic RAM) ได้อย่างถูกต้อง
16. อธิบายการทำงานของหน่วยความจำแบบลบเลือน (Volatile) ประเภท DDR SDRAM (Synchronous DRAM) ได้อย่างถูกต้อง

17. อธิบายการพัฒนาของ DDR RAM จากอดีตถึงปัจจุบันได้อย่างถูกต้อง
18. อธิบายประเภทและคุณสมบัติของ DIMM (Dual In-line Memory Module) ได้อย่างถูกต้อง
19. อธิบายการทำงานของหน่วยความจำหลัก (Main Memory) ประเภท NVDIMM (Non-volatile Dual inline memory module) ได้อย่างถูกต้อง
20. อธิบายการทำงานของหน่วยความจำสำรอง (Secondary Memory) ประเภท หน่วยความจำแสง (Optical Memory) ได้อย่างถูกต้อง
21. อธิบายการทำงานของหน่วยความจำสำรอง (Secondary Memory) แบบแม่เหล็กประเภทเทปแม่เหล็ก (Magnetic Tape) ได้อย่างถูกต้อง
22. อธิบายการทำงานของหน่วยความจำสำรอง (Secondary Memory) แบบแม่เหล็กประเภท แผ่นฟลอปปี้ดิสก์ Floppy Disk ได้อย่างถูกต้อง
23. อธิบายการทำงานของหน่วยความจำสำรอง (Secondary Memory) แบบแม่เหล็กประเภท จานแม่เหล็ก (Magnetic Disk) ได้อย่างถูกต้อง
24. อธิบายการทำงานของหัวอ่าน/เขียน (Read/Write Heads) หน่วยความจำจานแม่เหล็ก (Magnetic Disk) ได้อย่างถูกต้อง
25. อธิบายการพัฒนาหน่วยความจำจานแม่เหล็ก (Magnetic Disk) จากอดีตถึงปัจจุบันได้อย่างถูกต้อง
26. อธิบายมาตรฐานการเชื่อมต่อในการส่งข้อมูลของหน่วยความจำจานแม่เหล็ก (Magnetic Disk) ได้อย่างถูกต้อง
27. อธิบายการทำงานของหน่วยความจำแคช (CACHE MEMORY) การพบข้อมูลในแคช (Cache Hit) และ การไม่พบข้อมูลในแคช (Cache Miss) ได้อย่างถูกต้อง
28. อธิบายการโอนย้ายข้อมูลระหว่างหน่วยความจำแคชกับหน่วยความจำหลักได้อย่างถูกต้อง
29. .อธิบายฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การวางข้อมูล (Placement) นำบล็อกของข้อมูลวางลงในแคชแบบการเชื่อมโยงแบบโดยตรง (Direct Mapping) ได้อย่างถูกต้อง
30. อธิบายกฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การวางข้อมูล (Placement) นำบล็อกของข้อมูลวางลงในแคชแบบการเชื่อมโยงแบบสัมพันธ์ (Associative Mapping) ได้อย่างถูกต้อง
31. อธิบายการฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การวางข้อมูล (Placement) นำบล็อกของข้อมูลวางลงในแคชแบบการเชื่อมโยงแบกลุ่มสัมพันธ์ (Set-Associative Mapping) ได้อย่างถูกต้อง

32. อธิบายการฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การแทนที่ข้อมูล (Cache Replacement Policy) บล็อกที่จะนำข้อมูลไปแทนที่ เมื่อไม่มีการพบข้อมูลในแคช แบบวิธีการใช้งานน้อยที่สุด (Least Recently Used: LRU ได้อย่างถูกต้อง
33. อธิบายการฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การแทนที่ข้อมูล (Cache Replacement Policy) บล็อกที่จะนำข้อมูลไปแทนที่ เมื่อไม่มีการพบข้อมูลในแคช แบบวิธีมาก่อนออกก่อน (First-in First-out: FIFO) ได้อย่างถูกต้อง
34. อธิบายการฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การแทนที่ข้อมูล (Cache Replacement Policy) บล็อกที่จะนำข้อมูลไปแทนที่ เมื่อไม่มีการพบข้อมูลในแคช แบบวิธีใช้งานถี่น้อยที่สุด (Least Frequently Used: LFU) ได้อย่างถูกต้อง
35. อธิบายการฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การแทนที่ข้อมูล (Cache Replacement Policy) บล็อกที่จะนำข้อมูลไปแทนที่ เมื่อไม่มีการพบข้อมูลในแคช แบบวิธีสุ่ม (Random) ได้อย่างถูกต้อง
36. อธิบายการฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) การเขียนทันที (Write-through) และการเขียนทีหลัง (Write-back) ได้อย่างถูกต้อง
37. อธิบายหลักการทำงานของหน่วยความจำเสมือน (Virtual Memory) ได้อย่างถูกต้อง
38. อธิบายข้อดีและข้อเสียของหน่วยความจำเสมือน (Virtual Memory) ได้อย่างถูกต้อง
39. อธิบายการทำงานหน่วยความจำเสมือนระบบเพจ (Paging System) ได้อย่างถูกต้อง
40. อธิบายการทำงานของระบบ Paging System แบบการทำงานปกติ (ไม่มี Page Fault) และแบบกาผิดเพจ (Page Fault) ได้อย่างถูกต้อง
41. อธิบายการคำนวนเพื่อสร้างหน่วยความจำเสมือน (Virtual Memory) ได้อย่างถูกต้อง
42. อธิบายการคำนวนเพื่อสร้างหน่วยความจำเสมือน (Virtual Memory) ระบบเพจ (Paging System) ได้อย่างถูกต้อง
43. อธิบายการทำงานของระบบเพจการแปลงที่อยู่เสมือน (Virtual Address) ไปเป็นที่อยู่จริง (Physical Address) ได้อย่างถูกต้อง
44. อธิบายการแปลงส่งแอดเดรส (Address Mapping) แบบ Dynamic Address Translation - DAT ได้อย่างถูกต้อง
45. อธิบายการบริหารการจัดการหน่วยความจำเสมือน (Virtual Memory) แบบการสับเปลี่ยนหน้า (PAGE REPLACEMENT ALGORITHMS) ได้อย่างถูกต้อง
46. อธิบายการจัดการหน่วยความจำเสมือน (Virtual Memory) แบบวิธีสับเปลี่ยนแบบมาก่อน-ออกก่อน (First-in First-out : FIFO) ได้อย่างถูกต้อง

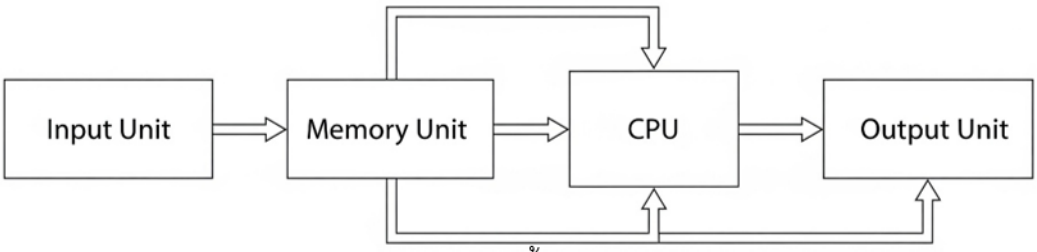
47. อธิบายการจัดการหน่วยความจำเสมือน (Virtual Memory) แบบวิธีสับเปลี่ยนแบบดีที่สุด (Optimal Page Replacement : OPT) ได้อย่างถูกต้อง
48. อธิบายการจัดการหน่วยความจำเสมือน (Virtual Memory) แบบวิธีสับเปลี่ยนแบบใช้มานานที่สุด (Least Recently Used : LRU) ได้อย่างถูกต้อง

1.หน้าที่และองค์ประกอบของหน่วยความจำ (Memory Unit)

หน่วยความจำ (Memory Unit) ทำหน้าที่เก็บชุดคำสั่งหรือข้อมูลที่รับมาจากหน่วยรับข้อมูล เพื่อเตรียมส่งให้ CPU ประมวลผล และส่งผลลัพธ์ไปหน่วยความจำหรือหน่วยแสดงผลข้อมูล

หน้าที่ของหน่วยความจำ (Memory Unit)

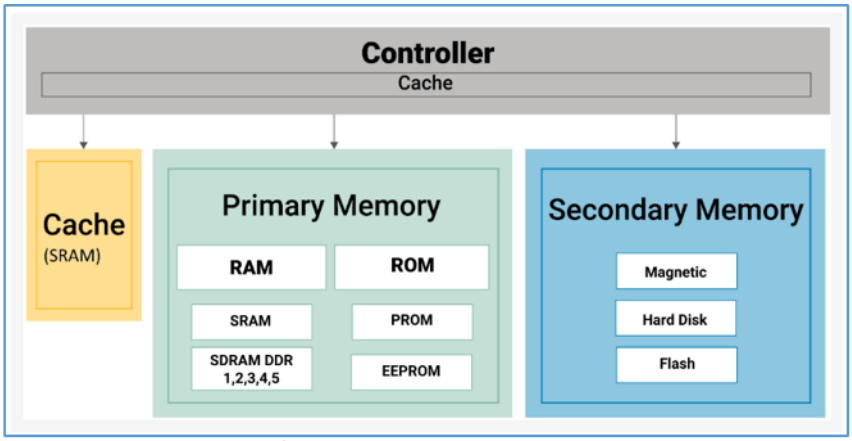
หน่วยความจำในระบบคอมพิวเตอร์ทำหน้าที่หลักในการ **เก็บชุดคำสั่ง** (Instructions) และ **ข้อมูล** (Data) ซึ่งเป็นส่วนสำคัญในการทำงานของระบบทั้งหมด



รูปที่ 7-1 แผนผังองค์ประกอบพื้นฐานของระบบคอมพิวเตอร์

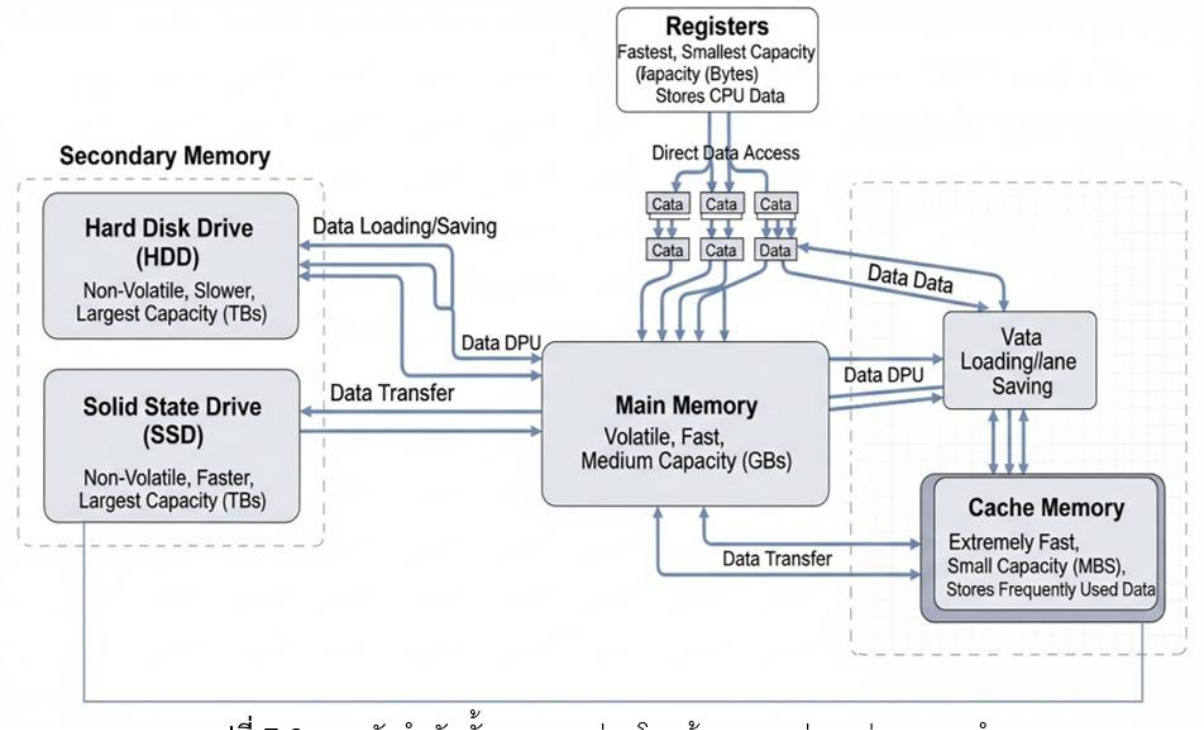
- **รับข้อมูล:** หน่วยความจำจะรับข้อมูลจาก **หน่วยรับข้อมูล (Input Unit)** เช่น ข้อมูลที่ป้อนจากคีย์บอร์ด หรือข้อมูลจากไฟล์ที่เปิดขึ้นมา
- **เตรียมส่งให้ CPU:** ข้อมูลและชุดคำสั่งเหล่านี้จะถูกจัดเก็บไว้ในหน่วยความจำเพื่อรอให้ **หน่วยประมวลผลกลาง (CPU)** เรียกใช้งาน
- **เก็บผลลัพธ์:** เมื่อ CPU ประมวลผลเสร็จแล้ว ผลลัพธ์ที่ได้จะถูกส่งกลับมาเก็บไว้ในหน่วยความจำอีกครั้ง ก่อนที่จะส่งต่อไปยัง **หน่วยแสดงผลข้อมูล (Output Unit)** เช่น หน้าจอคอมพิวเตอร์หรือเครื่องพิมพ์

2.ประเภทและองค์ประกอบของหน่วยความจำ (Memory Unit)



รูปที่ 7-2 แผนผังโครงสร้างและประเภทของหน่วยความจำในคอมพิวเตอร์

ในระบบคอมพิวเตอร์ หน่วยความจำไม่ได้มีเพียงประเภทเดียว แต่ถูกจัดลำดับชั้นตามคุณสมบัติ ได้แก่ ความเร็ว, ความจุ, และราคา เพื่อให้เกิดประสิทธิภาพสูงสุดในการทำงาน เราเรียกการจัดเรียงนี้ว่า **ลำดับชั้นหน่วยความจำ (Memory Hierarchy) ประกอบด้วยดังนี้**



รูปที่ 7-3 แผนผังลำดับชั้นและการถ่ายโอนข้อมูลระหว่างหน่วยความจำ

2.1 รีจิสเตอร์ (Register)

- **องค์ประกอบและหน้าที่:** รีจิสเตอร์เป็นหน่วยความจำขนาดเล็กที่อยู่ภายใน CPU โดยตรง ทำหน้าที่เป็นที่เก็บข้อมูลชั่วคราวสำหรับคำสั่งและข้อมูลที่ CPU กำลังประมวลผลอยู่ทันที มีความเร็วในการเข้าถึงข้อมูลสูงที่สุด
- **ตัวอย่างการทำงาน:** เมื่อ CPU ต้องการบวกเลขสองจำนวน ค่าของตัวเลขทั้งสองจะถูกนำมาเก็บไว้ในรีจิสเตอร์ก่อน จากนั้นหน่วยคำนวณและตรรกะ (ALU) ภายใน CPU จะทำการบวกและนำผลลัพธ์ไปเก็บไว้ในรีจิสเตอร์ตัวอื่น เพื่อรอส่งต่อไปยังหน่วยความจำอื่นหรือนำไปใช้ในคำสั่งถัดไป

2.2 หน่วยความจำแคช (Cache Memory)

- **องค์ประกอบและหน้าที่:** หน่วยความจำแคชเป็นหน่วยความจำความเร็วสูงที่อยู่ระหว่าง CPU กับหน่วยความจำหลัก ทำหน้าที่เก็บสำเนาของข้อมูลและคำสั่งที่คาดว่า CPU จะเรียกใช้บ่อยๆ เพื่อลดเวลาการเข้าถึงข้อมูลจากหน่วยความจำหลัก ซึ่งทำงานช้ากว่าแคชมาก
- **ตัวอย่างการทำงาน:** หาก CPU เรียกใช้ข้อมูลจากหน่วยความจำหลักเป็นครั้งแรก ข้อมูลนั้นจะถูกคัดลอกมาเก็บไว้ในแคชด้วย เมื่อ CPU เรียกใช้ข้อมูลชุดเดิมอีกครั้ง ระบบจะตรวจสอบที่แคชก่อน หากพบ (เรียกว่า **Cache Hit**) ก็จะสามารถดึงข้อมูลได้ทันทีโดยไม่ต้องไปที่หน่วยความจำหลัก ทำให้การประมวลผลเร็วขึ้นมาก

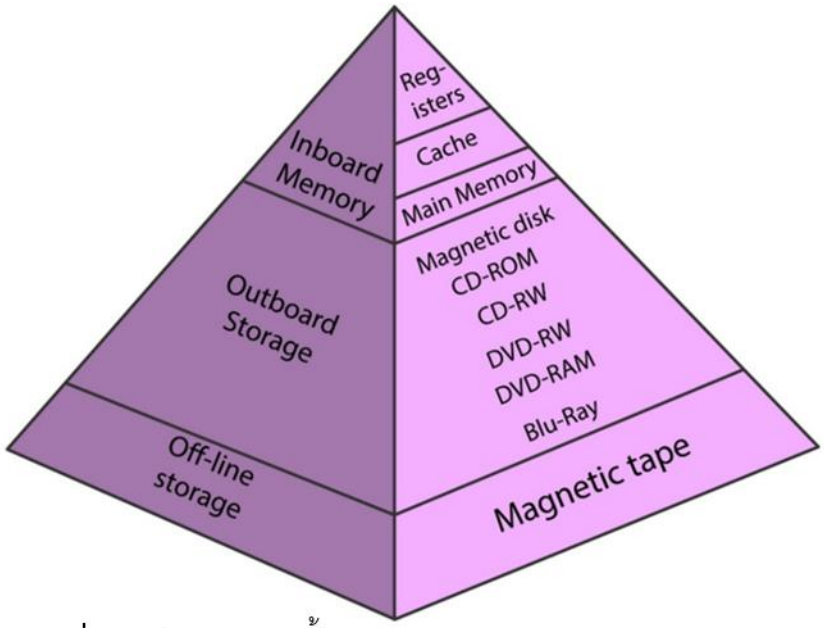
2.3 หน่วยความจำหลัก (Main Memory หรือ RAM)

- **องค์ประกอบและหน้าที่:** หน่วยความจำหลักหรือ RAM เป็นหน่วยความจำหลักของระบบ ใช้เก็บโปรแกรมและข้อมูลที่กำลังทำงานอยู่ในขณะนั้น CPU สามารถติดต่อและอ้างแอดเดรสได้โดยตรง แต่เป็นหน่วยความจำแบบระเหย (ข้อมูลจะหายไปเมื่อปิดเครื่อง)
- **ตัวอย่างการทำงานร่วม:** เมื่อนักศึกษาเปิดโปรแกรม Microsoft Word และเริ่มพิมพ์งาน โปรแกรม Word และข้อมูลที่นักศึกษากำลังพิมพ์จะถูกโหลดจากฮาร์ดดิสก์ (หน่วยความจำสำรอง) เข้ามาเก็บไว้ใน RAM เพื่อให้ CPU เข้าถึงและประมวลผลได้อย่างรวดเร็ว

2.4 หน่วยความจำสำรอง (Secondary Memory)

- **องค์ประกอบและหน้าที่:** หน่วยความจำสำรองเป็นที่เก็บข้อมูลแบบถาวร มีความจุสูงและราคาถูกที่สุด ทำหน้าที่เก็บข้อมูลและโปรแกรมต่างๆ ที่ยังไม่ได้ถูกเรียกใช้งาน
- **ตัวอย่างการทำงานร่วม:** เมื่อนักศึกษาปิดโปรแกรม Word ระบบปฏิบัติการจะแจ้งเตือนให้บันทึกไฟล์ ซึ่งข้อมูลที่อยู่ใน RAM จะถูกเขียนลงไปในฮาร์ดดิสก์หรือ SSD (หน่วยความจำสำรอง) เพื่อให้ข้อมูลนั้นไม่สูญหายและสามารถเปิดใช้งานได้อีกครั้งในอนาคต

2.5 ความเร็วในการเข้าถึงข้อมูลโดย CPU (CPU Data Access Speed)



รูปที่ 7-4 พีระมิดลำดับชั้นของหน่วยความจำและการแบ่งกลุ่มตามลักษณะการติดตั้ง

ลำดับชั้นหน่วยความจำคือการจัดเรียงหน่วยความจำต่างๆ ในระบบคอมพิวเตอร์ตามคุณสมบัติ ได้แก่ ความเร็ว ความจุ และราคา โดยมีเป้าหมายเพื่อสร้างสมดุลระหว่างประสิทธิภาพกับต้นทุน หน่วยความจำที่อยู่ใกล้ CPU มากที่สุดจะมีความเร็วสูงและราคาแพง แต่มีความจุน้อย ส่วนหน่วยความจำที่อยู่ไกลออกไปจะมีราคาถูกลงและมีความจุสูงขึ้น แต่ความเร็วในการเข้าถึงจะลดลง

2.5.1. รีจิสเตอร์ (Register)

- **คำอธิบาย:** เปรียบเสมือน **เขียงและมีด** ที่อยู่ในมือของพ่อครัว เป็นอุปกรณ์ที่เข้าถึงได้เร็วที่สุดสำหรับเตรียมวัตถุดิบชิ้นเล็กๆ เพื่อปรุงอาหารในทันที ข้อมูลที่อยู่ในรีจิสเตอร์เป็นข้อมูลที่ CPU กำลังประมวลผลอยู่ ณ ขณะนั้น ซึ่งต้องการความเร็วสูงสุด
    - **คุณสมบัติ:** **ความเร็ว:** เร็วที่สุด **ความจุ:** น้อยที่สุด **ราคา:** แพงที่สุด

2.5.2. หน่วยความจำแคช (Cache Memory)

- **คำอธิบาย:** เปรียบเสมือน **ชั้นวางเครื่องปรุงและวัตถุดิบ** ที่อยู่ติดกับเตาแก๊ส เป็นที่เก็บวัตถุดิบที่ใช้บ่อยๆ หรือเพิ่งใช้ไป เพื่อให้พ่อครัวหยิบมาใช้ได้ทันทีโดยไม่ต้องเดินไปที่ตู้เก็บของหรือตู้เย็นบ่อยๆ
- **การทำงาน:** เมื่อ CPU ต้องการข้อมูล จะตรวจสอบที่หน่วยความจำแคชก่อน หากมีข้อมูลที่ต้องการอยู่ (เรียกว่า **Cache Hit**) จะสามารถนำไปประมวลผลได้ทันทีโดยไม่ต้องเข้าถึงหน่วยความจำหลัก
    - **คุณสมบัติ:** **ความเร็ว:** เร็วรองลงมาจากรีจิสเตอร์
        - **ความจุ:** มากกว่ารีจิสเตอร์ แต่ยังน้อยกว่าหน่วยความจำหลัก
        - **ราคา:** แพงกว่าหน่วยความจำหลัก

2.5.3. หน่วยความจำหลัก (Main Memory หรือ RAM)

- **คำอธิบาย:** เปรียบเสมือน **ตู้เย็น** ที่เก็บวัตถุดิบหลักทั้งหมดสำหรับทำอาหาร พ่อครัวต้องเดินไปที่ตู้เย็นเพื่อหยิบวัตถุดิบ แต่ใช้เวลามากกว่าการหยิบจากชั้นวางที่อยู่ติดกับเตา
- **การทำงาน:** ใช้เก็บโปรแกรมและข้อมูลที่กำลังทำงานอยู่ เมื่อเปิดโปรแกรม โปรแกรมจะถูกโหลดจากหน่วยความจำสำรองมาเก็บไว้ใน RAM เพื่อให้ CPU เข้าถึงได้เร็วกว่าการอ่านจากฮาร์ดดิสก์โดยตรง
    - **คุณสมบัติ:** **ความเร็ว:** เร็วกว่าหน่วยความจำสำรอง แต่ช้ากว่าแคช
      ความจุ: มากกว่าแคช **ราคา:** ถูกกว่าแคช

2.5.4. หน่วยความจำสำรอง (Secondary Memory)

- **คำอธิบาย:** เปรียบเสมือน **ห้องเก็บของ** หรือ **โกดังขนาดใหญ่** ที่ใช้เก็บวัตถุดิบทั้งหมดที่ไม่ได้ใช้ในทันที เป็นที่เก็บข้อมูลและโปรแกรมแบบถาวร มีความจุสูงที่สุดแต่การเข้าถึงช้าที่สุด
- **การทำงาน:** เมื่อปิดโปรแกรม ข้อมูลจะถูกบันทึกจาก RAM ลงในฮาร์ดดิสก์หรือ SSD เพื่อเก็บข้อมูลไว้ใช้งานในครั้งถัดไป

  คุณสมบัติ: **ความเร็ว:** ช้าที่สุด
  ความจุ: มากที่สุด
  ราคา: ถูกที่สุด

การจัดลำดับชั้นแบบนี้ช่วยให้ระบบคอมพิวเตอร์สามารถจัดการข้อมูลได้อย่างมีประสิทธิภาพ ทำให้ CPU ไม่ต้องรอข้อมูลนาน และสามารถทำงานได้อย่างต่อเนื่อง

3.คุณลักษณะของหน่วยความจำ (Memory Characteristics)

หน่วยความจำแต่ละประเภทมีคุณสมบัติแตกต่างกัน เราสามารถจำแนกตามคุณลักษณะต่างๆ ได้ดังนี้:

3.1. ตำแหน่ง (Location)

- **หน่วยความจำภายใน (Internal Memory):** เป็นหน่วยความจำที่ CPU สามารถเข้าถึงได้โดยตรง โดยผ่านระบบบัส (System Bus) ได้แก่
    - **รีจิสเตอร์ (Register):** อยู่ใน CPU โดยตรง
    - **หน่วยความจำแคช (Cache Memory):** อยู่ระหว่าง CPU กับ RAM
    - **หน่วยความจำหลัก (RAM, ROM):** เป็นหน่วยความจำหลักของระบบ
    - **ตัวอย่างการทำงาน:** เมื่อ CPU ต้องการข้อมูลจาก RAM มันจะส่งสัญญาณผ่าน Address Bus เพื่อระบุตำแหน่งของข้อมูล และรับข้อมูลกลับมาผ่าน Data Bus โดยตรง
- **หน่วยความจำภายนอก (External Memory):** เป็นหน่วยความจำสำรอง (Secondary Memory) ที่ CPU ไม่สามารถเข้าถึงได้โดยตรง แต่ต้องผ่านหน่วยควบคุม I/O (I/O Controller) ซึ่งจะทำหน้าที่ในการจัดการการถ่ายโอนข้อมูลระหว่างหน่วยความจำภายนอกกับหน่วยความจำหลัก
    - **ตัวอย่างการทำงาน:** เมื่อเปิดไฟล์งานจากฮาร์ดดิสก์ ระบบปฏิบัติการจะสั่งให้หน่วยควบคุม I/O ทำการอ่านข้อมูลจากฮาร์ดดิสก์แล้วนำไปเก็บไว้ใน RAM จากนั้น CPU จึงจะสามารถเข้าถึงข้อมูลนั้นได้

3.2. ความจุ (Capacity)

- **คำอธิบาย:** คือปริมาณข้อมูลที่หน่วยความจำสามารถเก็บได้ หน่วยวัดพื้นฐานคือ **บิต (Bit)** ซึ่งแทนด้วย 0 หรือ 1 และ 8 บิตเท่ากับ **1 ไบต์ (Byte)**
- **ตัวอย่าง:** หน่วยความจำ RAM ขนาด 8 GB (กิกะไบต์) หมายความว่าสามารถเก็บข้อมูลได้ 8 x 10^9 ไบต์ ซึ่งมากกว่าฮาร์ดดิสก์ขนาด 1 TB (เทระไบต์) ที่เก็บได้ 1 x 10^12 ไบต์

3.3. หน่วยของการถ่ายโอนข้อมูล (Unit of Transfer)

- **คำอธิบาย:** คือขนาดของข้อมูลที่สามารถถ่ายโอนได้ในแต่ละครั้ง
    - **เวิร์ด (Word):** มีขนาดเท่ากับความยาวของชุดคำสั่ง เช่น CPU 32-bit จะมี Word size เท่ากับ 32 บิต

- **บล็อก (Block):** เป็นกลุ่มของเวิร์ดหลายๆ เวิร์ด ใช้ในการถ่ายโอนข้อมูลจำนวนมาก เช่น การถ่ายโอนข้อมูลระหว่างฮาร์ดดิสก์กับ RAM

- **ตัวอย่าง:** เมื่อ CPU อ่านข้อมูลจาก RAM จะอ่านข้อมูลทีละ Word แต่เมื่อมีการดึงข้อมูลจากฮาร์ดดิสก์มาเก็บใน RAM ระบบจะดึงข้อมูลมาทีละ Block ซึ่งประกอบด้วยหลาย Word เพื่อเพิ่มประสิทธิภาพ



รูปที่ 7- 5 โครงสร้างกายภาพของฮาร์ดดิสก์ไดรฟ์

3.4. วิธีการเข้าถึงข้อมูล (Access Method)

- **การเข้าถึงแบบตามลำดับ (Sequential Access):** เข้าถึงข้อมูลตามลำดับก่อน-หลังเท่านั้น เช่น **เทปแม่เหล็ก (Magnetic Tape)** หากต้องการข้อมูลที่อยู่ท้ายๆ ของเทป จะต้องเลื่อนเทปผ่านข้อมูลที่อยู่ก่อนหน้าทั้งหมด

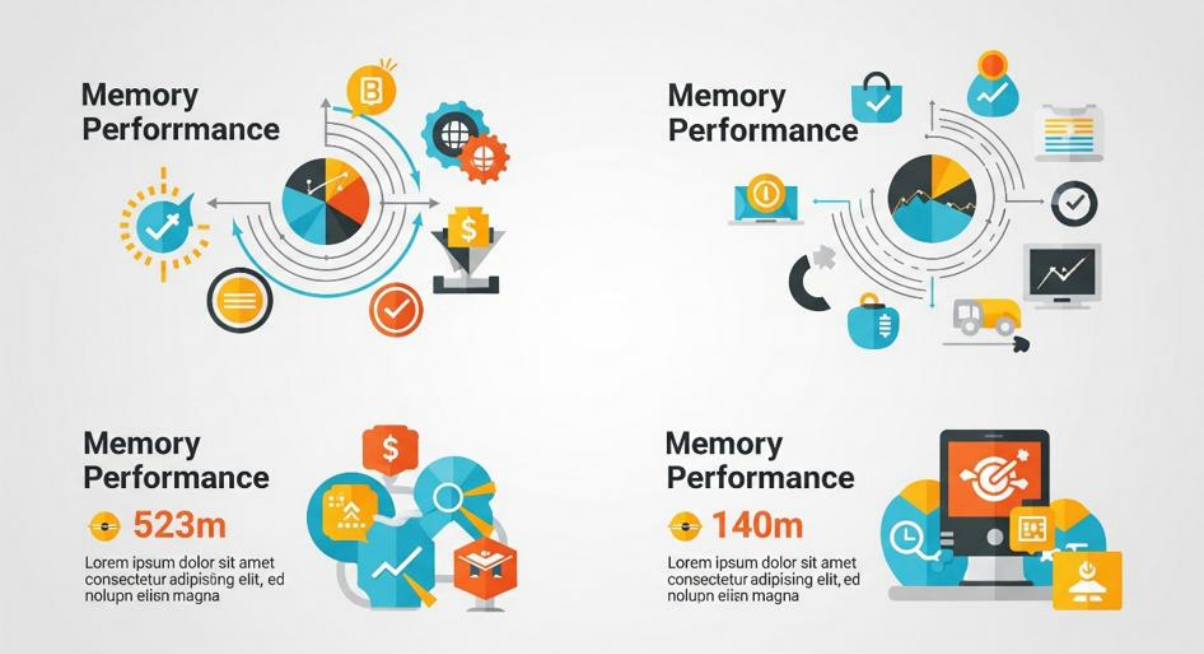
- **การเข้าถึงโดยตรง (Direct Access):** สามารถเข้าถึงข้อมูลในตำแหน่งที่ต้องการได้โดยประมาณ แต่ยังต้องมีการค้นหาอีกเล็กน้อย เช่น **ฮาร์ดดิสก์ (Hard Disk)** เมื่อต้องการเข้าถึงข้อมูล ระบบจะสั่งให้หัวอ่านเลื่อนไปยังแทร็ก (Track) ที่ถูกต้องก่อน จากนั้นจึงจะอ่านข้อมูลในแทร็กนั้นตามลำดับ

- **การเข้าถึงแบบสุ่ม (Random Access):** สามารถเข้าถึงข้อมูลในตำแหน่งที่ต้องการได้โดยตรงทันที ไม่ต้องเลื่อนผ่านข้อมูลอื่น เช่น **หน่วยความจำหลัก (RAM)** เมื่อ CPU ต้องการข้อมูลในแอดเดรสที่ระบุ จะสามารถเข้าถึงข้อมูลนั้นได้ทันที

- **การเข้าถึงแบบมีความเกี่ยวข้องกัน (Associative Access):** เป็นการเข้าถึงแบบสุ่มที่พิเศษ โดยการเข้าถึงไม่ได้ขึ้นอยู่กับที่อยู่ แต่ขึ้นอยู่กับเนื้อหาของข้อมูลที่ใช้เปรียบเทียบ เช่น **หน่วยความจำแคช (Cache Memory)**

3.5. ประสิทธิภาพหน่วยความจำ (Performance)

ประสิทธิภาพของหน่วยความจำวัดได้จากปัจจัยต่างๆ ที่เกี่ยวข้องกับความเร็วในการเข้าถึงและถ่ายโอนข้อมูล ซึ่งเราจะพิจารณาจาก 4 คุณลักษณะหลักดังนี้



รูปที่ 7- 6 ตัวชี้วัดและกราฟวิเคราะห์ประสิทธิภาพของหน่วยความจำ

3.5.1. เวลาที่ใช้ในการเข้าถึงข้อมูล (Access Time)

- **คำอธิบาย:** คือระยะเวลาที่เริ่มตั้งแต่การส่งคำสั่งเพื่อเข้าถึงตำแหน่งหน่วยความจำ ไปจนถึงเวลาที่ได้รับข้อมูลที่ต้องการออกมา พูดง่ายๆ คือ **เวลาที่ใช้ในการอ่านข้อมูลเพียงอย่างเดียว**
- **ตัวอย่าง:** ในหน่วยความจำหลัก (RAM) Access Time คือเวลาที่ CPU ใช้ในการระบุตำแหน่งข้อมูลที่ต้องการใน RAM แล้วดึงข้อมูลนั้นออกมา ยิ่ง Access Time น้อยเท่าไหร่ หน่วยความจำก็ยิ่งทำงานได้เร็วเท่านั้น

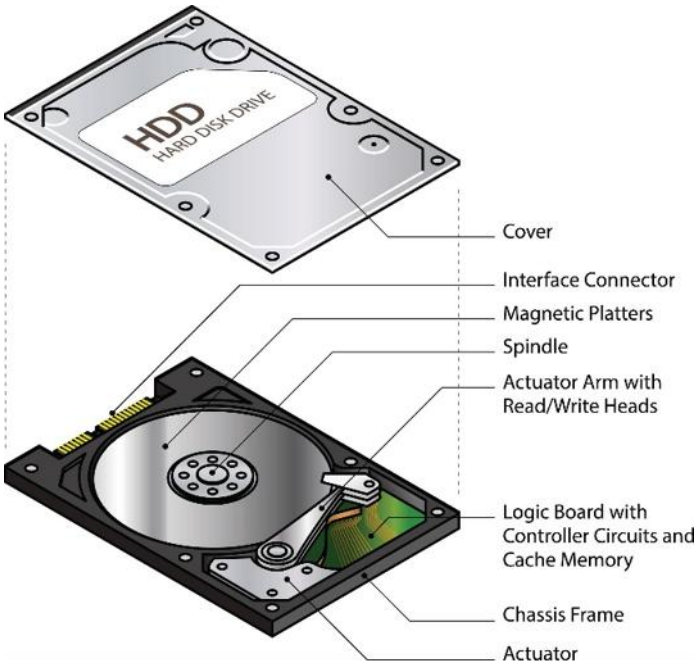
3.5.2. เวลาวงรอบหน่วยความจำ (Cycle Time)

- **คำอธิบาย:** คือเวลารวมทั้งหมดตั้งแต่เริ่มเข้าถึงหน่วยความจำ ไปจนถึงเวลาที่หน่วยความจำพร้อมที่จะรับคำสั่งถัดไป Cycle Time จะรวมเอา Access Time และเวลาเพิ่มเติมที่ใช้ในการเตรียมตัวเพื่อรับคำสั่งใหม่เข้าไปด้วย
- **ตัวอย่าง:** หลังจาก CPU อ่านข้อมูลจาก RAM เสร็จแล้ว RAM จะต้องใช้เวลาอีกเล็กน้อยในการรีเฟรชหรือเตรียมตัวก่อนที่จะสามารถรับคำสั่งใหม่ได้ Cycle Time จึงมีค่ามากกว่า Access Time เสมอ

- **สมการ:** Cycle Time = Access Time + Recovery Time

3.5.3. เวลาแฝง (Latency Time)

- **คำอธิบาย:** คือเวลาที่ใช้ในการเข้าถึง **เวิร์ด (Word)** หรือบล็อกข้อมูล **ครั้งแรก** ในการเข้าถึงแบบตามลำดับหรือแบบโดยตรง (Sequential/Direct Access)
- **ตัวอย่าง:** ในฮาร์ดดิสก์ (HDD) หัวอ่านต้องใช้เวลาในการหมุนจานแม่เหล็กจนกระทั่งตำแหน่งของข้อมูลที่ต้องการมาอยู่ใต้หัวอ่าน เวลานี้เรียกว่าเวลาแฝง ยิ่ง Latency Time ต่ำ การเข้าถึงข้อมูลก็ยิ่งเร็วขึ้น



รูปที่ 7- 7 ส่วนประกอบกายภาพและโครงสร้างภายในของฮาร์ดดิสก์ (HDD)

3.5 อัตราการรับส่งข้อมูล (Bandwidth)

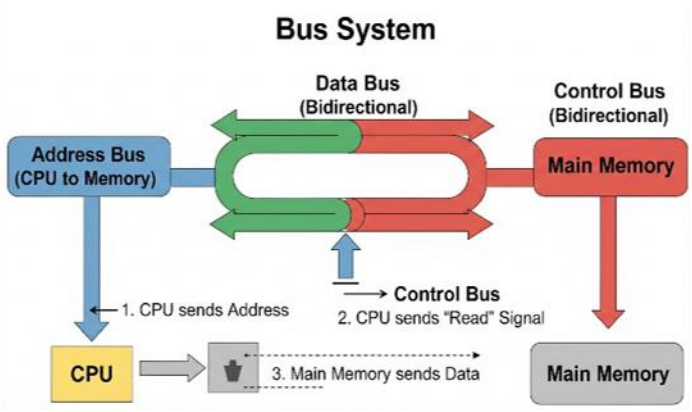
- **คำอธิบาย:** คือปริมาณข้อมูลที่หน่วยความจำสามารถถ่ายโอนได้ในช่วงระยะเวลาหนึ่ง มีหน่วยเป็นไบต์ต่อวินาที (Bytes per Second) ยิ่ง Bandwidth สูงเท่าไหร่ ก็ยิ่งส่งข้อมูลได้เร็วมากขึ้นเท่านั้น
- **ตัวอย่าง:** ถ้าเปรียบเทียบหน่วยความจำเหมือนกับท่อน้ำ Bandwidth ก็คือขนาดของท่อ ถ้าท่อใหญ่ก็สามารถส่งน้ำ (ข้อมูล) ได้ในปริมาณที่มากขึ้นในเวลาที่เท่ากัน โดย Bandwidth ของ RAM จะมีค่าสูงกว่า Bandwidth ของฮาร์ดดิสก์มาก

3.6 ลักษณะทางกายภาพ (Physical Type)

- **หน่วยความจำลบเลือน (Volatile Memory):** ข้อมูลจะสูญหายทันทีเมื่อไม่มีกระแสไฟฟ้าหล่อเลี้ยง เช่น **RAM** และ **Cache** ซึ่งเป็นเหตุผลว่าทำไมเราต้องบันทึกงานก่อนปิดเครื่อง
- **หน่วยความจำไม่ลบเลือน (Non-Volatile Memory):** ข้อมูลจะยังคงอยู่แม้ไม่มีกระแสไฟฟ้าหล่อเลี้ยง เช่น **ROM** และ **ฮาร์ดดิสก์ (HDD)** จึงเหมาะสำหรับเก็บโปรแกรมและข้อมูลแบบถาวร

4.การติดต่อระหว่าง CPU กับหน่วยความจำหลัก (Main Memory)

การสื่อสารระหว่าง CPU และหน่วยความจำหลัก (Main Memory) เกิดขึ้นผ่านระบบที่เรียกว่า **ระบบบัส (Bus System)** ซึ่งเป็นช่องทางในการรับส่งข้อมูล, แอดเดรส, และสัญญาณควบคุม โดยมีองค์ประกอบหลักดังนี้



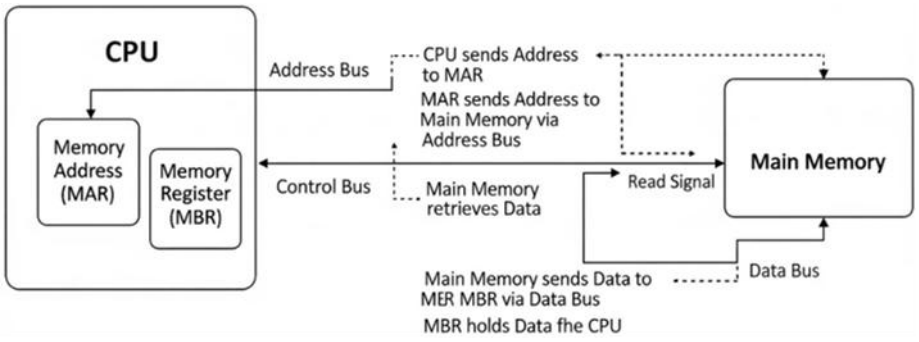
รูปที่ 7- 8 โครงสร้างระบบบัสและขั้นตอนการรับ-ส่งข้อมูลในระบบคอมพิวเตอร์

Address Bus: ใช้สำหรับส่งค่าแอดเดรส (ตำแหน่งที่อยู่) ของข้อมูลหรือคำสั่งในหน่วยความจำ

Data Bus: ใช้สำหรับรับส่งข้อมูลจริงระหว่าง CPU และหน่วยความจำ

Control Bus: ใช้สำหรับส่งสัญญาณควบคุมการทำงาน เช่น สัญญาณอ่าน (Read) หรือ สัญญาณเขียน (Write)

ขั้นตอนการอ่านข้อมูลจากหน่วยความจำ

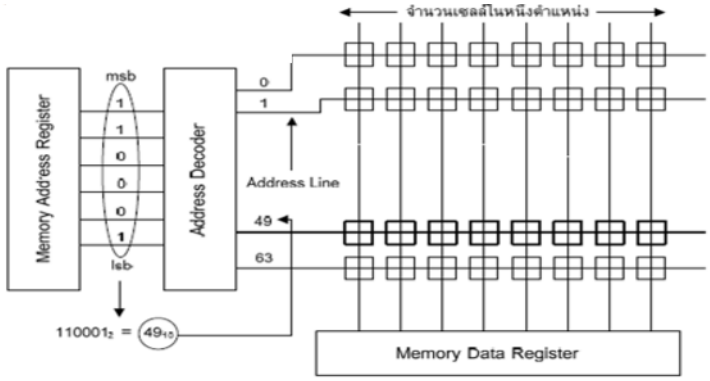


รูปที่ 7- 9 กลไกการทำงานระหว่างรีจิสเตอร์ MAR/MBR และหน่วยความจำหลัก

เมื่อ CPU ต้องการอ่านข้อมูลจากหน่วยความจำหลัก จะมีขั้นตอนการทำงานดังนี้

1. **การระบุตำแหน่ง (Address Calculation):** CPU จะคำนวณหาตำแหน่งของข้อมูลที่ต้องการในหน่วยความจำ และนำค่าแอดเดรสที่ได้ไปเก็บไว้ใน **Memory Address Register (MAR)** ซึ่งเป็นรีจิสเตอร์ที่ใช้เก็บแอดเดรสของหน่วยความจำ
    - **ตัวอย่าง:** สมมติว่า CPU ต้องการอ่านข้อมูลที่ตำแหน่งแอดเดรส 1001 ค่า 1001 จะถูกนำไปเก็บใน MAR
2. **การส่งค่าแอดเดรส:** CPU จะให้ **หน่วยควบคุม (Control Unit)** สั่งให้ MAR ส่งค่าแอดเดรสออกไปทาง **Address Bus** เพื่อระบุตำแหน่งของข้อมูลที่ต้องการในหน่วยความจำ

3. **การส่งสัญญาณควบคุม:** พร้อมกันนั้น หน่วยควบคุมจะส่งสัญญาณ **Read (อ่าน)** ออกไปทาง **Control Bus** เพื่อบอกให้หน่วยความจำทราบว่า CPU ต้องการอ่านข้อมูลจากตำแหน่งที่ระบุ



รูปที่ 7- 10 แผนผังการถอดรหัสเลขที่อยู่และการเข้าถึงเซลล์ข้อมูลในหน่วยความจำ

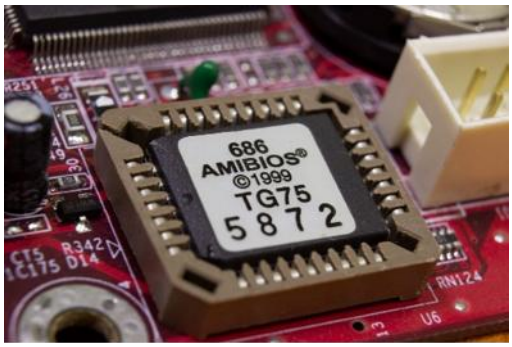
การรับข้อมูล: เมื่อหน่วยความจำได้รับสัญญาณ Read และตำแหน่งแอดเดรสแล้ว ก็จะดึงข้อมูลจากตำแหน่งนั้นมาเก็บไว้ใน **Memory Buffer Register (MBR)** จากนั้นจะส่งข้อมูลนั้นกลับมาทาง **Data Bus**

ตัวอย่าง: หน่วยความจำจะส่งข้อมูล A5 ที่เก็บอยู่ที่แอดเดรส 1001 กลับมายัง MBR ผ่าน Data Bus

4. **การนำข้อมูลไปใช้:** ข้อมูลที่อยู่ใน MBR จะถูกย้ายไปยังรีจิสเตอร์อื่นๆ ภายใน CPU เพื่อนำไปประมวลผลต่อไป

5.ประเภทการใช้งานของหน่วยความจำหลัก (Main Memory)

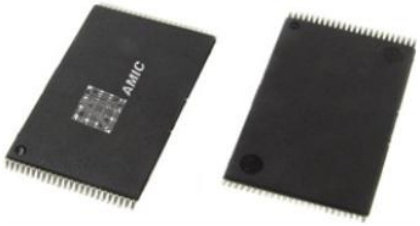
หน่วยความจำหลักเป็นหน่วยความจำที่ CPU สามารถเข้าถึงได้โดยตรง มี 2 ประเภทหลักคือ ROM และ RAM ซึ่งแต่ละประเภทมีหน้าที่และคุณสมบัติที่แตกต่างกัน



รูปที่ 7- 11 ลักษณะทางกายภาพของชิปหน่วยความจำ ROM (BIOS)

5.1. ROM (Read Only Memory)

คำอธิบาย: ROM คือหน่วยความจำที่สามารถอ่านได้อย่างเดียว ข้อมูลที่ถูกบันทึกไว้จะ **ไม่ลบเลือน (Non-Volatile)** แม้ไม่มีกระแสไฟฟ้าหล่อเลี้ยงเหมาะสำหรับเก็บโปรแกรมพื้นฐานที่ใช้ในการเริ่มทำงานของคอมพิวเตอร์ เช่น **BIOS** (Basic Input / Output System) ที่ใช้ในการบูตเครื่อง **<u>จากภาพคือชิป BIOS</u>** ซึ่งเป็น ROM ชนิดหนึ่งที่มักจะติดตั้งอยู่บนเมนบอร์ดของคอมพิวเตอร์ ทำหน้าที่เก็บชุดคำสั่งพื้นฐานที่จำเป็นต่อการบูตเครื่อง โดยข้อมูลในชิปนี้จะไม่หายไปแม้จะปิดเครื่องแล้วก็ตาม

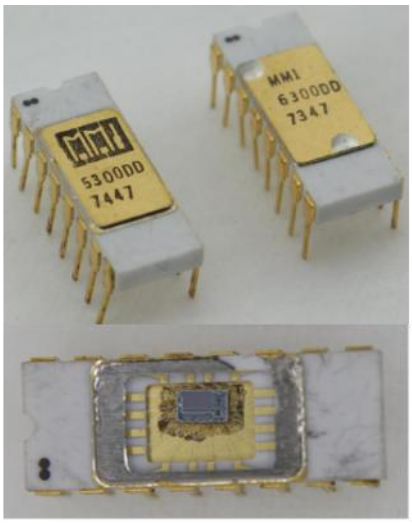


รูปที่ 7- 12 ลักษณะภายนอกของชิปหน่วยความจำสารกึ่งตัวนำ

<u>5.2ประเภทของ ROM:</u>

5.2.1.MROM (Mask ROM): ข้อมูลถูกบันทึกมาตั้งแต่การผลิต ไม่สามารถแก้ไขได้

5.2.2.PROM (Programmable ROM): ผู้ใช้สามารถบันทึกข้อมูลได้เพียงครั้งเดียว

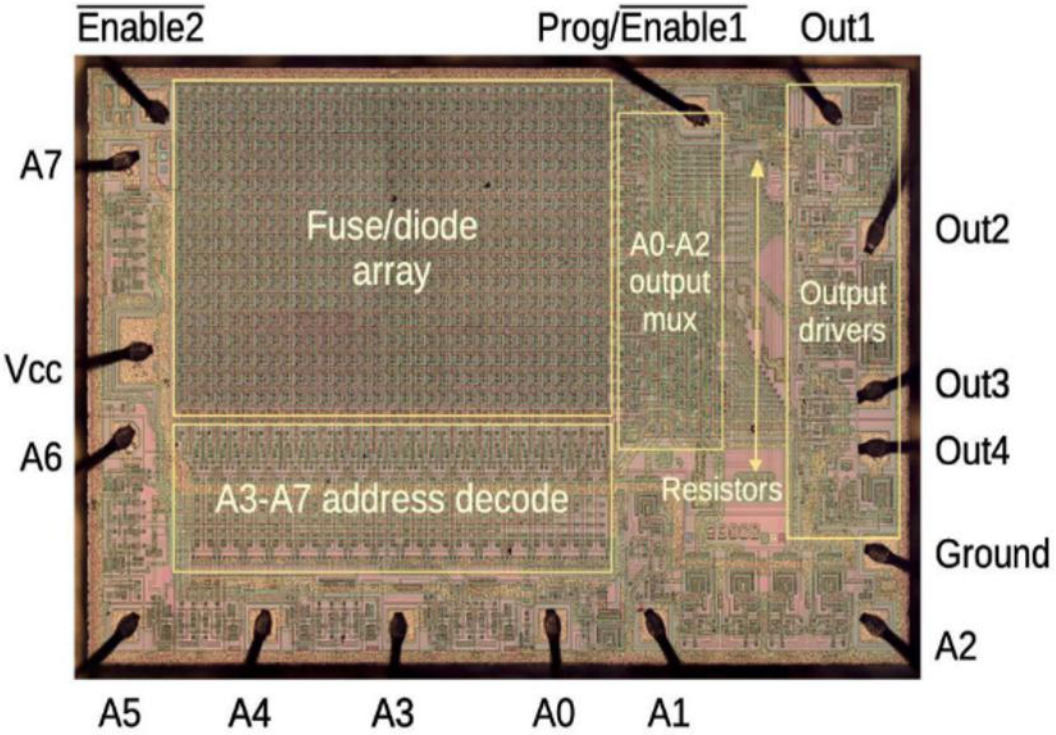


รูปที่ 7- 13 โครงสร้างภายในและชิปหน่วยความจำแบบเซรามิก



รูปที่ 7- 14 เครื่องเขียนโปรแกรมลงในหน่วยความจำ PROM

Inside the chip *The diagram below shows the main parts of the chip, with the pins labeled. The chip stores 1024 bits as 256 4-bit words. The 8 address lines A0-A7 select one of the 256 words, and the bits are output on pins Out1-Out4. The Program pin is used to store data in the chip by blowing fuses. The Vcc and ground pins power the chip*



รูปที่ 7- 15 แผนผังโครงสร้างวงจรภายในและส่วนประกอบของชิปหน่วยความจำ

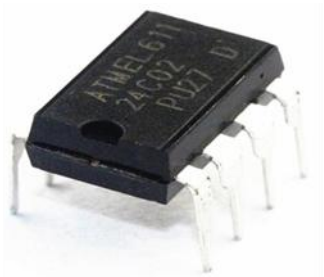
แผนรูปภาพแสดงส่วนประกอบหลักของชิป พร้อมกับป้ายกำกับขา (pins) ต่างๆ ชิปนี้สามารถจัดเก็บข้อมูลได้ 1,024 บิต โดยแบ่งเป็นคำ (words) ขนาด 4 บิต จำนวน 256 คำ ขาแอดเดรส A0-A7 จำนวน 8 ขา ใช้สำหรับเลือกคำข้อมูล (words) ใดคำหนึ่งจากทั้งหมด 256 คำ และบิตข้อมูลจะถูกส่งออกทางขา Out1-Out4 ขา Program ใช้สำหรับบันทึกข้อมูลลงในชิปโดยการเป่าฟิวส์ (blowing fuses) ส่วนขา Vcc และ Ground ใช้สำหรับจ่ายพลังงานให้กับชิป

5.3 EPROM (Erasable PROM): เมื่อเขียนข้อมูลผ่านเครื่องเขียนแล้ว จากนั้นถ้าต้องการล้างข้อมูลสามารถลบข้อมูลได้ด้วยรังสีอัลตราไวโอเลต



รูปที่ 7- 16 อุปกรณ์ลบข้อมูลและบันทึกข้อมูลสำหรับหน่วยความจำ EPROM

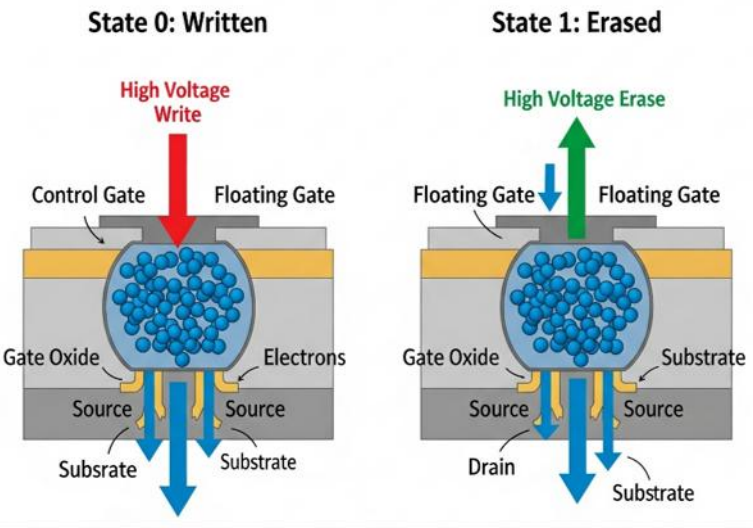
5.4.**EEPROM (Electrically Erasable Programmable Read-only Memory)**



รูปที่ 7- 17 ลักษณะทางกายภาพของชิปหน่วยความจำ EEPROM ขนาดเล็ก

เป็นหน่วยความจำประเภทหนึ่งที่สามารถเขียน, อ่าน, และลบข้อมูลได้ด้วยสัญญาณไฟฟ้า และข้อมูลจะยังคงอยู่แม้ไม่มีไฟเลี้ยง

5.4.1 หลักการทำงานและการเก็บข้อมูล: EEPROM ใช้หลักการของ Floating Gate Transistor ในการเก็บข้อมูล โดยแต่ละบิตข้อมูลจะถูกเก็บไว้ในเซลล์หน่วยความจำที่ประกอบด้วยทรานซิสเตอร์แบบพิเศษ เมื่อต้องการเขียนข้อมูลจะใช้แรงดันไฟฟ้าสูงเพื่อ "ขัง" อิเล็กตรอนไว้ในบริเวณที่เรียกว่า Floating Gate ซึ่งทำให้เซลล์มีสถานะเป็น 0 ส่วนการลบข้อมูลจะใช้แรงดันไฟฟ้าสูงในทิศทางตรงกันข้ามเพื่อ "ปล่อย" อิเล็กตรอนออกจาก Floating Gate ทำให้เซลล์กลับสู่สถานะเดิมเป็น 1 .



รูปที่ 7- 18 หลักการทำงานของ Floating Gate Transistor ในหน่วยความจำ EEPROM

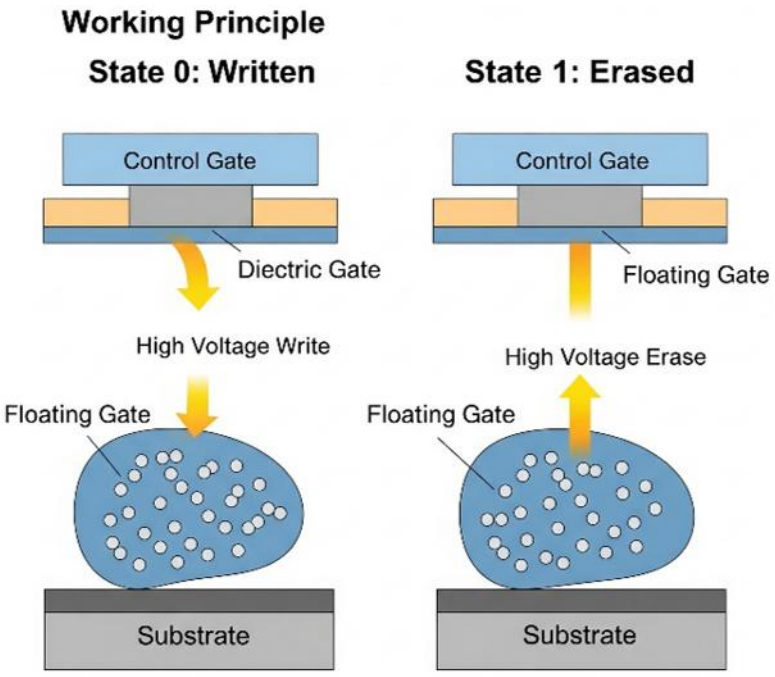
5.4.2 การพัฒนาและคุณสมบัติ

การพัฒนาและผลิต: EEPROM เริ่มมีการพัฒนาขึ้นในช่วงปี 1970s และเริ่มมีการใช้งานเชิงพาณิชย์ในช่วง **ต้นทศวรรษ 1980s** เพื่อมาทดแทน EPROM ที่ต้องลบข้อมูลด้วยแสง UV

5.4.3 ความจุ: EEPROM มีความจุค่อนข้างจำกัดเมื่อเทียบกับหน่วยความจำประเภทอื่น ๆ โดยทั่วไปมีขนาดตั้งแต่ไม่กี่กิโลบิตไปจนถึงหลายสิบเมกะบิต แต่ในปัจจุบันมีการใช้ **Flash Memory** (ซึ่งเป็น EEPROM ชนิดหนึ่ง) ที่มีความจุสูงกว่ามากและได้รับความนิยมมากกว่า

ขนาดหน่วยความจำ: มีขนาดค่อนข้างเล็ก โดยทั่วไปจะอยู่ในช่วง **ไม่กี่ร้อยไบต์ไปจนถึงหลายกิโลไบต์ (KB)** เช่น 256 ไบต์, 512 ไบต์, 1 KB, 2 KB, 4 KB, 8 KB หรือใหญ่กว่านั้นในบางรุ่น

ความเร็ว: EEPROM มีความเร็วในการอ่านข้อมูลที่ค่อนข้างเร็ว (ในระดับนาโนวินาที) แต่มีความเร็วในการเขียนและลบข้อมูลที่ช้ากว่ามาก (ในระดับมิลลิวินาที) เนื่องจากต้องใช้กระบวนการทางไฟฟ้าที่ซับซ้อน



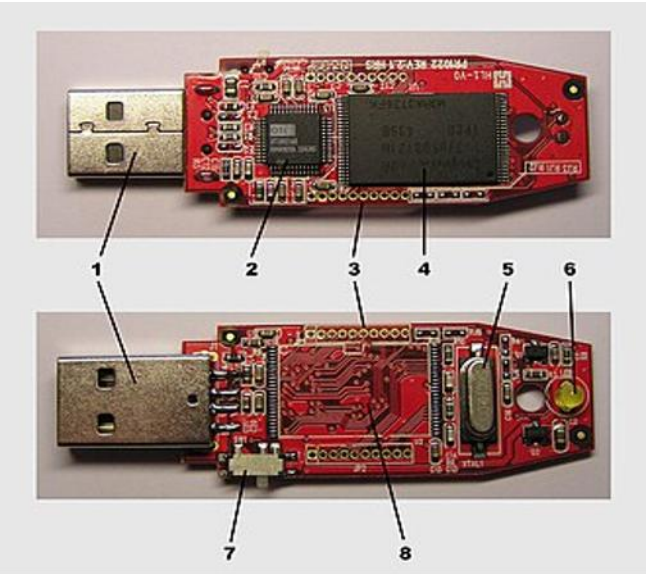
รูปที่ 7- 19 กลไกการเคลื่อนที่ของประจุไฟฟ้าในชั้น Floating Gate ขณะเขียนและล้างข้อมูล

5.4.4 การลบข้อมูลและอุปกรณ์ที่ใช้งาน

การลบข้อมูล: EEPROM สามารถลบข้อมูลทีละไบต์ (byte) ได้ ทำให้สามารถแก้ไขข้อมูลบางส่วนโดยไม่ต้องลบข้อมูลทั้งหมดแตกต่างจาก Flash Memory ที่ต้องลบข้อมูลเป็นบล็อกใหญ่ ๆ EEPROM ถูกออกแบบมาเพื่อจัดเก็บข้อมูลขนาดเล็กที่จำเป็นต้องเปลี่ยนแปลงได้เป็นครั้งคราวและต้องคงอยู่ถาวร มักใช้ในอุปกรณ์ดังนี้:

- **5.4.4.1 ไมโครคอนโทรลเลอร์ (Microcontrollers):** ใช้เก็บข้อมูลการตั้งค่า (Configuration) ของอุปกรณ์ เช่น บอร์ด Arduino หรือชิปสำหรับควบคุมเครื่องใช้ไฟฟ้า
- **5.4.4.2 อุปกรณ์อิเล็กทรอนิกส์:** ใช้เก็บค่าการปรับตั้งจากผู้ใช้งาน เช่น ระดับเสียงและช่องสถานีที่ตั้งไว้บนโทรทัศน์ หรือข้อมูลการตั้งค่าในวิทยุติดรถยนต์
- **5.4.4.3 BIOS/UEFI Chips:** ชิปบนเมนบอร์ดคอมพิวเตอร์ที่เก็บข้อมูลการตั้งค่าฮาร์ดแวร์พื้นฐาน ซึ่งสามารถอัปเดตหรือแก้ไขได้

5.5.Flash Memory: เป็นหน่วยความจำที่ได้รับความนิยมสามารถลบและเขียนข้อมูลได้โดยใช้ไฟฟ้า และมีความเร็วสูง แฟลชไดรฟ์ที่ใช้เทคโนโลยียูเอสบี 2.0 นั้นถูกพัฒนาขึ้นเป็นครั้งแรกในช่วงปี ค.ศ. 2000 ก่อนที่แฟลชไดรฟ์ชนิดยูเอสบี 3.0 จะถูกพัฒนาขึ้นในปี ค.ศ. 2008 โดยมีความเร็วในการอ่านและเขียนข้อมูลสูงสุดเพิ่มขึ้นเกือบ 10 เท่า ปัจจุบันยูเอสบี 3.0 ได้ถูกเรียกชื่อใหม่ว่า ยูเอสบี 3.1 เจน 1 (USB 3.1 Gen 1) ขณะที่ USB 3.1 เจน 2 (USB 3.1 Gen 2) ได้ถูกพัฒนาขึ้นเพื่อรองรับความเร็วในการรับ-ส่งข้อมูลที่สูงขึ้นไปอีกเท่าตัว



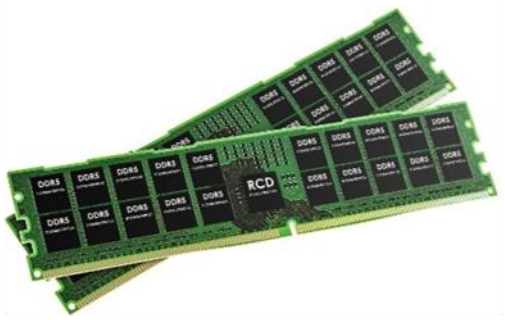
1	USB connector
2	USB mass storage controller device
3	Test points
4	Flash memory chip
5	Crystal oscillator
6	LED
7	Write-protect switch
8	Unpopulated space for second flash memory chip

รูปที่ 7- 20 โครงสร้างและส่วนประกอบภายในของหน่วยความจำแบบแฟลชไดรฟ์

- **ตัวอย่างการทำงาน:** เมื่อนักศึกษากดปุ่มเปิดเครื่องคอมพิวเตอร์ CPU จะเข้าถึง BIOS ที่เก็บอยู่ในชิป ROM เพื่อเรียกใช้ชุดคำสั่งพื้นฐานในการตรวจสอบอุปกรณ์ฮาร์ดแวร์ต่างๆ และโหลดระบบปฏิบัติการต่างๆ ขึ้นมาทำงานเช่น Windows

6. RAM (Random Access Memory)

คำอธิบาย: RAM เป็นหน่วยความจำที่ CPU ใช้สำหรับเก็บโปรแกรมและข้อมูลที่กำลังทำงานอยู่ ณ ขณะนั้น มีคุณสมบัติ **ลบเลือน (Volatile)** หมายถึงข้อมูลจะหายไปเมื่อปิดเครื่องหรือไม่มีกระแสไฟฟ้า



รูปที่ 7- 21 ลักษณะทางกายภาพของแผงหน่วยความจำหลักแบบ DDR5 SDRAM

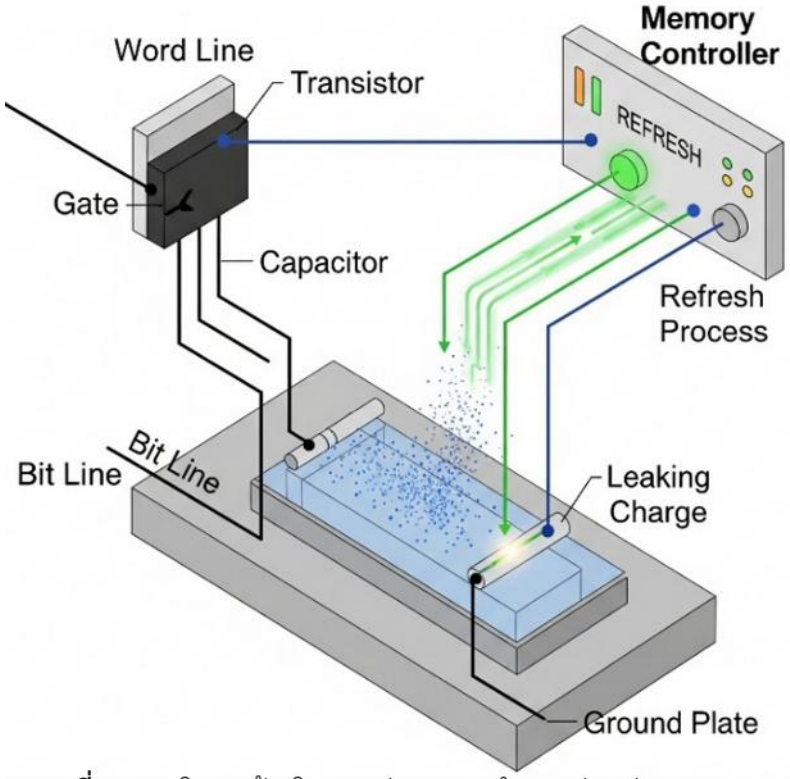
จากภาพโมดูล RAM รุ่นใหม่ล่าสุดอย่าง DDR5 ที่ใช้ในคอมพิวเตอร์ จะเห็นได้ว่ามีลักษณะคล้ายกับ RAM รุ่นก่อนหน้า แต่มีการออกแบบฮีทซิงค์ (heatsink) ที่ทันสมัยและสวยงามขึ้น

ประเภทของ RAM (Random Access Memory)

RAM เป็นหน่วยความจำหลักที่คอมพิวเตอร์ใช้สำหรับเก็บโปรแกรมและข้อมูลที่กำลังทำงานอยู่ มีคุณสมบัติลบเลือน (Volatile) เป็นหน่วยความจำชั่วคราวที่ใช้เก็บข้อมูลและโปรแกรมที่กำลังทำงานอยู่ เมื่อปิดเครื่องข้อมูลจะหายไป ซึ่ง RAM สามารถแบ่งออกเป็นสองประเภทหลักๆ คือ **DRAM** และ **SRAM** ที่มีโครงสร้างและการทำงานแตกต่างกัน

(https://www.youtube.com/watch?v=7J7X7aZvMXQ&t=202s)

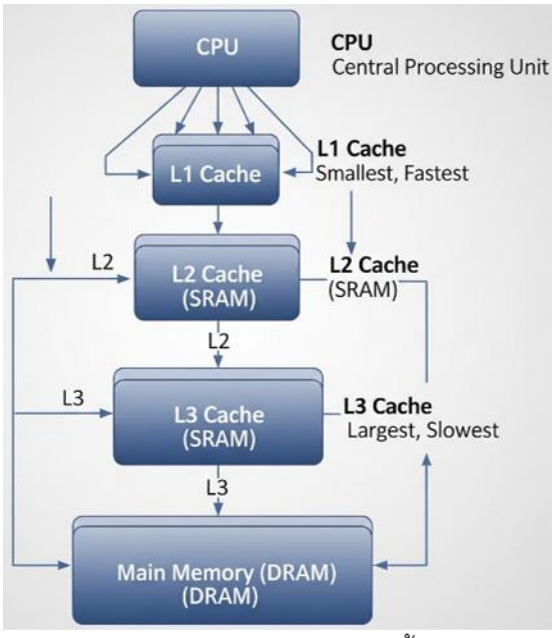
6.1.1.DRAM (Dynamic RAM): เป็นประเภทที่นิยมใช้เป็นหน่วยความจำหลักของคอมพิวเตอร์มากที่สุด เนื่องจากมีราคาถูกกว่าและมีความจุสูงกว่า **SRAM** โดยแต่ละบิตของข้อมูลใน DRAM ถูกเก็บไว้ในวงจรที่ประกอบด้วย **ทรานซิสเตอร์ 1 ตัว และตัวเก็บประจุ (Capacitor) 1 ตัว**



รูปที่ 7- 22 โครงสร้างโมดูลหน่วยความจำและส่วนประกอบบนแผงวงจรหลัก

ปัญหาการรีเฟรชและการทำงาน คือ **ตัวเก็บประจุมีคุณสมบัติที่ประจุจะค่อยๆ รั่วไหลออกไป** ทำให้ข้อมูลที่เก็บไว้สูญหายได้ในเวลาไม่กี่มิลลิวินาที จึงจำเป็นต้องมีกระบวนการ **"รีเฟรช" (Refresh)** เพื่อแก้ไขปัญหานี้

วิธีการทำงาน: ตัวควบคุมหน่วยความจำ (Memory Controller) จะทำหน้าที่เป็นเหมือนผู้ดูแลที่คอยตรวจสอบและส่งสัญญาณ **"รีเฟรช"** เป็นระยะๆ สัญญาณนี้จะสั่งให้อ่านข้อมูลจากตัวเก็บประจุทั้งหมดในแต่ละแถว แล้วทำการเขียนข้อมูลเดิมกลับเข้าไปใหม่ เพื่อเป็นการชาร์จประจุให้เต็มอยู่เสมอ ทำให้ข้อมูลยังคงอยู่ตราบใดที่ยังมีไฟฟ้าหล่อเลี้ยงอยู่

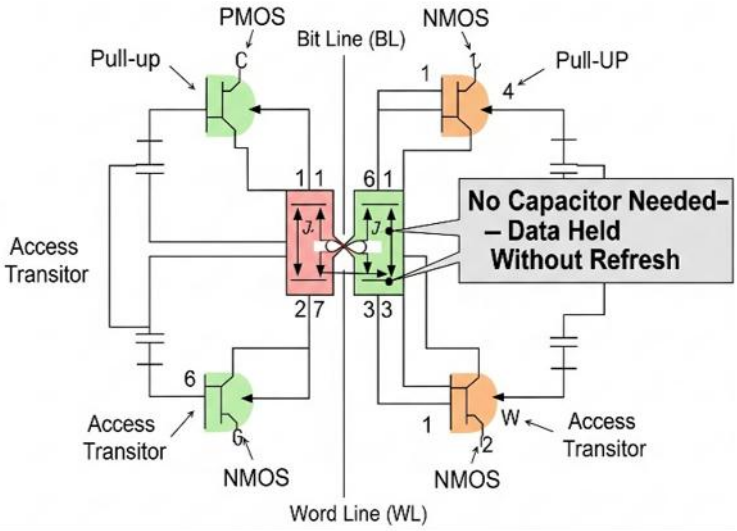


รูปที่ 7- 23 โครงสร้างลำดับชั้นของหน่วยความจำแคช

ข้อดี: ทำงานได้เร็วกว่า DRAM มาก

ข้อเสีย: มีราคาแพงและมีขนาดใหญ่กว่าต่อบิตข้อมูล

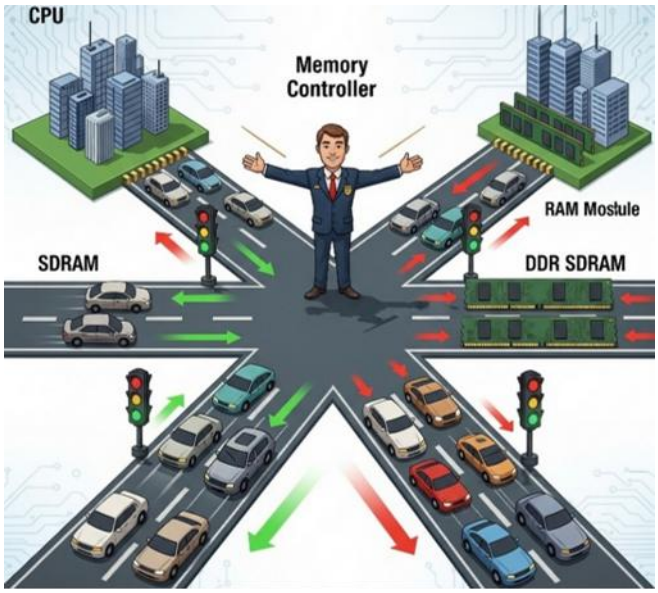
6.1.2.SRAM (Static RAM): ที่มีคุณสมบัติที่เร็วกว่าและมีราคาแพงกว่า DRAM แต่ละบิตของข้อมูลใน SRAM จะถูกเก็บไว้ในวงจรแบบ **"ฟลิปฟลอป" (Flip-flop)** ที่ประกอบด้วย **ทรานซิสเตอร์หลายตัว (ประมาณ 4-6 ตัว)** ซึ่งทำให้ข้อมูลสามารถคงอยู่เท่าที่มีไฟฟ้าหล่อเลี้ยง **โดยไม่ต้องมีการรีเฟรช**



รูปที่ 7- 24 โครงสร้างวงจรพื้นฐานของหน่วยความจำแบบ SRAM (Static RAM)

การนำไปใช้งาน: ด้วยความเร็วที่สูงมาก SRAM จึงถูกนำไปใช้เป็น **หน่วยความจำแคช (Cache Memory)** ใน CPU (เช่น L1, L2, L3 Cache) เพื่อช่วยลดความหน่วงในการเข้าถึงข้อมูลจากหน่วยความจำหลัก

ต่อมา DRAM ได้ถูกพัฒนามาอย่างต่อเนื่องเพื่อเพิ่มความเร็วและประสิทธิภาพ โดยจาก Asynchronous DRAM ได้มีการพัฒนาเป็น **SDRAM (Synchronous DRAM)** ซึ่งสามารถทำงานได้สอดคล้องกับสัญญาณนาฬิกาของ CPU ทำให้การส่งข้อมูลมีประสิทธิภาพมากขึ้น และในที่สุดก็พัฒนาเป็น **DDR (Double Data Rate) SDRAM** ที่สามารถส่งข้อมูลได้ทั้งจังหวะ **ขาขึ้นและขาลง** ของสัญญาณนาฬิกา ทำให้สามารถส่งข้อมูลได้เป็นสองเท่าในเวลาเท่าเดิม ลองนึกภาพถนนที่ใช้เดินทางระหว่าง CPU กับหน่วยความจำ DDR (RAM) เปรียบเสมือนรถยนต์ที่วิ่งบนถนน

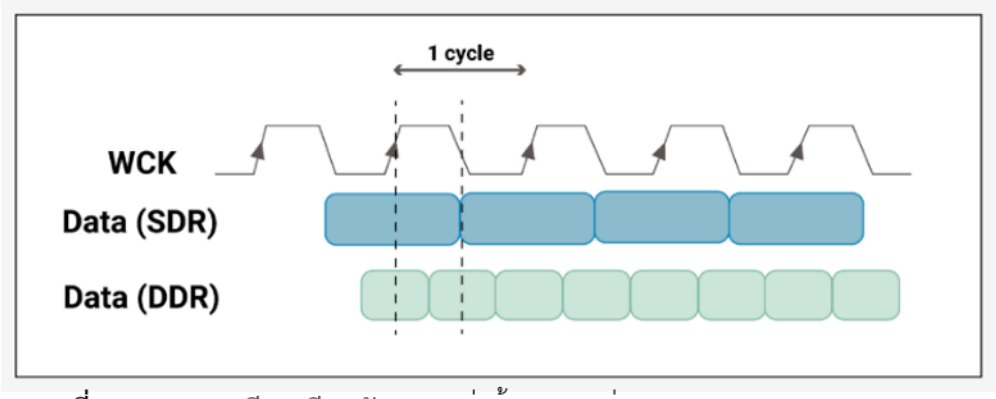


รูปที่ 7- 25 การเปรียบเทียบการรับส่งข้อมูลของหน่วยความจำแบบ SDRAM และ DDR SDRAM

SDRAM (รุ่นเก่า): เหมือนกับรถที่วิ่งได้แค่ตอนไฟเขียวเท่านั้น (ขาขึ้นของสัญญาณนาฬิกา)

DDR SDRAM: เหมือนรถที่วิ่งได้ทั้งตอนไฟเขียวและไฟแดง (ทั้งขาขึ้นและขาลงของสัญญาณนาฬิกา) ทำให้สามารถส่งข้อมูลได้เป็น **2 เท่า** ภายในเวลาเท่าเดิม การทำงานที่เร็วขึ้นนี้ช่วยให้ Memory Controller (ตัวควบคุมการจราจร) สามารถทำงานได้ดีขึ้น ส่งข้อมูลไปยัง CPU ได้ไวขึ้น และช่วยให้ระบบโดยรวมทำงานได้ลื่นไหลกว่าเดิม

6.3 การพัฒนาของ DDR RAM จากอดีตถึงปัจจุบัน



รูปที่ 7- 26 การเปรียบเทียบอัตราการส่งข้อมูลระหว่าง SDRAM และ DDR SDRAM ต่อรอบสัญญาณนาฬิกา

DDR คือเทคโนโลยีหน่วยความจำที่พัฒนาต่อยอดมาจาก SDRAM ซึ่งเป็น RAM แบบเดิม โดยหัวใจสำคัญของ DDR คือความสามารถในการ **รับส่งข้อมูลได้ 2 ครั้งต่อหนึ่งรอบสัญญาณนาฬิกา** (Double Data Rate)

ตัวอย่าง ลองนึกภาพสัญญาณนาฬิกาเหมือนกับจังหวะการก้าวเดิน

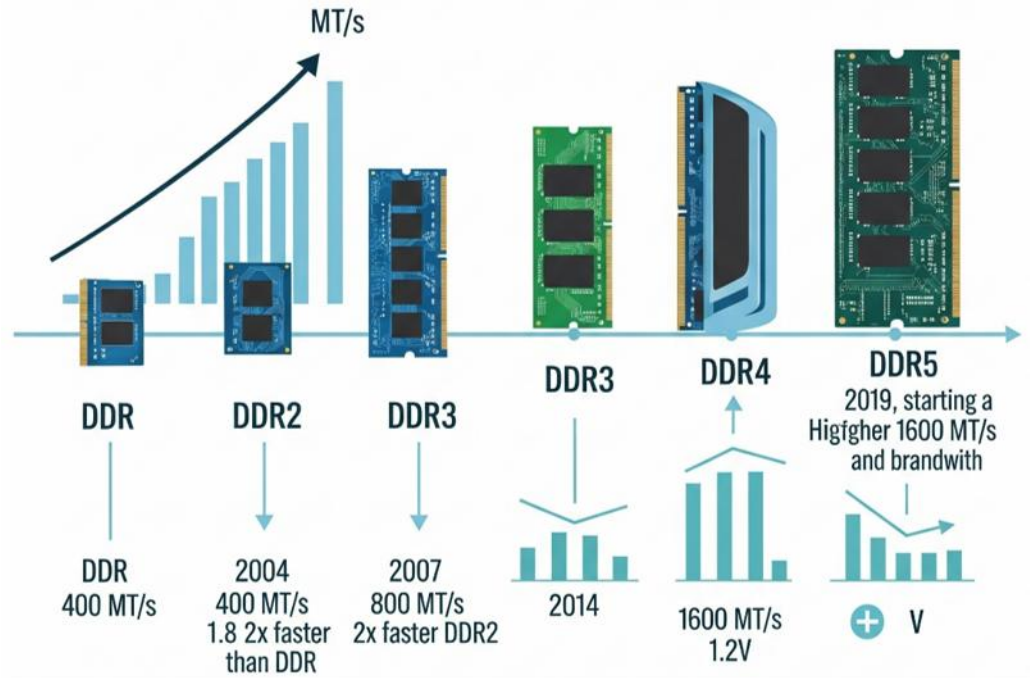
- **SDRAM แบบเดิม** จะส่งข้อมูลได้แค่ตอนจังหวะยกเท้า (ขอบขาขึ้น) เท่านั้น
- **DDR** สามารถส่งข้อมูลได้ทั้งจังหวะยกเท้าและตอนจังหวะที่วางเท้าลง (ขอบขาขึ้นและขอบขาลง)

ด้วยวิธีนี้ ทำให้ DDR มีความเร็วในการรับส่งข้อมูลเป็น 2 เท่าของ SDRAM ในรอบสัญญาณนาฬิกาที่เท่ากัน และยังช่วยประหยัดพลังงาน

6.3.1 คุณสมบัติสำคัญของ DDR

- **สมดุล:** มีความสมดุลระหว่างความเร็วในการส่งข้อมูล, ราคา, และการใช้พลังงาน
- **ประสิทธิภาพสูง:** มีความเร็วและประสิทธิภาพที่ดีขึ้น
- **ความน่าเชื่อถือ:** รองรับฟังก์ชัน ECC (Error Correcting Code) และ CRC (Cyclic Redundancy Check) เพื่อตรวจจับและแก้ไขข้อผิดพลาดของข้อมูล
- **ประหยัดพลังงาน:** มีฟังก์ชัน Automatic self refresh ทำให้หน่วยความจำรีเฟรชตัวเองตามอุณหภูมิการทำงาน

6.3.2 วิวัฒนาการของ DDR : การพัฒนาอย่างต่อเนื่องเพื่อตอบสนองความต้องการของ CPU ที่ทรงพลังขึ้นเรื่อยๆ โดยแต่ละรุ่นจะมีความเร็วเพิ่มขึ้นและใช้พลังงานลดลง



รูปที่ 7- 27 วิวัฒนาการของหน่วยความจำ DDR SDRAM และแนวโน้มความแรง-การประหยัดพลังงาน

DDR (ดั้งเดิม): เริ่มต้นเทคโนโลยี DDR ความเร็ว ประมาณ 400 MT/s (Mega transfers per second)

DDR2 (เปิดตัวปี 2004): ความเร็วเริ่มต้นที่ 400 MT/s มีความเร็วเพิ่มขึ้นเป็น 2 เท่าของ DDR และใช้พลังงานลดลง (1.8 โวลต์)

DDR3 (เปิดตัวปี 2007): ความเร็วเริ่มต้นที่ 800 MT/s ความเร็วเพิ่มขึ้นเป็น 2 เท่าของ DDR2 (2133 MT/s) และใช้พลังงานน้อยลง

DDR4 (เปิดตัวปี 2014): ความเร็วเริ่มต้นที่ 1600 MT/s และใช้พลังงานต่ำเพียง 1.2 โวลต์

DDR5 (เปิดตัวปี 2019): ความเร็วเริ่มต้นที่ 1600 MT/sรุ่นล่าสุดที่มีประสิทธิภาพสูงกว่า DDR4 ทั้งในด้านความเร็วและแบนด์วิดท์

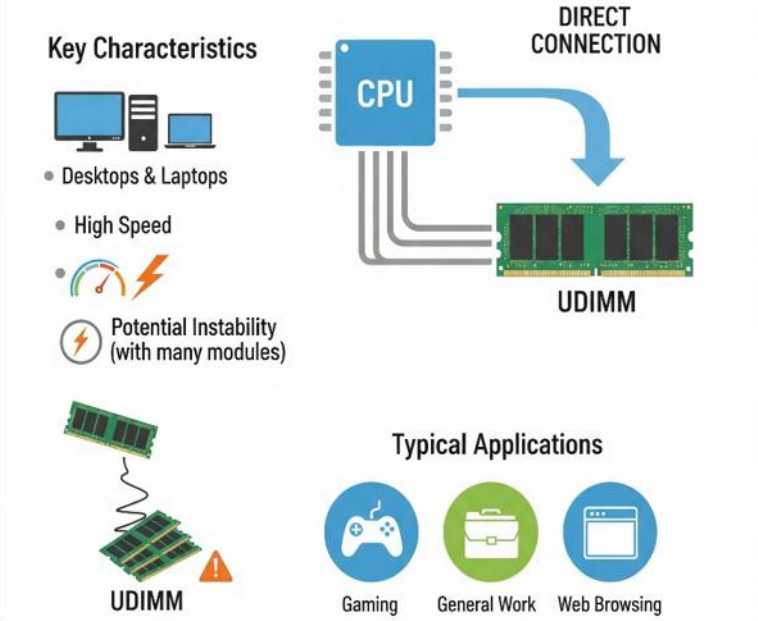
7.ประเภทและคุณสมบัติของ DIMM (Dual In-line Memory Module)

DIMM คือ **แผงวงจร** ที่ใช้สำหรับติดตั้งชิปหน่วยความจำ (RAM) เพื่อนำไปเสียบบนเมนบอร์ดของคอมพิวเตอร์

ในอดีตมีการใช้ **SIMM (Single In-line Memory Module)** ที่มีช่องทางการส่งข้อมูลกว้างเพียง 32 บิต ซึ่งคอมพิวเตอร์ส่วนใหญ่ต้องการช่องทาง 64 บิต ทำให้ผู้ใช้ต้องซื้อ SIMM มาเสียบเป็นคู่ (2 แผง) เพื่อให้ทำงานได้แต่ **DIMM** ถูกออกแบบมาให้มีช่องทางการส่งข้อมูลกว้าง 64 บิตในตัวเดียว สามารถเปรียบเทียบได้

SIMM (32-bit Data Path)	DIMM (64-bit Data Path)
ตัวอย่าง: ต้องการ RAM 8 GB	**ตัวอย่าง:** ต้องการ RAM 8 GB
SIMM **ขนาด 4 GB จำนวน 2 แผง** เพื่อให้ได้ช่องทางการส่งข้อมูลรวม 64 บิตที่คอมพิวเตอร์ต้องการ	DIMM **ขนาด 8 GB เพียงแผงเดียว** เพื่อให้ได้ช่องทางการส่งข้อมูล 64 บิตในตัวเดียว

ตารางที่ 7-2 การเปรียบเทียบสถาปัตยกรรมเส้นทางข้อมูล (Data Path) และวิธีการติดตั้งโมดูลหน่วยความจำแบบ SIMM และ DIMM



รูปที่ 7- 28 คุณสมบัติและการประยุกต์ใช้งานหน่วยความจำหลักแบบ UDIMM

การใช้ DIMM จึงทำให้การติดตั้งง่ายขึ้นและยังช่วยให้เมนบอร์ดสามารถรองรับ RAM ได้ในปริมาณที่มากขึ้นการเลือกใช้ DIMM ขึ้นอยู่กับประเภทของอุปกรณ์และความต้องการในการใช้งานที่แตกต่างกัน

7.1 แบบ UDIMM (Un-buffered DIMM)

ลักษณะ: เป็น RAM ที่ใช้กันทั่วไปใน **คอมพิวเตอร์ตั้งโต๊ะและแล็ปท็อป**

การทำงาน: หน่วยประมวลผล (CPU) จะส่งคำสั่งไปหาชิปหน่วยความจำโดยตรง ทำให้มีความเร็วสูง

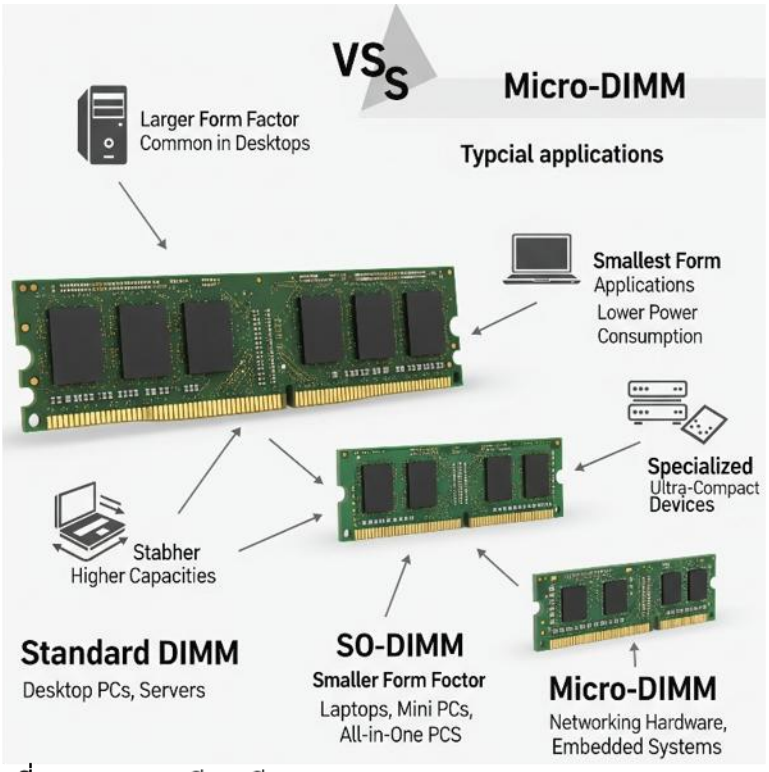


รูปที่ 7- 29 ลักษณะทางกายภาพด้านหน้าและด้านหลังของโมดูลหน่วยความจำ UDIMM

ข้อเสีย: เมื่อติดตั้ง RAM จำนวนมาก อาจทำให้สัญญาณอ่อนลงและไม่เสถียร

ตัวอย่างการใช้งาน: ทำงานทั่วไป, ท่องเว็บ

7.2.แบบ SO-DIMM และ Micro-DIMM

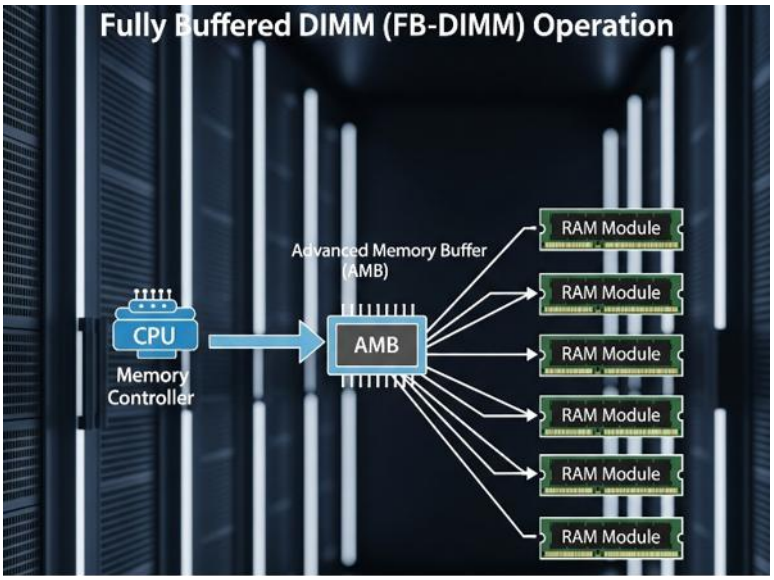


รูปที่ 7- 30 การเปรียบเทียบขนาดและรูปแบบมาตรฐาน (Form Factor) ของหน่วยความจำแบบ DIMM, SO-DIMM และ Micro-DIMM

- **ลักษณะ:** เป็น RAM ที่มีขนาดเล็กกว่า DIMM ทั่วไป
- **การทำงาน:** มีหน้าที่เหมือนกับ UDIMM แต่ถูกออกแบบมาเพื่อ **อุปกรณ์ขนาดเล็ก**
- **ตัวอย่างการใช้งาน: โน้ตบุ๊ก**, คอมพิวเตอร์ขนาดเล็ก (Mini PC), หรืออุปกรณ์เครือข่ายขนาดเล็ก

7.3 DIMM สำหรับเซิร์ฟเวอร์: FB-DIMM, RDIMM และ LRDIMM

DIMM เหล่านี้ถูกออกแบบมาเป็นพิเศษสำหรับเซิร์ฟเวอร์ที่ต้องการความเสถียรและความจุสูงเป็นหลัก เนื่องจากมีภาระการทำงานที่หนักกว่าคอมพิวเตอร์ทั่วไป



รูปที่ 7- 31 หลักการทำงานของหน่วยความจำแบบ Fully Buffered DIMM (FB-DIMM) ผ่านชิป AMB

7.3.1.FB-DIMM (Fully Buffered DIMM) เป็น RAM สำหรับเซิร์ฟเวอร์ที่ใช้เทคโนโลยี **AMB (Advanced Memory Buffers)** เป็นตัวกลางในการสื่อสารทั้งหมดระหว่าง Memory Controller และชิปหน่วยความจำ

- **การทำงาน:** แทนที่ Memory Controller จะส่งคำสั่งไปหาชิปหน่วยความจำโดยตรงเหมือน UDIMM ตัว AMB จะเป็นผู้รับคำสั่งทั้งหมดก่อน จากนั้นจึงส่งต่อให้กับชิปหน่วยความจำแต่ละตัว

ข้อดี: เทคโนโลยีนี้ช่วยให้การเดินสายบนแผงวงจรลดความซับซ้อนลงได้อย่างมาก ทำให้สามารถติดตั้งโมดูล RAM จำนวนมากในระบบเดียวได้โดยไม่เกิดปัญหาเรื่องสัญญาณรบกวน เหมาะสำหรับเซิร์ฟเวอร์ที่ต้องการความจุ RAM สูงสุด



รูปที่ 7- 32 ชิป AMB (Advanced Memory Buffer) สำหรับหน่วยความจำ FB-DIMM

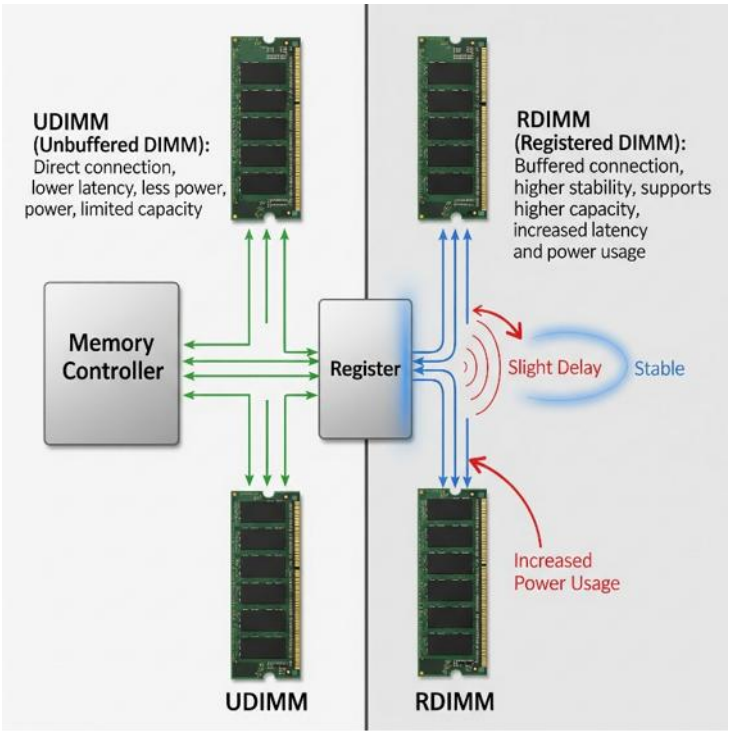
4607 AMB: The new Infineon FB-DIMMs are based on an Advanced Memory Buffer (AMB) chip, DDR2 DRAM chips and a proprietary heat sink all sourced from Infineon. Additionally, Infineon offers the AMB logic chip to other FB-DIMM manufacturers and has already shipped samples to first customers



รูปที่ 7- 33 โมดูลหน่วยความจำแบบ FB-DIMM พร้อมชิปควบคุม AMB

Fb-DIMM 1GB: Infineon Technologies is sampling the industry's only Double Data Rate 2 (DDR2) Fully Buffered Dual-In-line Memory Modules (FB-DIMMs) with all key components designed and manufactured by a single DRAM supplier.

7.3.2.RDIMM (Registered DIMM) เป็น RAM สำหรับเซิร์ฟเวอร์ที่เน้นความน่าเชื่อถือและความสามารถในการรองรับ RAM ที่มีความจุสูง



รูปที่ 7- 34 เปรียบเทียบการจัดลำดับสัญญาณระหว่างหน่วยความจำแบบ Unbuffered และ Registered

- **การทำงาน:** RDIMM มีชิปที่เรียกว่า **Register (หรือ RCD - Registered Clock Driver)** ซึ่งทำหน้าที่ **บัฟเฟอร์สัญญาณควบคุม** ที่มาจาก Memory Controller
- **ข้อดี:** การบัฟเฟอร์นี้ช่วยลดภาระทางไฟฟ้าบน Memory Controller ทำให้สามารถจัดการกับโมดูล RAM ความจุสูงหลายตัวได้ดีขึ้น ส่งผลให้ระบบเสถียรขึ้น

- **ข้อเสีย:** การมี Register เพิ่มขึ้นมา ทำให้มีค่าความหน่วง (Latency) เพิ่มขึ้นหนึ่งรอบสัญญาณนาฬิกา และใช้พลังงานมากกว่า UDIMM เล็กน้อย

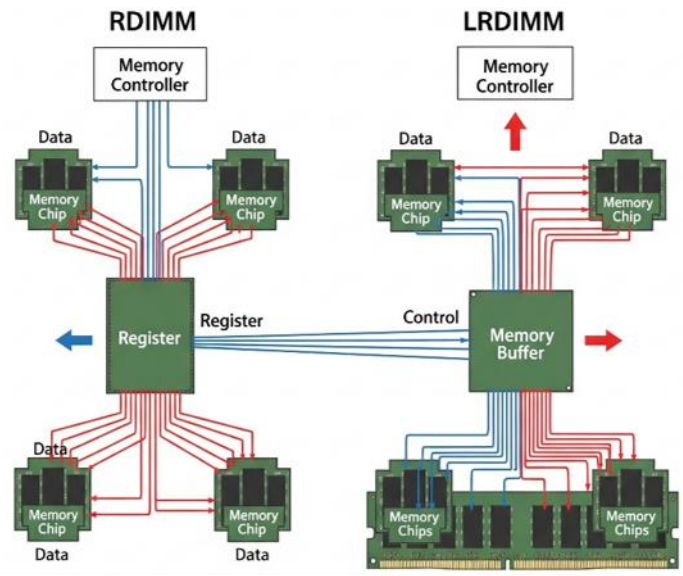


รูปที่ 7- 35 ลักษณะทางกายภาพเปรียบเทียบระหว่างโมดูลหน่วยความจำแบบ RDIMM และ LRDIMM

Discover the powerful HPE 16GB DDR5-4800 RDIMM *Server RAM, designed for HP Gen11 Servers. Learn about its features, advantages, compatibility, and use cases in data centers, virtualization, cloud computing, and more. Optimize your server performance with HPE DDR5 server memory.*

HPE 16GB DDR5-4800 RDIMM *คือหน่วยความจำเซิร์ฟเวอร์ (Server RAM) ที่ออกแบบมาเพื่อเพิ่มประสิทธิภาพและความน่าเชื่อถือสำหรับเซิร์ฟเวอร์* ***HPE ProLiant Gen11*** *โดยเฉพาะ ซึ่งเป็นการเปลี่ยนผ่านจากเทคโนโลยี* ***DDR4*** *ไปสู่* ***DDR5*** *ที่ให้ประสิทธิภาพสูงกว่าและใช้พลังงานน้อยกว่า ความเร็วสูงสุดที่* ***4800 MT/s (Megatransfers/second)*** *ซึ่งช่วยให้การถ่ายโอนข้อมูลรวดเร็วขึ้นและลดความหน่วง (Latency) ได้อย่างมีนัยสำคัญ มีคุณสมบัติ* ***ECC*** *ในตัวเพื่อตรวจสอบและแก้ไขข้อผิดพลาดของข้อมูลโดยอัตโนมัติ ลดโอกาสที่ระบบจะล่ม การทำงานที่แรงดันไฟฟ้าต่ำลง (****1.1V****) เมื่อเทียบกับ DDR4 ทำให้ช่วยประหยัดพลังงานและลดค่าใช้จ่ายในการดำเนินงานของดาต้าเซ็นเตอร์ได้*

7.3.3 LRDIMM (Load-Reduced DIMM) เป็นวิวัฒนาการล่าสุดของ RAM สำหรับเซิร์ฟเวอร์ที่ช่วยแก้ไขข้อจำกัดของ RDIMM



รูปที่ 7- 36 เปรียบเทียบโครงสร้างการจัดการสัญญาณระหว่างหน่วยความจำแบบ RDIMM และ LRDIMM

- **การทำงาน:** นอกจากจะมี Register เหมือน RDIMM แล้ว LRDIMM ยังมีชิป **Memory Buffer** ที่คอยจัดการสัญญาณข้อมูลทั้งหมดด้วย
- **ข้อดี:** การจัดการทั้งสัญญาณควบคุมและข้อมูลนี้ช่วยลดภาระทางไฟฟ้าบนบัสหน่วยความจำได้อย่างมาก (Load-Reduced) ทำให้ระบบสามารถทำงานที่ความเร็วสูงได้ แม้จะมีการติดตั้งชิปหน่วยความจำจำนวนมากบนแต่ละแผงก็ตาม

ความแตกต่างกับ RDIMM: ในขณะที่ RDIMM ลดภาระเฉพาะสัญญาณควบคุม LRDIMM ลดภาระของสัญญาณข้อมูลด้วย ทำให้สามารถรองรับความจุและความเร็วที่สูงกว่า RDIMM ได้



รูปที่ 7- 37 ภาพเปรียบเทียบระหว่างโมดูลหน่วยความจำ RDIMM และ LRDIMM

Crucial's DDR4 LRDIMM Server Memory modules can help applications such as virtualization, cloud computing, and high performance computing (HPC) on two fronts. For one, they can allow for more DRAM in a server through higher densities, modules go up to 128GB. For a server with 12 memory slots per CPU could potentially bring the RAM up to 1.5TB per CPU. The DRAM has speeds up to 2,666MT/s (with the lower capacities running 2,400MT/s). The RAM also pulls 1.2V of power consumption. All of these can not only help deliver promised performance, but can save on costs as it can consolidate usage down through density and help avoid shutdowns.

7.3.4 คำอธิบายและคุณสมบัติของ RAM รุ่น Crucial DDR4 LRDIMM

*โมดูลหน่วยความจำ **Crucial DDR4 LRDIMM** สำหรับเครื่องเซิร์ฟเวอร์สามารถช่วยเพิ่มประสิทธิภาพให้กับงานประมวลผลที่ซับซ้อน เช่น ระบบเสมือน (virtualization), คลาวด์คอมพิวติ้ง (cloud computing), และการประมวลผลประสิทธิภาพสูง (HPC - High Performance Computing) ได้ 2 ประการหลักๆ:*

1. ***ความจุที่สูงขึ้น:** โมดูล RAM รุ่นนี้มีความหนาแน่นของชิป (density) ที่สูงขึ้น ทำให้สามารถเพิ่มปริมาณ DRAM ในเซิร์ฟเวอร์ได้มากขึ้น โดยแต่ละโมดูลสามารถมีความจุได้สูงสุดถึง **128 GB** ซึ่งหมายความว่า หากเซิร์ฟเวอร์มีช่องสำหรับใส่ RAM 12 ช่องต่อ CPU ก็จะสามารถเพิ่ม RAM ได้สูงสุดถึง **1.5 TB (เทระไบต์) ต่อ CPU***
2. ***ความเร็วและประสิทธิภาพ:** RAM รุ่นนี้มีความเร็วในการถ่ายโอนข้อมูลสูงสุดถึง **2,666 MT/s (เมกะทรานส์เฟอร์ต่อวินาที)** โดยรุ่นที่มีความจุต่ำจะทำงานที่ความเร็ว **2,400 MT/s** นอกจากนี้ ยังใช้พลังงานเพียง **1.2 โวลต์** ซึ่งช่วยลดการใช้พลังงานได้*

คุณสมบัติเหล่านี้ไม่เพียงแต่ช่วยให้ได้ประสิทธิภาพการทำงานตามที่ต้องการ แต่ยังช่วยประหยัดค่าใช้จ่ายได้ด้วย เนื่องจากความหนาแน่นที่สูงขึ้นทำให้สามารถรวมการใช้งานในเซิร์ฟเวอร์ให้เหลือจำนวนน้อยลง และ

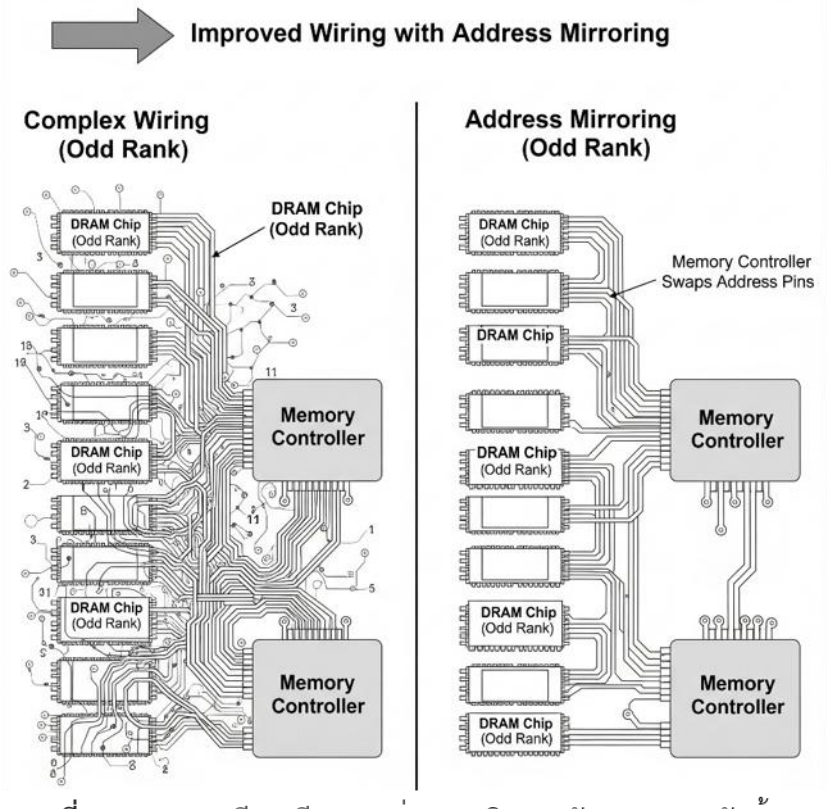
ยังช่วยป้องกันปัญหาเครื่องหยุดทำงาน (shutdown) จากการทำงานหนัก ความหนาแน่นที่สูงขึ้นยังช่วยให้การวางแผนการใช้ DRAM เป็นไปตามการใช้งานจริงได้อีกด้วย VDI (Virtual Desktop Infrastructure) เป็นตัวอย่างที่น่าสนใจ เนื่องจาก GPUs รุ่นใหม่ๆ ช่วยให้สามารถทำงานด้านกราฟิกดีไซน์ได้มากขึ้นเรื่อยๆ

ยกตัวอย่างเช่น ระบบ VDI อาจมีประสิทธิภาพลดลงอย่างมากหาก VM (Virtual Machine) แต่ละเครื่องไม่ได้รับการจัดสรร RAM ที่เพียงพอ ปัญหาหนึ่งในเรื่องนี้คือ แอปพลิเคชันส่วนใหญ่จะต้องการ RAM เพิ่มขึ้นเมื่อเวลาผ่านไป ดังนั้น ผู้ดูแลระบบจึงไม่เพียงแต่ต้องพิจารณาว่า VM จะต้องการ RAM เท่าไหร่ในตอนเริ่มต้นเท่านั้น แต่ยังต้องคาดการณ์ปริมาณการใช้งานในอนาคตด้วย

7.3.5 ข้อมูลเชิงสถาปัตยกรรม DIMM (Dual In-line Memory Module) มีคุณสมบัติพิเศษ 2 อย่างคือ Address Mirroring และ Output Inversion

7.3.5.1. Address Mirroring

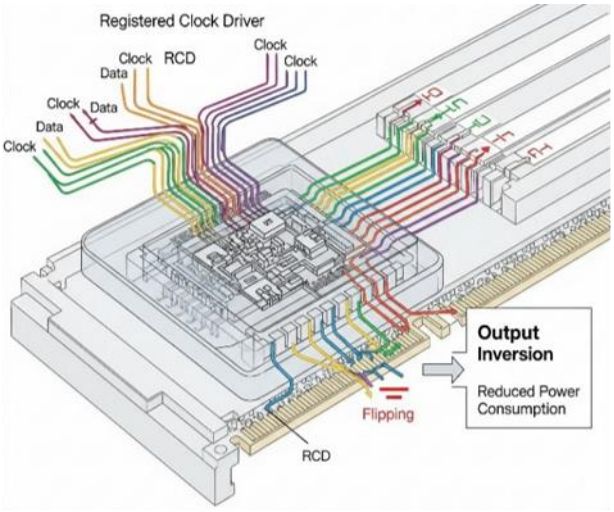
Address Mirroring คือเทคนิคที่ช่วยให้การออกแบบแผงวงจรของ RAM ง่ายขึ้น โดยการสลับคู่ของขา Address บางส่วนบนชิป RAM แถวที่สองหรือแถวเลขคี่ (Odd Ranks) โดยปกติแล้ว การเชื่อมต่อระหว่างชิป RAM และ Memory Controller บนเมนบอร์ดต้องใช้เส้นทาง (Traces) ที่ซับซ้อน มีจุดโค้ง (Conductor corners) และรูทะลุแผงวงจร (Vias) จำนวนมากทำให้สัญญาณเดินทางได้ไม่ดี



รูปที่ 7- 38 การเปรียบเทียบระหว่างการเดินสายสัญญาณแบบซับซ้อนและการใช้เทคนิค Address Mirroring

- **หลักการคือ:** ไม่ว่าการเชื่อมต่อทางกายภาพจะเป็นอย่างไร สิ่งสำคัญคือข้อมูลที่เขียนไปยังตำแหน่งหนึ่งจะต้องถูกอ่านกลับจากตำแหน่งเดิมอย่างถูกต้อง
- **ประโยชน์:** เทคนิค Address Mirroring ช่วยให้นักออกแบบสามารถจัดวางเส้นทางของขา Address และ Data ให้สั้นที่สุดและมีจำนวนรูทะลุแผงวงจรน้อยที่สุดได้

- **การทำงาน:** ในชิป RAM แถวคู่ (Even Rank DRAM) ขา Address จะเชื่อมต่อตรงๆ โดยไม่มีการสลับ แต่ในชิป RAM แถวคี่ (Odd Rank DRAM) ขา Address บางส่วนจะถูกสลับตำแหน่ง เพื่อให้การเดินสายง่ายขึ้น ซึ่งตัว **Memory Controller** (วงจรควบคุมหน่วยความจำ) บนเมนบอร์ดจะต้องรับรู้และปรับเปลี่ยนคำสั่งเพื่อรองรับการสลับตำแหน่งนี้ เพื่อให้การทำงานเป็นไปอย่างถูกต้อง



รูปที่ 7- 39 การเปรียบเทียบโครงสร้างโมดูลหน่วยความจำแบบ Single Rank และ Dual Rank

7.3.5.2 Output Inversion เป็นฟังก์ชันที่ทำงานใน RDIMM และ LRDIMM เพื่อช่วยประหยัดพลังงาน

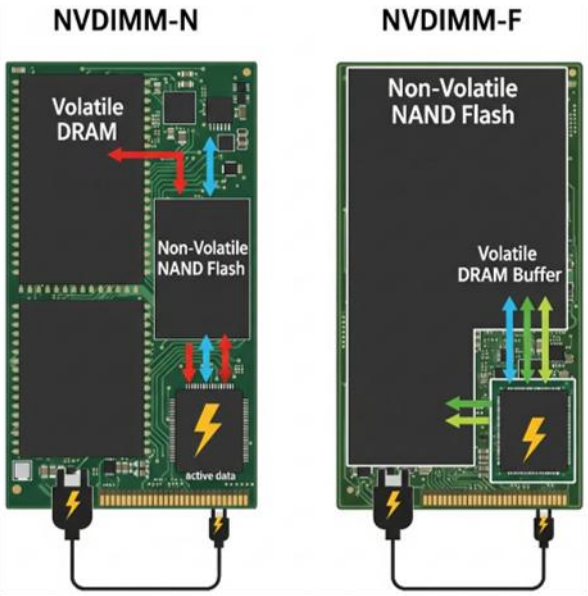
- **หลักการ:** ชิป RCD (Registered Clock Driver) จะสลับสัญญาณเอาต์พุตบางส่วนเพื่อให้ใช้พลังงานน้อยลงในขณะที่ส่งสัญญาณ

- คลิปการทำงานของ RAM (https://www.youtube.com/watch?v=7J7X7aZvMXQ&t=202s)

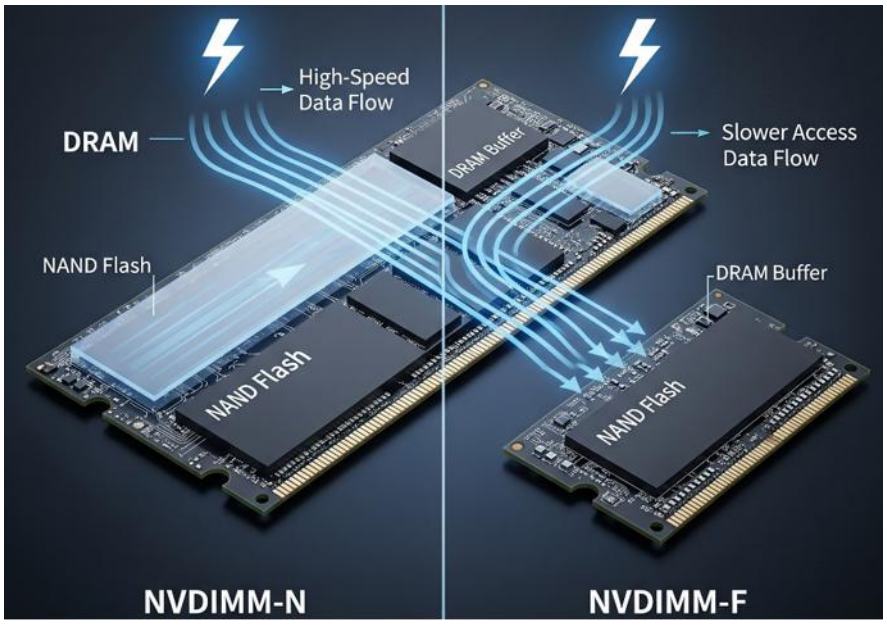
7.4 การทำงานของ RAM: แบบNVDIMM

7.4.1 NVDIMM (Non-volatile Dual inline memory module) เป็นเทคโนโลยีที่ผสมผสานระหว่าง RAM (DRAM) และหน่วยความจำแบบถาวร (NAND Flash) เข้าไว้ในแผงเดียวกัน

- **การทำงาน:** เมื่อระบบไฟฟ้าดับลง ข้อมูลที่อยู่ใน DRAM จะถูกคัดลอกไปยัง NAND Flash โดยอัตโนมัติ ทำให้ข้อมูลไม่สูญหาย
- **ประโยชน์:** เมื่อระบบกลับมาทำงาน ข้อมูลสามารถถูกกู้คืนได้อย่างรวดเร็วจาก NAND Flash โดยไม่ต้องโหลดใหม่จากอุปกรณ์จัดเก็บข้อมูลอื่นๆ



รูปที่ 7- 40 โครงสร้างและการทำงานของหน่วยความจำแบบ NVDIMM-N และ NVDIMM-F



รูปที่ 7- 41 กลไกการรับส่งข้อมูลและการสำรองข้อมูลของหน่วยความจำ NVDIMM-N และ NVDIMM-F

7.4.2 NVDIMM-N: เน้นความเร็วในการเข้าถึงข้อมูล โดยมี DRAM และ Flash ในปริมาณที่ใกล้เคียงกัน เหมาะสำหรับแอปพลิเคชันที่ต้องการการเข้าถึงข้อมูลที่ต่อเนื่องและรวดเร็ว *(เช่น 16 GB) และ Flash ในปริมาณเท่ากัน เน้นความเร็วในการเข้าถึงข้อมูล*

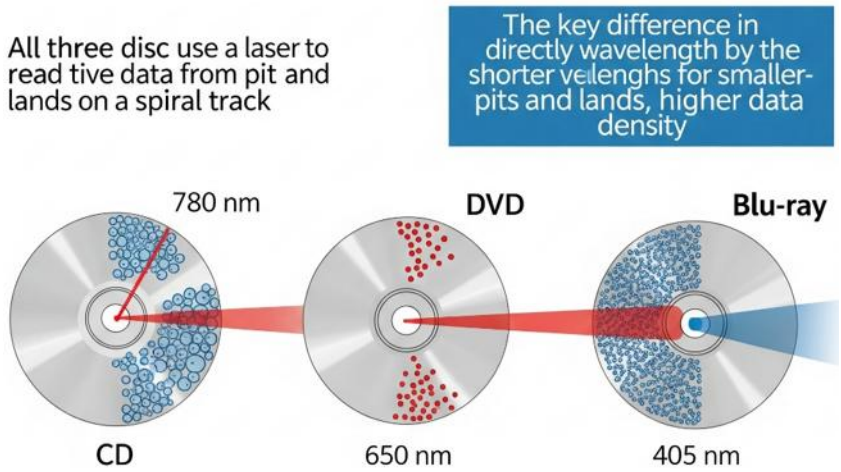
- **7.4.3 NVDIMM-F:** ใช้ DRAM เป็นเพียงบัฟเฟอร์เล็กๆ สำหรับ Flash ที่มีความจุสูงมาก เหมาะสำหรับงานที่ต้องการพื้นที่จัดเก็บข้อมูลขนาดใหญ่ Flash (หลายเทราไบต์) มีความเร็วต่ำกว่าแต่มีความจุสูงกว่าแต่ไม่จำเป็นต้องเข้าถึงข้อมูลที่รวดเร็วเท่า NVDIMM-N

8.ประเภทของหน่วยความจำสำรอง (Secondary Memory) หน่วยความจำสำรองมีหลายประเภท

8.1 หน่วยความจำแสง (Optical Memory): ใช้แผ่นพลาสติกเคลือบอะลูมิเนียมสะท้อนแสง เช่น CD, DVD, และ Blu-ray ซึ่งใช้เลเซอร์ที่มีความยาวคลื่นต่างกันในการอ่านและบันทึกข้อมูล

8.1.1 หลักการทำงานของหน่วยความจำแสง (Optical Memory)

หน่วยความจำแสง เช่น **CD, DVD, และ Blu-ray Disc** ใช้เทคโนโลยีเลเซอร์ในการอ่านและบันทึกข้อมูลบนแผ่นพลาสติกที่เคลือบด้วยวัสดุสะท้อนแสง ข้อมูลจะถูกบันทึกในรูปแบบของ **หลุม (pits)** และ **พื้นผิวเรียบ (lands)** บนแทร็กวนเป็นเกลียว (spiral track) จากด้านในออกด้านนอก



รูปที่ 7- 42 การเปรียบเทียบความยาวคลื่นเลเซอร์และความหนาแน่นของข้อมูลบนแผ่น CD, DVD และ Blu-ray

การบันทึกข้อมูล: หัวบันทึกจะใช้ลำแสงเลเซอร์ที่มีความเข้มสูงเพื่อ **เผาหรือสร้างหลุม** เล็กๆ บนชั้นผิวของแผ่น

การอ่านข้อมูล: หัวอ่านจะใช้ลำแสงเลเซอร์ที่มีความเข้มต่ำส่องไปบนแทร็ก ข้อมูลจะถูกอ่านจากแสงที่สะท้อนกลับมา

- ถ้าลำแสงส่องไปที่ **พื้นผิวเรียบ (land)** แสงจะสะท้อนกลับมาเต็มที่
- ถ้าลำแสงส่องไปที่ **หลุม (pit)** แสงจะกระเจิงออกไป ทำให้มีแสงสะท้อนกลับมาน้อย

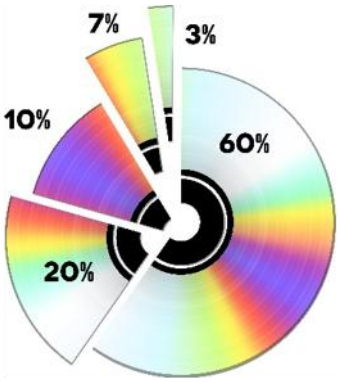
- **การแปลงข้อมูล:** เซ็นเซอร์จะตรวจจับความแตกต่างของแสงที่สะท้อนกลับมา แล้วแปลงเป็นข้อมูลดิจิทัล (0 และ 1) ซึ่งเป็นหลักการพื้นฐานของการอ่านข้อมูลนั่นเอง

8.1.2 การพัฒนายุคสมัยต่างๆ ของประเภทของหน่วยความจำแสง

วิวัฒนาการของหน่วยความจำแสงจะเน้นไปที่การเพิ่มความจุโดยการใช้เทคโนโลยีเลเซอร์ที่ล้ำสมัยขึ้น โดยมี **ความยาวคลื่นของเลเซอร์ (Wavelength)** ที่สั้นลงเพื่อสร้างหลุมและแทร็กที่เล็กลง ทำให้สามารถบรรจุข้อมูลได้มากขึ้นบนพื้นที่เท่าเดิม

ประเภท	ปีเริ่มต้น	ความจุเริ่มต้น	ความยาวคลื่นเลเซอร์
CD (Compact Disc)	1982	650 MB	780 nm (แสงอินฟราเรด)
DVD (Digital Versatile Disc)	1995	4.7 GB (Single-layer)	650/635 nm (แสงสีแดง)
Blu-ray Disc	2006	25 GB (Single-layer)	405 nm (แสงสีน้ำเงิน-ม่วง)

ตารางที่ 7-3 สรุปวิวัฒนาการและคุณสมบัติทางเทคนิคของสื่อจัดเก็บข้อมูลประเภท CD, DVD และ Blu-ray



รูปที่ 7- 43 สัดส่วนการจัดสรรพื้นที่จัดเก็บข้อมูลและรหัสแก้ไขข้อผิดพลาดบนแผ่น Optical Disc

8.1.3 CD (Compact Disc) : CD ถูกพัฒนาขึ้นครั้งแรกเพื่อใช้สำหรับบันทึกเสียงต่อมาจึงพัฒนาเป็น CD-ROM สำหรับการจัดเก็บข้อมูลคอมพิวเตอร์ ซึ่งมีความจุมาตรฐานอยู่ที่ประมาณ 650-700 MB เพียงพอสำหรับการเก็บข้อมูลเล็กๆ

8.1.4 DVD (Digital Versatile Disc) DVD ถูกพัฒนาขึ้นเพื่อใช้ทดแทน VHS และ CD ด้วยความจุที่สูงขึ้น โดยใช้เลเซอร์แสงสีแดงที่สั้นกว่าเลเซอร์อินฟราเรดของ CD ทำให้สามารถอ่านและเขียนข้อมูลได้อย่างหนาแน่นขึ้น มีหลายรูปแบบ เช่น DVD-ROM, DVD-R, DVD-RW ที่สามารถบันทึกและลบข้อมูลได้



รูปที่ 7- 44 เครื่องอ่านและเขียนแผ่น DVD แบบติดตั้งภายนอกผ่านพอร์ต USB



รูปที่ 7- 45 ไดรฟ์ Blu-ray แบบติดตั้งภายในที่รองรับเทคโนโลยี M-DISC

8.1.5 Blu-ray Disc เป็นหน่วยความจำแสงในยุคปัจจุบันที่เน้นการจัดเก็บข้อมูลความละเอียดสูงอย่างวิดีโอระดับ HD และ 4K ชื่อ "Blu-ray" มาจากสีของเลเซอร์ที่ใช้คือแสงสีน้ำเงิน-ม่วง ซึ่งมีความยาวคลื่นสั้นที่สุดในบรรดาหน่วยความจำแสง ทำให้สามารถบันทึกข้อมูลในความหนาแน่นที่สูงมาก ความจุเริ่มต้นที่ 25 GB สำหรับแผ่นชั้นเดียวและสูงถึง 50 GB สำหรับแผ่นสองชั้น

8.1.6 หลักการหมุนของแผ่นดิสก์ (Constant Linear Velocity - CLV)



รูปที่ 7- 46 ลักษณะของแผ่น Blu-ray Disc และการสะท้อนของลำแสงเลเซอร์ สีน้ำเงิน-ม่วง (Blue-Violet Laser)

แผ่นดิสก์จะหมุนด้วยความเร็วรอบที่แตกต่างกัน เพื่อให้ความเร็วเชิงเส้น (Linear Velocity) ของหัวอ่าน/เขียนคงที่เสมอ ไม่ว่าหัวอ่านจะอยู่ส่วนใดของแผ่น

- **เมื่อหัวอ่านอยู่ด้านใน:** แผ่นจะหมุนด้วยความเร็วรอบสูง (rpm สูง)
- **เมื่อหัวอ่านอยู่ด้านนอก:** แผ่นจะหมุนด้วยความเร็วรอบต่ำ (rpm ต่ำ)

8.1.6.1 การควบคุมความเร็วเชิงเส้นให้คงที่นี้มีเหตุผลคือ:

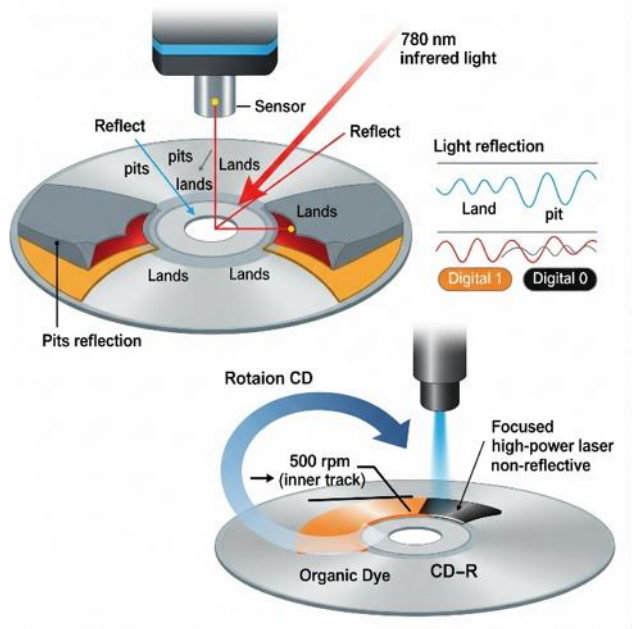
1. **รักษาอัตราการส่งข้อมูล:** ข้อมูลถูกจัดเรียงอย่างสม่ำเสมอในทุกจุดบนแทร็ก เพื่อให้หัวอ่านสามารถส่งข้อมูลได้อย่างต่อเนื่องและสม่ำเสมอ

2. **ประสิทธิภาพการอ่าน:** หัวอ่านสามารถอ่านข้อมูลได้อย่างแม่นยำและไม่สะดุด

ประเภทแผ่น	ความเร็วเชิงเส้น	ความเร็วรอบ (ประมาณ)
CD	1.2–1.4 m/s	500-200 rpm
DVD	3.49 m/s	1600-600 rpm
Blu-ray	4.9 m/s	10000-2000 rpm

ตารางที่ 7-4 การเปรียบเทียบความเร็วเชิงเส้นและความเร็วรอบในการทำงานของสื่อจัดเก็บข้อมูลแบบใช้แสง

8.1.7 การทำงานของแผ่นแต่ละประเภท การทำงานต่างกันตามโครงสร้างทางกายภาพแผ่นและการใช้เลเซอร์

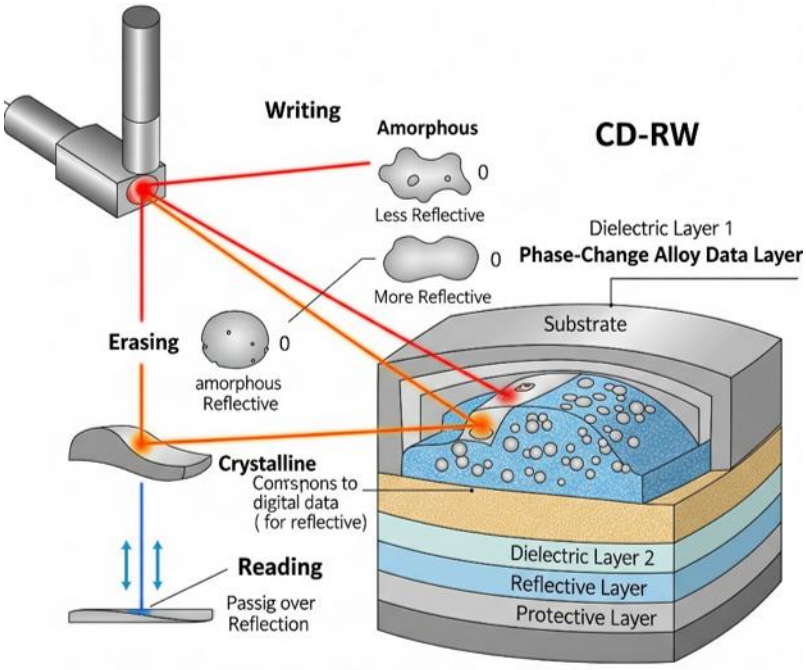


รูปที่ 7- 47 กลไกการสะท้อนของแสงเลเซอร์บนพื้นผิว Pits และ Lands เพื่อการอ่านและเขียนข้อมูลดิจิทัล

<u>CD (Compact Disc)</u>

- **การอ่าน:** หัวอ่านจะใช้เลเซอร์อินฟราเรด (780 nm) ยิงลงบนพื้นผิวแผ่น แสงที่สะท้อนจาก "พื้นผิวเรียบ" (land) และ "หลุม" (pit) จะมีความแตกต่างกัน ซึ่งเซ็นเซอร์จะแปลงเป็นข้อมูลดิจิทัล
- **การเขียน:** สำหรับแผ่น CD-R ที่บันทึกได้ครั้งเดียว จะใช้เลเซอร์ที่มีกำลังสูงกว่าในการเปลี่ยนสีของชั้นสารเคมีอินทรีย์ ทำให้เกิด "รอย" ที่ไม่สะท้อนแสง ซึ่งทำหน้าที่แทนหลุม
- **ความเร็วพร้อมใช้งาน:** เมื่อความเร็วรอบหมุนถึงระดับที่กำหนด (สำหรับ CD-ROM คือ 500 rpm เมื่อเริ่มอ่านจากด้านใน) ก็จะสามารถเริ่มอ่านข้อมูลได้ทันที

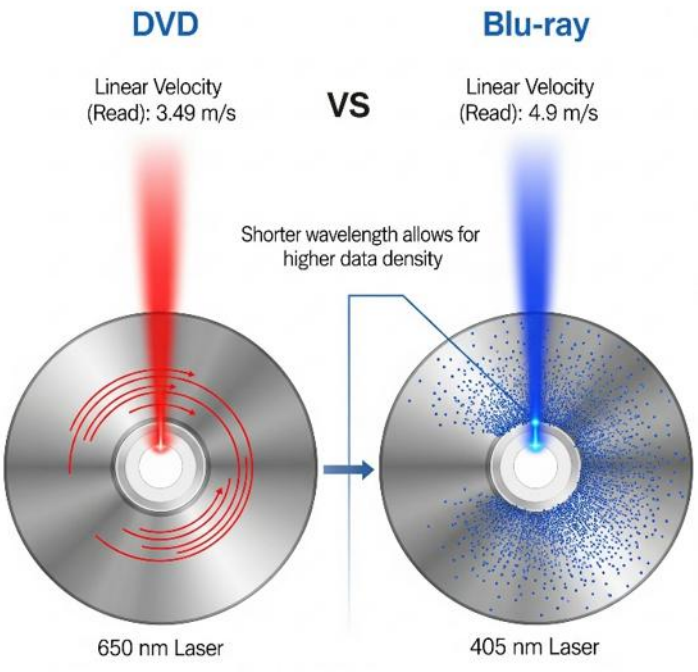
<u>8.1.8 CD-RW (Compact Disc Re Writable)</u>



รูปที่ 7- 48 กลไกการเปลี่ยนสถานะของสาร (Phase-Change) เพื่อการเขียน ลบ และอ่านข้อมูลบนแผ่น CD-RW

- **การอ่านและเขียนซ้ำ:** แผ่น CD-RW ใช้ชั้นสารประกอบโลหะที่เปลี่ยนสถานะได้ (Phase-change alloy) ระหว่างผลึก (Crystal) กับอสัณฐาน (Amorphous)
  - **การเขียน (Writing):** ใช้เลเซอร์กำลังสูงเพื่อเปลี่ยนสถานะของวัสดุให้เป็นอสัณฐาน ซึ่งจะทำหน้าที่เหมือนหลุมที่แสงสะท้อนกลับมาน้อย
  - **การลบ (Erasing):** ใช้เลเซอร์กำลังปานกลางเพื่อคืนสถานะของวัสดุจากอสัณฐานให้เป็นผลึก ทำให้แสงสะท้อนกลับมาเต็มที่ สามารถเขียนข้อมูลใหม่ทับได้

8.1.9 DVD และ Blu-ray แผ่นทั้งสองประเภทมีหลักการทำงานคล้ายกันกับ CD แต่ใช้เลเซอร์ที่มีความยาวคลื่นสั้นกว่าเพื่อเพิ่มความหนาแน่นของข้อมูล

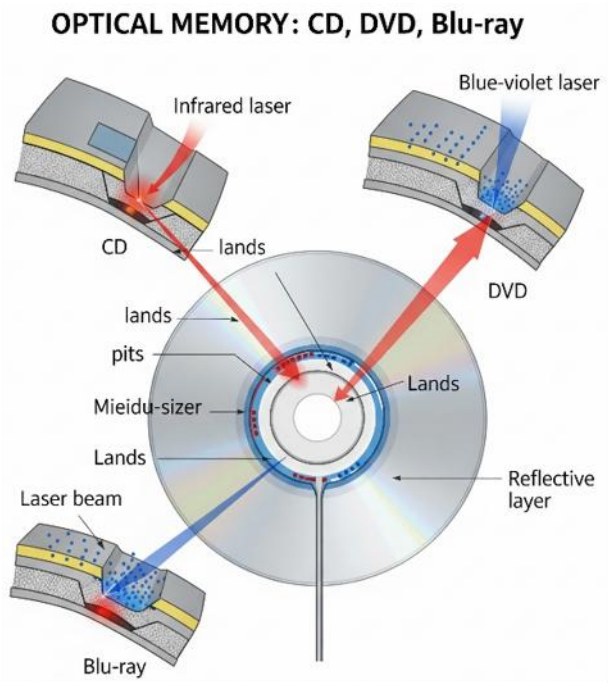


รูปที่ 7- 49 การเปรียบเทียบขนาดหลุมข้อมูล (Pits) และความลึกของชั้นบันทึกข้อมูลระหว่าง DVD และ Blu-ray

- **DVD:** ใช้เลเซอร์สีแดง (650 nm) ซึ่งมีความยาวคลื่นสั้นกว่าอินฟราเรดของ CD ทำให้สามารถสร้างหลุมและแทร็กที่เล็กลงได้
- **Blu-ray:** ใช้เลเซอร์สีน้ำเงิน-ม่วง (405 nm) ที่มีความยาวคลื่นสั้นที่สุดในบรรดาแผ่นดิสก์ ทำให้สามารถวางข้อมูลได้หนาแน่นยิ่งขึ้นไปอีกหลายเท่าตัว

ความเร็วพร้อมใช้งาน: ทั้ง DVD และ Blu-ray จะเริ่มทำงานเมื่อแผ่นหมุนถึงความเร็วเชิงเส้นที่กำหนด (3.49 m/s และ 4.9 m/s ตามลำดับ) ซึ่งจะใช้เวลาเพียงไม่กี่วินาที

8.1.10 ทำไมแผ่นถึงใช้ไม่ได้เมื่อเก็บไว้นานๆ ? แผ่นดิสก์ทุกประเภทจะเสื่อมสภาพไปตามกาลเวลาด้วยเหตุผลหลักดังนี้:



รูปที่ 7- 50 การเปรียบเทียบขนาดลำแสงเลเซอร์และพื้นที่จัดเก็บข้อมูล (Pits & Lands) ของสื่อประเภท CD, DVD และ Blu-ray

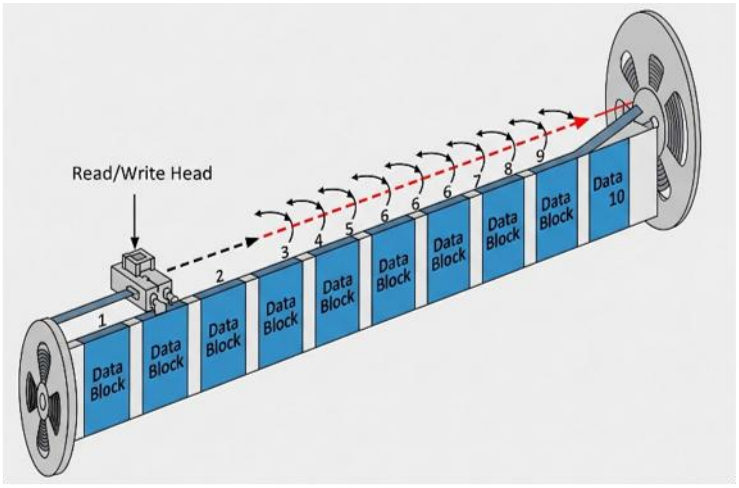
1. **การเสื่อมสภาพของชั้นข้อมูล:** ชั้นข้อมูลของแผ่น CD-R หรือ DVD-R ที่ทำจากสีย้อมอินทรีย์ (Organic dye) จะค่อยๆ สลายตัวไปตามการสัมผัสกับแสง ความร้อน และความชื้น

2. **การกัดกร่อนของชั้นโลหะ:** ชั้นสะท้อนแสงที่เป็นอะลูมิเนียมอาจเกิดการออกซิเดชัน (Oxidation) หรือการกัดกร่อนจากความชื้น ทำให้ไม่สามารถสะท้อนแสงได้อย่างถูกต้อง

3. **รอยขีดข่วนและฝุ่น:** การจัดเก็บที่ไม่เหมาะสมทำให้เกิดรอยขีดข่วนหรือคราบฝุ่นบนพื้นผิว ซึ่งจะรบกวนการอ่านของเลเซอร์และทำให้ข้อมูลเสียหายในที่สุด

8.2 หน่วยความจำแม่เหล็ก (Magnetic Memory): ใช้สื่อทางแม่เหล็กในการเก็บข้อมูล

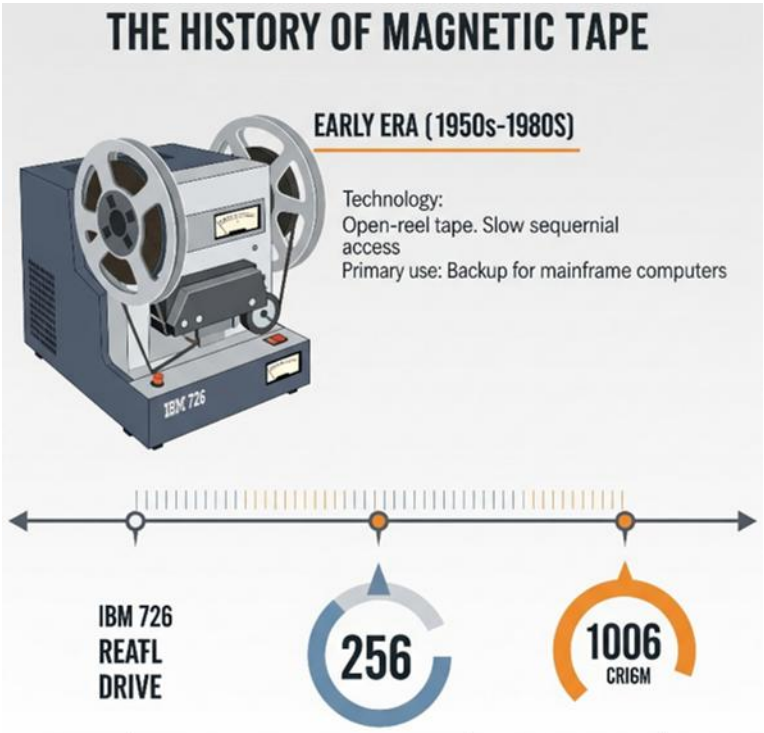
หน่วยความจำแม่เหล็กเป็นเทคโนโลยีการจัดเก็บข้อมูลแบบไม่ลบเลือน (Non-volatile) ที่เคยมีบทบาทสำคัญอย่างยิ่งในประวัติศาสตร์คอมพิวเตอร์ โดยใช้คุณสมบัติทางแม่เหล็กของสารเคลือบผิวในการบันทึกข้อมูลแบบไบนารี (0s และ 1s) ซึ่งแบ่งออกเป็นสองประเภทหลักตามลักษณะการเข้าถึงข้อมูล ได้แก่ **เทปแม่เหล็ก** และ **จานแม่เหล็ก**



รูปที่ 7- 51 การเข้าถึงข้อมูลแบบลำดับ (Sequential Access) บนสื่อประเภทเทปแม่เหล็ก

8.2.1. เทปแม่เหล็ก (Magnetic Tape)

ประวัติความเป็นมาและการพัฒนา เทปแม่เหล็กถูกคิดค้นขึ้นครั้งแรกในปี 1928 โดย Fritz Pfleumer เพื่อใช้บันทึกเสียง ก่อนที่ IBM จะนำมาใช้เป็นสื่อจัดเก็บข้อมูลคอมพิวเตอร์อย่างเป็นทางการในปี 1952 ด้วยเทปไดรฟ์ IBM 726 ซึ่งเป็นอุปกรณ์ขนาดใหญ่ที่ใช้เทปแบบม้วน นับเป็นยุคเริ่มต้นของการสำรองข้อมูลแบบดิจิทัล



รูปที่ 7- 52 วิวัฒนาการของเทปแม่เหล็กในยุคแรกเริ่ม

8.2.2 ยุคสมัยและการใช้งาน

ยุคเริ่มต้น: เทปม้วนขนาดใหญ่ (1950s-1980s)

จุดเริ่มต้นของการใช้เทปแม่เหล็กกับคอมพิวเตอร์อย่างจริงจัง

- **เทคโนโลยี:** เทปเป็นแบบม้วนขนาดใหญ่ (Open-Reel Tape) โดยต้องติดตั้งบนไดรฟ์ขนาดใหญ่ที่มักใช้ในคอมพิวเตอร์เมนเฟรม
- **วิธีการใช้งาน:** ข้อมูลถูกบันทึกและเข้าถึงแบบตามลำดับอย่างช้าๆ โดยใช้เป็นหลักในการสำรองข้อมูลประจำวันและย้ายข้อมูลระหว่างคอมพิวเตอร์



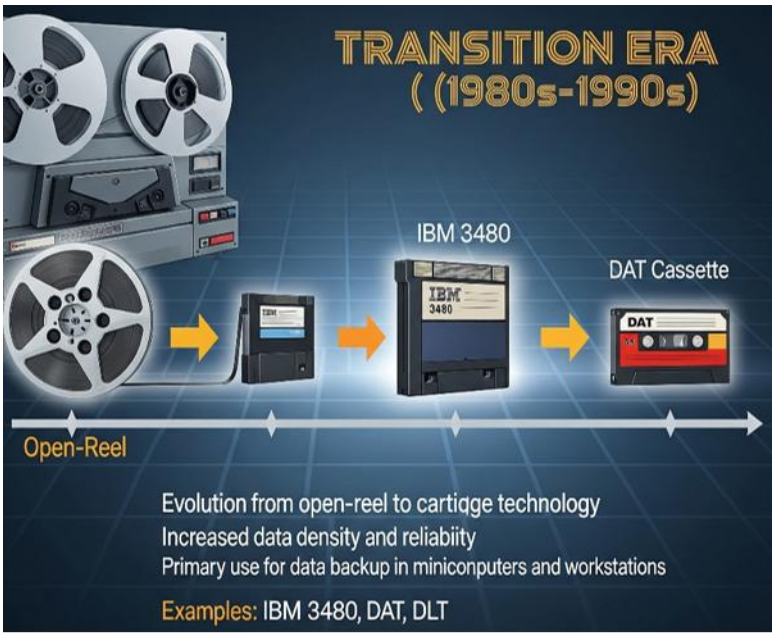
รูปที่ 7- 53 เครื่องขับเทปแม่เหล็กแบบม้วนใหญ่ (Open-Reel Tape Drive) ในยุคแรกเริ่ม

ตัวอย่างอุปกรณ์: IBM 726: เทปไดรฟ์ตัวแรกที่ IBM เปิดตัวในปี 1952 มีขนาดใหญ่และเป็นต้นแบบของการเก็บข้อมูลบนเทป

The IBM 726 introduced inexpensive coated plastic tape for data storage. Clever vacuum column tape buffers allowed the tape to start and stop quickly.

*เครื่อง **IBM 726** ได้เปิดตัวเทปพลาสติกเคลือบสารราคาไม่แพงสำหรับใช้ในการจัดเก็บข้อมูลคอมพิวเตอร์ นอกจากนี้ ยังมีคอลัมน์สุญญากาศ (Vacuum Column) ที่ชาญฉลาดทำหน้าที่เป็นบัฟเฟอร์ให้กับเทป ทำให้สามารถเริ่มและหยุดการทำงานได้อย่างรวดเร็ว*

8.2.3 ยุคเปลี่ยนผ่าน: ตลับเทปขนาดเล็ก (1980s-1990s) ในยุคนี้ เทคโนโลยีเริ่มมีการปรับปรุงให้ใช้งานง่ายและมีขนาดเล็กลง



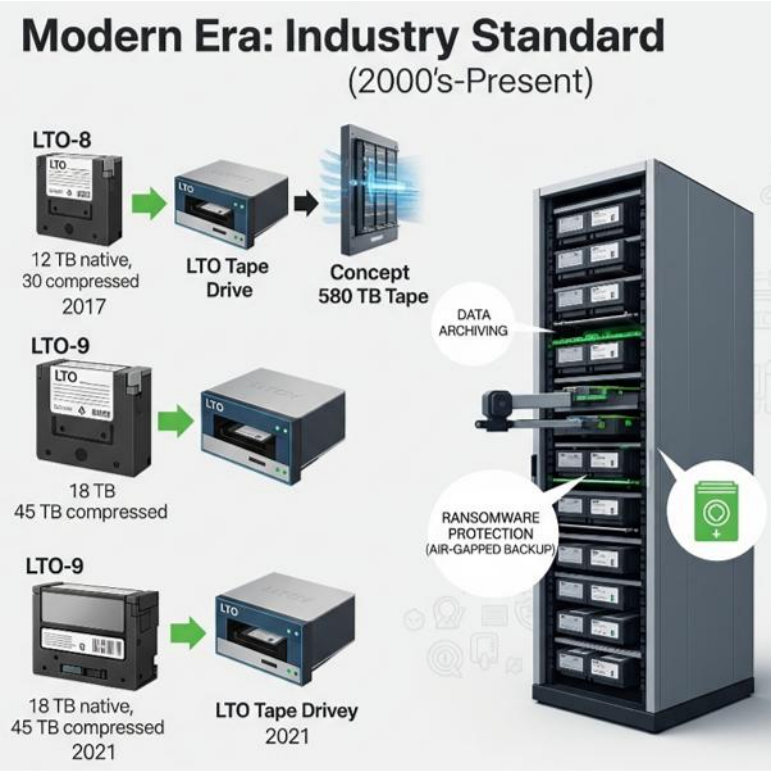
รูปที่ 7- 54 วิวัฒนาการของเทปแม่เหล็กในยุคเปลี่ยนผ่าน (Transition Era of Magnetic Tape)

- **เทคโนโลยี:** จากเทปม้วนขนาดใหญ่เปลี่ยนมาเป็น **ตลับเทป (Cartridge)** ที่มีขนาดเล็กลงและทนทานกว่า ทำให้การจัดการและการขนส่งทำได้ง่ายขึ้น
- **วิธีการใช้งาน:** เริ่มนำมาใช้ในคอมพิวเตอร์มินิคอมพิวเตอร์และเซิร์ฟเวอร์ขนาดเล็กเพื่อการสำรองข้อมูล
- **ตัวอย่างอุปกรณ์:**
  - **IBM 3480:** ตลับเทปแบบสี่เหลี่ยมที่กลายเป็นมาตรฐานใหม่แทนเทปม้วน
  - **DAT (Digital Audio Tape):** รูปแบบตลับเทปที่เดิมใช้บันทึกเสียง แต่ถูกนำมาใช้สำหรับสำรองข้อมูลคอมพิวเตอร์ขนาดเล็กด้วย

8.2.4 ยุคปัจจุบัน: มาตรฐานอุตสาหกรรม (2000s-ปัจจุบัน)

เทคโนโลยีเทปแม่เหล็กยังคงพัฒนาตามความต้องการด้านความจุและความเร็วในการสำรองข้อมูลขนาดใหญ่

- **8.2.4.1 เทคโนโลยี: LTO (Linear Tape-Open)** กลายเป็นมาตรฐานหลักของอุตสาหกรรม โดยมีการพัฒนาอย่างต่อเนื่องในแต่ละเจนเนอเรชันเพื่อเพิ่มความจุและความเร็ว
    - **LTO-8** สามารถบันทึกข้อมูลได้ถึง **12 TB** (ไม่บีบอัด)
    - **LTO-9** เพิ่มความจุเป็น **18 TB** (ไม่บีบอัด)
    - นอกจากนี้ยังมีการวิจัยพัฒนาเทปที่สามารถเก็บข้อมูลได้สูงถึง **580 TB**



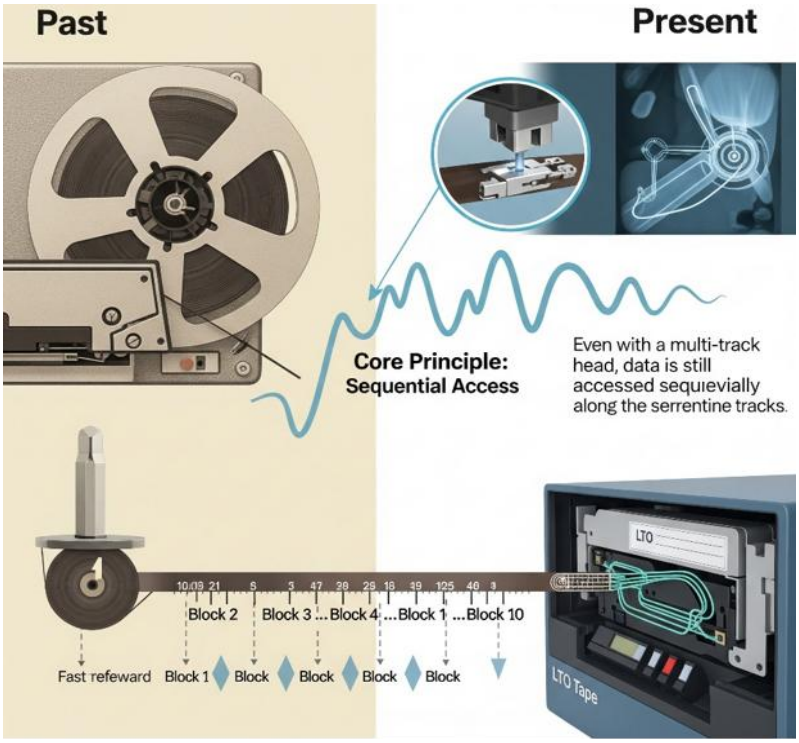
รูปที่ 7- 55 วิวัฒนาการของเทปแม่เหล็กในยุคปัจจุบัน (Modern Era of Magnetic Tape)

8.2.4.2 การสำรองข้อมูลระยะยาว (Archiving): เหมาะสำหรับการเก็บข้อมูลที่ไม่ค่อยได้ใช้งานแต่ต้องคงอยู่เป็นเวลานาน (เช่น ข้อมูลทางวิทยาศาสตร์, สื่อมัลติมีเดีย, ข้อมูลทางการแพทย์)

8.2.4.3 Cyber Resilience: ด้วยคุณสมบัติที่สามารถถอดเทปออกได้ (Air-Gapped Backup) ทำให้เป็นโซลูชันที่มีประสิทธิภาพสูงในการป้องกันการโจมตีจากมัลแวร์เรียกค่าไถ่ (Ransomware)

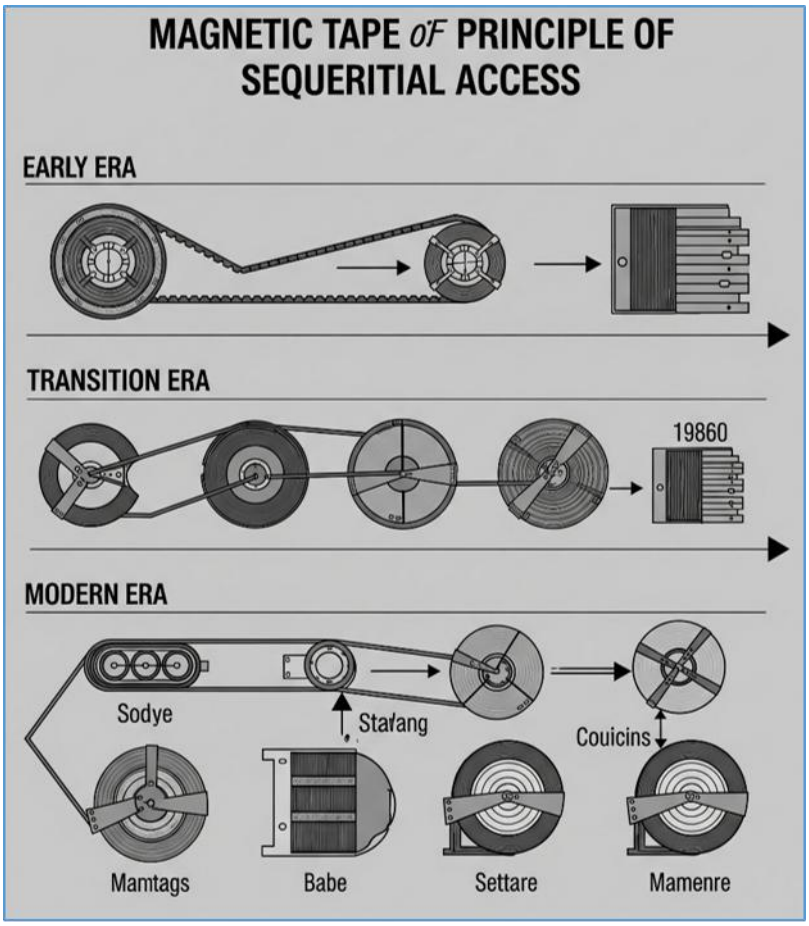
ตัวอย่างอุปกรณ์: LTO Tape Drives: ไดรฟ์สำหรับอ่านและเขียนตลับเทป LTO

LTO Tape Libraries: อุปกรณ์ขนาดใหญ่ที่บรรจุ LTO Tape Drive และตลับเทปจำนวนมาก พร้อมแขนหุ่นยนต์ที่สามารถเปลี่ยนตลับเทปอัตโนมัติเพื่อสำรองข้อมูลได้ต่อเนื่อง 24 ชั่วโมง



รูปที่ 7- 56 หลักการเข้าถึงข้อมูลแบบลำดับ (Sequential Access) ของเทปแม่เหล็กจากอดีตสู่ปัจจุบัน

8.2.4.4วิธีการเก็บข้อมูล (Sequential Access) การบันทึกข้อมูลบนเทปแม่เหล็กเป็นการบันทึกแบบ "ตามลำดับ" (Sequential) หมายความว่าหัวอ่าน/เขียนจะเคลื่อนที่ไปตามความยาวของเทปเท่านั้น การเข้าถึงข้อมูลจึงต้องเป็นไปตามลำดับเช่นกัน หากต้องการอ่านข้อมูลที่อยู่ท้ายเทปจะต้องกรอเทปผ่านข้อมูลทั้งหมดที่อยู่ก่อนหน้า .



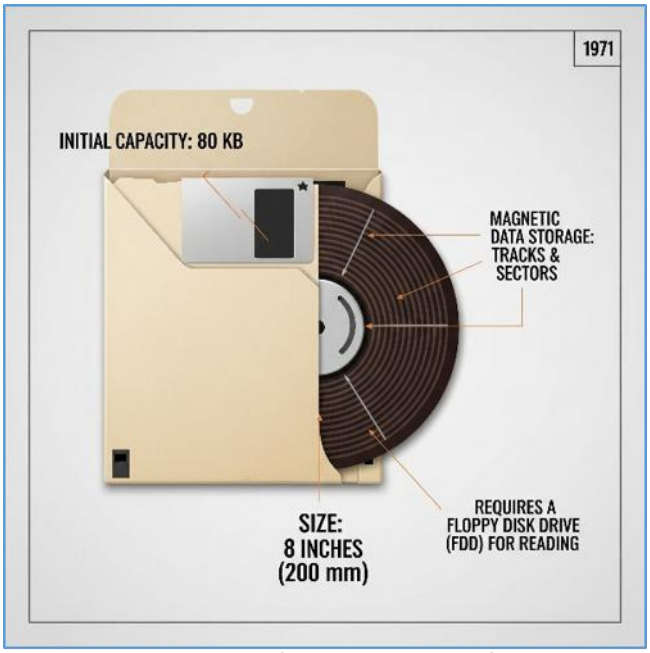
รูปที่ 7- 57 กลไกการขับเคลื่อนสายเทปแม่เหล็กในยุคต่าง ๆ (Magnetic Tape Drive Mechanisms)

8.2.5 ยุคเริ่มต้น (Early Era): แสดงให้เห็นภาพไดรฟ์เทปม้วนขนาดใหญ่ พร้อมกลไกของหัวอ่าน/เขียนที่ต้องเคลื่อนที่ไปตามความยาวของเทปเพื่อเข้าถึงข้อมูล

8.2.6 ยุคเปลี่ยนผ่าน (Transition Era): แสดงภาพที่เล็กลงมาในรูปแบบตลับเทป ซึ่งกลไกการเข้าถึงข้อมูลยังคงใช้หลักการเดียวกัน แต่มีขนาดกะทัดรัดขึ้น

8.2.7 ยุคปัจจุบัน (Modern Era): แสดงภาพตลับเทป LTO ที่ทันสมัย พร้อมเน้นย้ำให้เห็นถึงหลักการทำงานแบบ **Sequential Access** ที่ยังคงเป็นหัวใจหลักของเทคโนโลยีนี้ แม้ความจุจะเพิ่มขึ้นอย่างมหาศาลก็ตาม

9. ประวัติและการพัฒนาหน่วยความจำแม่เหล็กของแผ่นฟลอปปี้ดิสก์



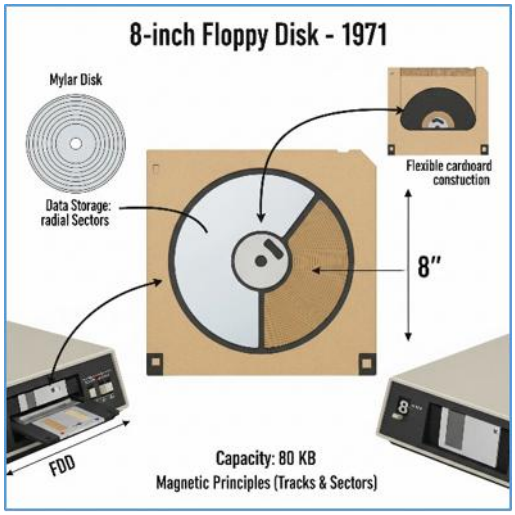
รูปที่ 7- 58 โครงสร้างและการจัดเก็บข้อมูลบนแผ่นฟล็อปปีดิสก์ขนาด 8 นิ้วในยุคแรกเริ่ม

ฟลอปปี้ดิสก์ถูกคิดค้นขึ้นครั้งแรกที่ **บริษัท IBM** โดยทีมวิศวกรที่นำโดย **อลัน ชูการ์ต (Alan Shugart)** ในปี **1967** โดยมีจุดประสงค์เพื่อใช้เป็นสื่อสำหรับโหลดซอฟต์แวร์ไปยังเมนเฟรมคอมพิวเตอร์

9.1 ยุคเริ่มต้น: ฟลอปปี้ดิสก์ขนาด 8 นิ้ว (1971)

ขนาด: เส้นผ่านศูนย์กลาง 8 นิ้ว มีลักษณะเป็นแผ่นกลมที่บรรจุในซองกระดาษแข็ง

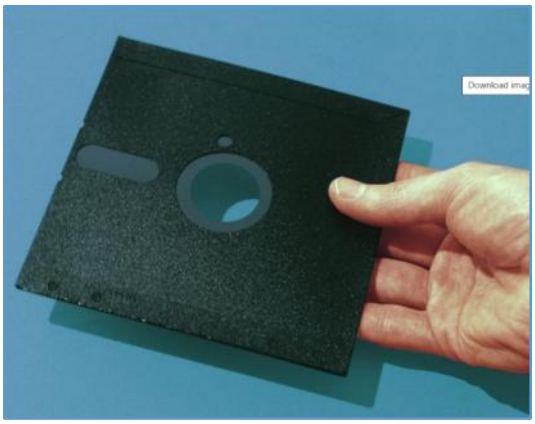
ความจุ: รุ่นแรกมีความจุเพียง 80 กิโลไบต์ (KB) ต่อมาพัฒนาเพิ่มเป็น 256 KB



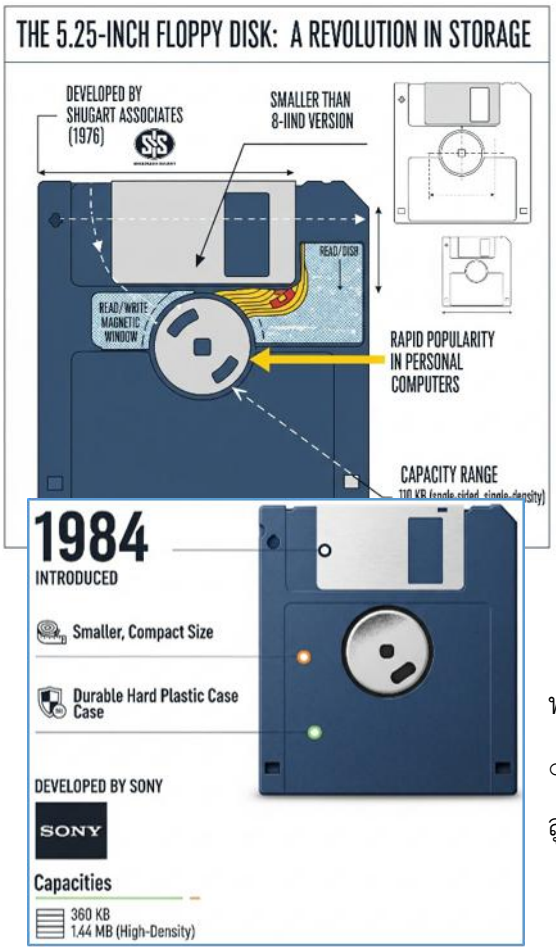
รูปที่ 7- 59 ส่วนประกอบและหลักการทำงานของฟล็อปปีดิสก์ขนาด 8 นิ้ว

วิธีการเก็บข้อมูล: ใช้หลักการทางแม่เหล็ก โดยหัวอ่าน/เขียนจะทำการบันทึกข้อมูลลงบนผิวแผ่นดิสก์ที่เป็นสารแม่เหล็กเป็นวงกลม (Track) และแบ่งเป็นส่วนย่อย (Sector)

ความจุ: รุ่นแรกมีความจุเพียง 80 กิโลไบต์ (KB) ต่อมาพัฒนาเพิ่มเป็น 256 KB



รูปที่ 7- 60 ลักษณะภายนอกและการหยิบจับแผ่นฟล็อปปีดิสก์ขนาด 8 นิ้ว



รูปที่ 7- 62 วิวัฒนาการสู่แผ่นฟล็อปปีดิสก์ขนาด 3.5 นิ้ว

อุปกรณ์การอ่าน: ต้องใช้กับเครื่องอ่านที่เรียกว่า **Floppy Disk Drive (FDD)**

9.1.2 ยุคกลาง: ฟลอปปี้ดิสก์ขนาด 5¼ นิ้ว (1976)

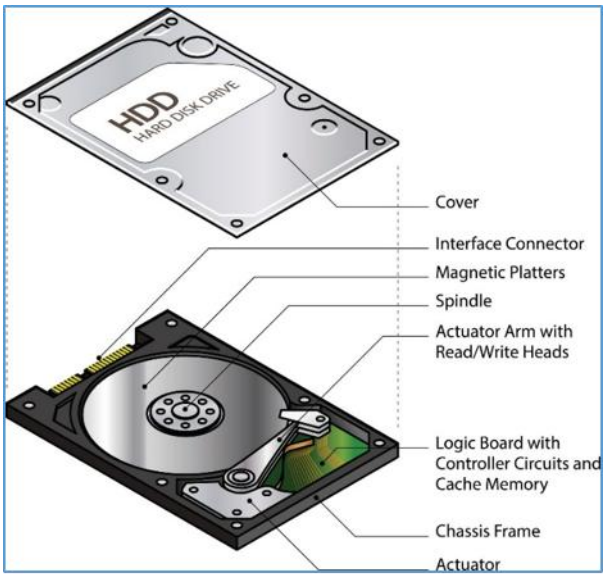
พัฒนาโดย: Shugart Associates ซึ่งก่อตั้งโดย Alan Shugart หลังจากออกจาก IBM

ขนาด: เล็กลงกว่าเดิมอย่างเห็นได้ชัด และได้รับความนิยมอย่างรวดเร็วในเครื่องคอมพิวเตอร์ส่วนบุคคล (PC)

ความจุ: เริ่มต้นที่ 110 KB และพัฒนาต่อเนื่องจนมีความจุสูงสุดถึง 1.2 MB ในรุ่น High-Density

9.1.3 ยุคทอง: ฟลอปปี้ดิสก์ขนาด 3½ นิ้ว (1984)

- **พัฒนาโดย:** บริษัท Sony
- **ขนาด:** มีขนาดเล็กและกะทัดรัดมากขึ้น บรรจุในเคสพลาสติกแข็งทำให้ทนทานกว่ารุ่นก่อนหน้า
- **ความจุ:** เริ่มต้นที่ 360 KB และรุ่นที่ได้รับความนิยมสูงสุดคือ **High-Density (HD)** ที่มีความจุ **1.44 MB**



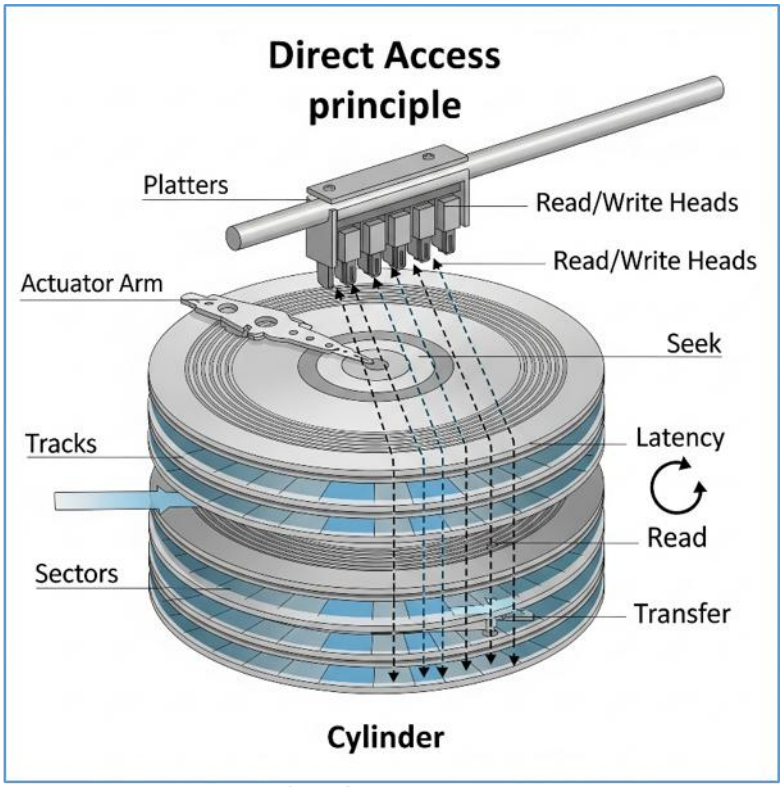
รูปที่ 7- 63 ส่วนประกอบและโครงสร้างภายในของฮาร์ดดิสก์ไดรฟ์

9.2 จานแม่เหล็ก (Magnetic Disk)

ประวัติความเป็นมาและพัฒนาการ จานแม่เหล็กนับเป็นวิวัฒนาการที่สำคัญที่ช่วยแก้ไขข้อจำกัดของเทปแม่เหล็ก **IBM** เป็นผู้บุกเบิกในเทคโนโลยีนี้ โดยได้เปิดตัว **Hard Disk Drive (HDD)** เชิงพาณิชย์ตัวแรกของโลกในปี **1956** ในชื่อ **IBM 305 RAMAC** ซึ่งเป็นตู้ขนาดใหญ่ที่ประกอบด้วยจานแม่เหล็กหลายแผ่น HDD ได้ปฏิวัติการจัดเก็บข้อมูลการเข้าถึงข้อมูลแบบสุ่ม โดยใช้หลักการเข้าถึงข้อมูลแบบ **"โดยตรง" (Direct Access)** แทนซึ่งเป็นจุดเด่นที่เหนือกว่าเทปแม่เหล็ก

ทฤษฎีการทำงานและการเก็บข้อมูล

HDD เป็นอุปกรณ์จัดเก็บข้อมูลแบบ **ไม่ลบเลือน** (Non-volatile) ที่ใช้หลักการของ **แม่เหล็ก** ในการบันทึกข้อมูลแบบดิจิทัล (0s และ 1s) ลงบนแผ่นจานที่เคลือบด้วยสารแม่เหล็ก



รูปที่ 7- 64 หลักการเข้าถึงข้อมูลโดยตรง (Direct Access) และโครงสร้างเชิงซ้อนของฮาร์ดดิสก์

- **9.2.1 ส่วนประกอบหลัก**: HDD ประกอบด้วย
    - **จานแม่เหล็ก (Platters)**: แผ่นโลหะกลมที่หมุนด้วยความเร็วสูงมาก
    - **หัวอ่าน/เขียน (Read/Write Heads)**: แขนเล็กๆ ที่ทำหน้าที่เปลี่ยนคุณสมบัติทางแม่เหล็กบนแผ่นจานเพื่อบันทึกข้อมูล หรืออ่านค่าเหล่านั้นกลับมา
    - **แขนขับ (Actuator Arm)**: แขนที่ควบคุมการเคลื่อนที่ของหัวอ่าน/เขียนให้เลื่อนเข้าออกไปในตำแหน่งที่ต้องการ
- **9.2.2 การบันทึกและอ่านข้อมูล**:

    เมื่อต้องการบันทึกข้อมูล หัวอ่าน/เขียนจะส่งสัญญาณแม่เหล็กเพื่อ **จัดเรียงขั้วแม่เหล็ก** บนพื้นผิวของจานให้เป็นตามข้อมูลดิจิทัล (0 หรือ 1) และเมื่อต้องการอ่านข้อมูล หัวอ่าน/เขียนจะตรวจจับทิศทางของขั้วแม่เหล็กที่จัดเรียงไว้ แล้วแปลงกลับเป็นข้อมูลดิจิทัล

9.2.3 วิธีการเข้าถึงข้อมูล (Direct Access)

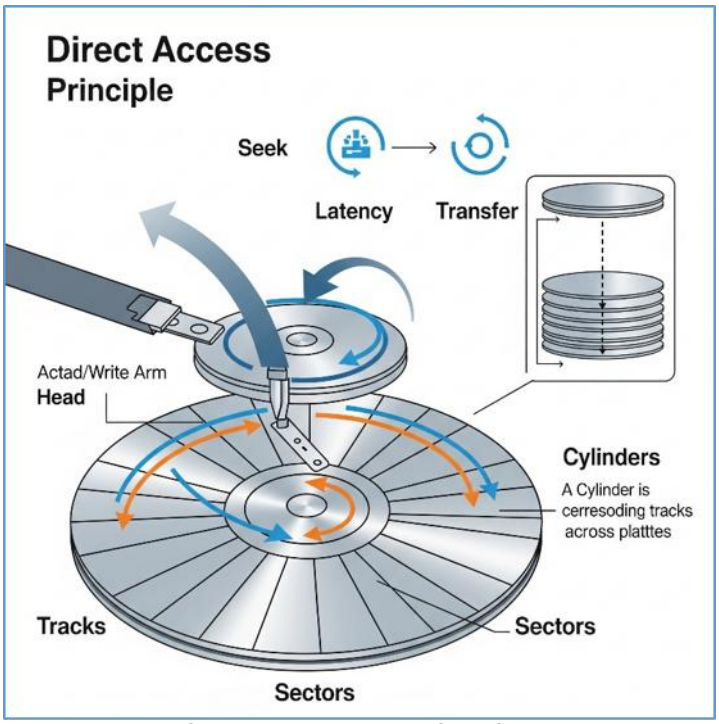
HDD ใช้หลักการเข้าถึงข้อมูลแบบ **"โดยตรง" (Direct Access)** ซึ่งเป็นจุดเด่นที่เหนือกว่าเทปแม่เหล็ก โดยการเข้าถึงข้อมูลจะเกิดขึ้นอย่างเป็นลำดับขั้นตอน:

9.2.4 การจัดเรียงข้อมูลบนจาน:

Tracks: ข้อมูลจะถูกบันทึกเป็นวงกลมหลายๆ วงบนจาน เรียกว่า **Track**

Sectors: แต่ละ Track จะถูกแบ่งออกเป็นส่วนย่อยๆ ที่เรียกว่า **Sector** ซึ่งเป็นพื้นที่เล็กที่สุดสำหรับการบันทึกข้อมูล

Cylinders: เมื่อมีหลายจาน ข้อมูลที่อยู่ใน Track เดียวกันในแต่ละจานจะถูกเรียกว่า **Cylinder** ซึ่งช่วยให้หัวอ่าน/เขียนสามารถเข้าถึงข้อมูลในทุกๆ จานพร้อมกันได้โดยไม่ต้องเคลื่อนที่



รูปที่ 7- 65 ขั้นตอนและลำดับการเข้าถึงข้อมูลโดยตรงบนจานแม่เหล็ก (Direct Access Mechanism)

9.2.5 ขั้นตอนการเข้าถึงข้อมูล:

- **การเคลื่อนที่ของหัวอ่าน (Seek)**: หัวอ่าน/เขียนจะเคลื่อนที่ไปยัง **Track** ที่ต้องการ ซึ่งใช้เวลามากที่สุดในการเข้าถึงข้อมูล
- **การหมุนของจาน (Latency)**: จานแม่เหล็กจะหมุนจนกระทั่ง **Sector** ที่ต้องการเลื่อนมาอยู่ใต้หัวอ่าน/เขียน
- **การอ่าน/เขียนข้อมูล (Transfer)**: เมื่อหัวอ่าน/เขียนอยู่ในตำแหน่งที่ถูกต้องก็จะเริ่มกระบวนการอ่านหรือเขียนข้อมูล .

9.3 หลักการทำงานของหัวอ่าน/เขียน (Read/Write Heads)

ใน HDD ที่มีหลายแผ่นจาน แต่ละแผ่นจะมีหัวอ่าน/เขียนแยกกัน โดยหัวอ่าน/เขียนทั้งหมดจะติดตั้งอยู่บน **แขนขับ (Actuator Arm)** ชิ้นเดียวกัน ทำให้หัวอ่าน/เขียนทุกตัวเคลื่อนที่ไปพร้อมกันเสมอ เมื่อแขนขับเคลื่อนที่ไปยังตำแหน่งใดก็ตาม หัวอ่าน/เขียนทุกตัวจะอยู่ในตำแหน่งที่สอดคล้องกันบนจานแม่เหล็กของตัวเอง

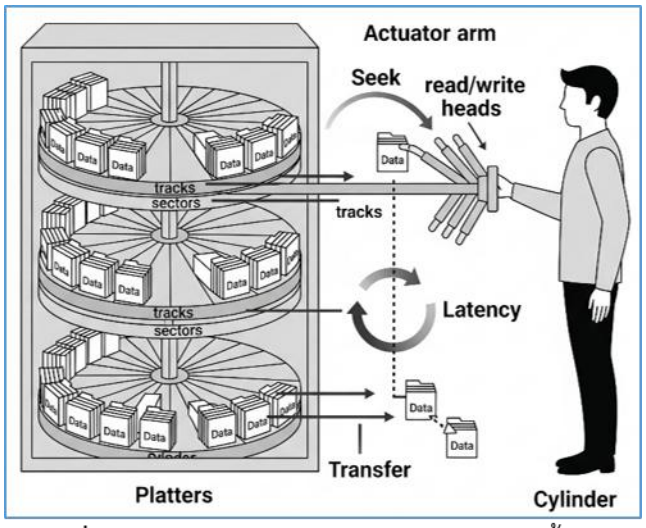
9.3.1 การเข้าถึงข้อมูลด้วยแนวคิด Cylinder

- **Cylinder (ไซลินเดอร์)**: คือชุดของ **Tracks** ที่อยู่ในแนวตั้งเดียวกันบนทุกๆ แผ่นจานของฮาร์ดดิสก์ เมื่อแขนขับเคลื่อนที่ไปยังตำแหน่งหนึ่ง หัวอ่าน/เขียนทุกตัวก็จะถูกวางอยู่เหนือ Track ที่สอดคล้องกันบนแต่ละแผ่นจาน จึงทำให้เกิดเป็นรูปทรงกระบอกในจินตนาการขึ้นมา .
- **วิธีการเข้าถึงข้อมูล**: เมื่อระบบต้องการอ่านหรือเขียนข้อมูล ระบบจะสั่งให้หัวอ่าน/เขียนทุกตัวเคลื่อนที่ไปยัง Cylinder ที่ถูกต้องในคราวเดียว เมื่อหัวอ่าน/เขียนทุกตัวอยู่ในตำแหน่งที่ต้องการแล้ว ระบบจะสลับไปใช้หัวอ่าน/เขียนตัวที่อยู่เหนือข้อมูลที่ต้องการเท่านั้น โดยไม่ต้องขยับแขนขับอีก

ด้วยวิธีนี้ ฮาร์ดดิสก์จึงสามารถเข้าถึงข้อมูลบนแผ่นจานทั้งหมดได้อย่างรวดเร็ว โดยไม่จำเป็นต้องเคลื่อนที่แขนขับซ้ำไปซ้ำมา ทำให้ลดเวลาในการค้นหาข้อมูล (Seek Time) และเพิ่มประสิทธิภาพในการทำงานโดยรวม

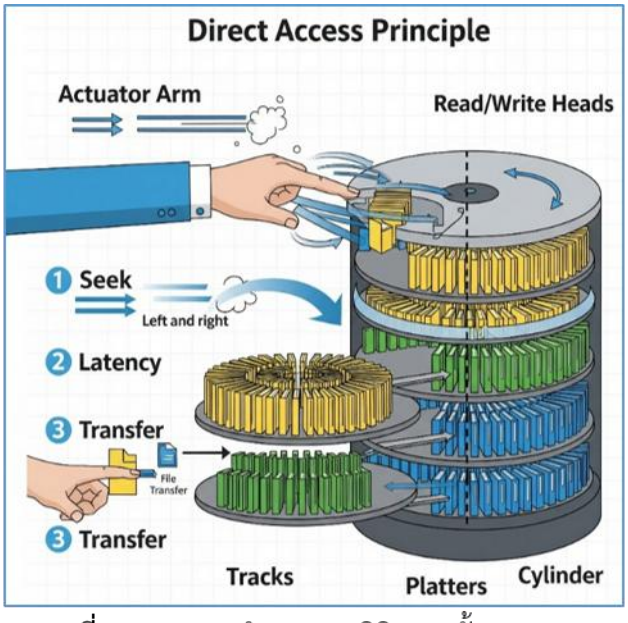
9.3.2 ยกตัวอย่างการทำงานเปรียบเทียบกับ Hard Disk :

ลองจินตนาการถึง **ตู้เก็บเอกสารแบบหมุนได้ (Rotary Filing Cabinet)** ที่มีหลายๆ ชั้น โดยแต่ละชั้นมีแฟ้มเอกสารจัดเรียงอยู่



รูปที่ 7- 66 แบบจำลองเปรียบเทียบกลไกการค้นหาและเข้าถึงข้อมูลบนฮาร์ดดิสก์

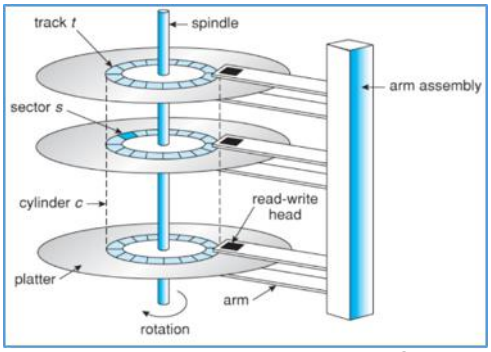
- **แผ่นจาน (Platters)**: เปรียบเสมือน **แต่ละชั้นของตู้เก็บเอกสาร** ซึ่งมีเอกสาร (ข้อมูล) จัดเก็บอยู่
- **แขนขับ (Actuator Arm)**: เปรียบเสมือน **แขนของคน** ที่จะยื่นเข้าไปเพื่อหยิบแฟ้มเอกสาร โดยแขนนี้จะสามารถเคลื่อนที่เข้า-ออกได้
- **หัวอ่าน/เขียน (Read/Write Heads)**: เปรียบเสมือน **นิ้วมือ** ของคุณ ที่อยู่ติดกับแขน โดยคุณมีนิ้วมือหลายนิ้วเพื่อหยิบแฟ้มที่อยู่แต่ละชั้นพร้อมกัน
- **Track (แทร็ก)**: เปรียบเสมือน **แถวของแฟ้ม** ที่เรียงกันเป็นวงกลมบนแต่ละชั้น
- **Sector (เซกเตอร์)**: เปรียบเสมือน **แฟ้มเอกสารแต่ละเล่ม** ที่อยู่ในแถวนั้น
- **Cylinder (ไซลินเดอร์)**: เปรียบเสมือน **ชุดของแฟ้มที่อยู่ในแนวตั้งเดียวกัน** ในทุกๆ ชั้น (เช่น แฟ้มที่อยู่แถวที่ 5 ของทุกชั้น)



รูปที่ 7- 67 แบบจำลองสามมิติแสดงขั้นตอนการเข้าถึงข้อมูลโดยตรงบนฮาร์ดดิสก์

9.3.3 วิธีการเข้าถึงข้อมูล (Direct Access)

1. **การเคลื่อนที่ของแขน (Seek)**: เมื่อคุณต้องการหาแฟ้มใดแฟ้มหนึ่ง คุณจะ **เลื่อนแขนของคุณ** เข้าไปในตำแหน่งที่ถูกต้องของแถวแฟ้ม (Track) ที่คุณต้องการ
2. **การหมุนของตู้ (Latency)**: ตู้เก็บเอกสารก็จะ **หมุน** จนกระทั่งแฟ้มที่คุณต้องการมาอยู่ตรงหน้าคุณพอดี
3. **การอ่าน/เขียน (Transfer)**: เมื่อแฟ้มมาอยู่ตรงหน้าแล้ว คุณจะ **ใช้นิ้วมือหยิบ** แฟ้มนั้นขึ้นมา



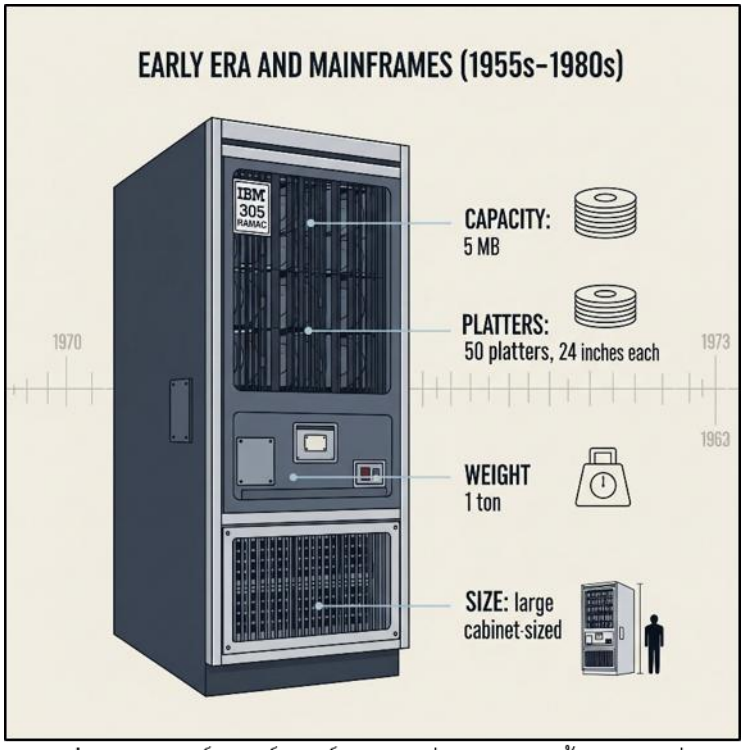
รูปที่ 7- 68 การจัดกลุ่มเซกเตอร์ข้อมูลเป็นคลัสเตอร์ บนแผ่นจานแม่เหล็ก



รูปที่ 7- 69 โครงสร้างทางกายภาพจริงและการจัดวางชิ้นส่วนภายในของฮาร์ดดิสก์

9.3.4 ความสำคัญของ Cylinder

หากคุณต้องการหาแฟ้มในชั้นที่ 3 และชั้นที่ 7 ที่อยู่ในแถวเดียวกัน (Track เดียวกัน) แทนที่จะเลื่อนแขนเข้าไปหาแฟ้มในชั้นที่ 3 ก่อน แล้วค่อยเลื่อนแขนออกและเลื่อนกลับเข้าไปใหม่เพื่อหาแฟ้มในชั้นที่ 7 **คุณสามารถยื่นแขนเข้าไปเพียงครั้งเดียว** แล้วใช้นิ้วที่อยู่ตรงชั้น 3 และชั้น 7 เพื่อหยิบแฟ้มทั้งสองขึ้นมาพร้อมกันได้เลย นี่คือหลักการของ **Cylinder** ที่ทำให้ HDD สามารถเข้าถึงข้อมูลบนแผ่นจานหลายแผ่นได้อย่างรวดเร็ว โดยไม่จำเป็นต้องเคลื่อนที่แขนขับซ้ำไปซ้ำมา ทำให้ลดเวลาในการค้นหาข้อมูล (Seek Time) และเพิ่มประสิทธิภาพในการทำงานโดยรวม



รูปที่ 7- 70 ฮาร์ดดิสก์ไดรฟ์ยุคแรกเริ่มและการใช้งานในเครื่องเมนเฟรม (IBM 305 RAMAC)

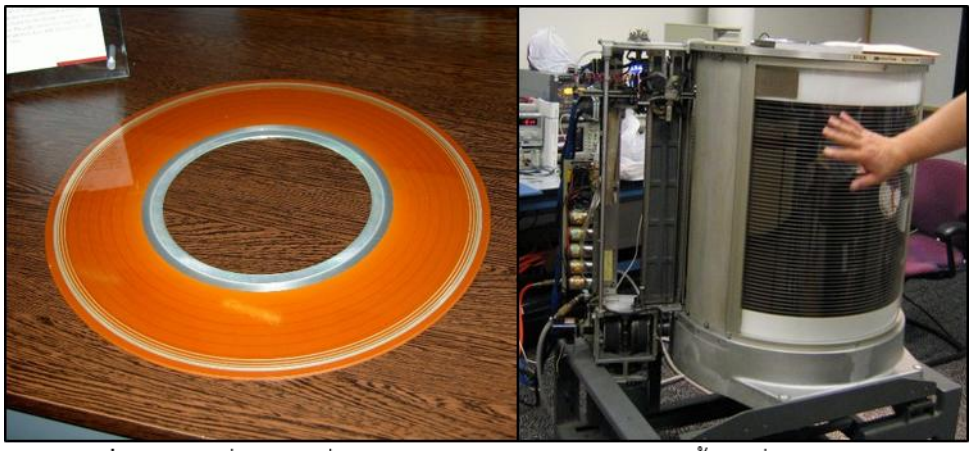
<u>9.4 ยุคบุกเบิกและเมนเฟรม (1950s - 1980s)</u> นี่คือจุดเริ่มต้นของ HDD ที่มีขนาดใหญ่ใช้ในวงการธุรกิจเป็นหลัก

ต้นกำเนิดและการพัฒนา: IBM เปิดตัว HDD เชิงพาณิชย์ตัวแรกของโลกคือ **IBM 305 RAMAC** ในปี **1956** ซึ่งมีขนาดใหญ่เท่าตู้เสื้อผ้า .

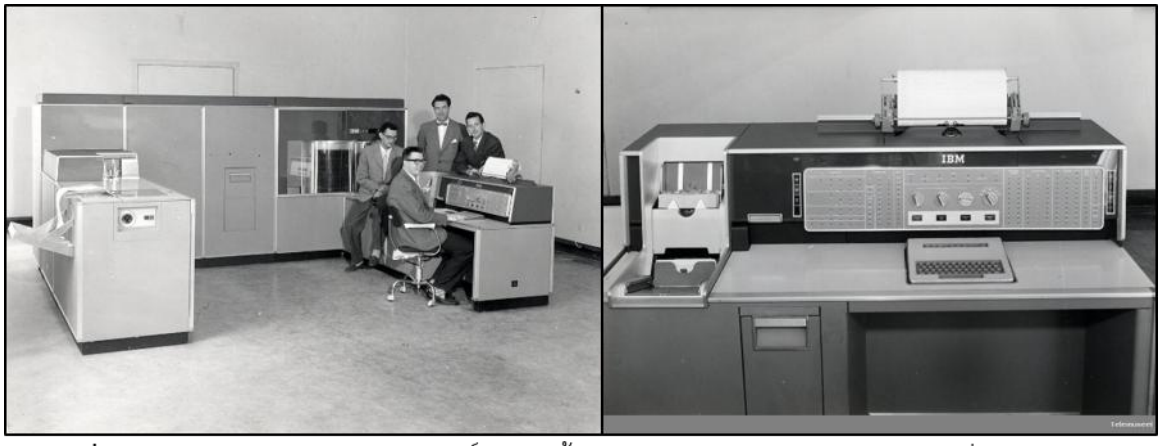
ความจุเริ่มต้น: มีความจุเพียง **5 MB** เท่านั้น โดยใช้จานขนาด 24 นิ้ว จำนวน 50 แผ่น

ขนาด: มีขนาดใหญ่มาก (ประมาณ 68 นิ้ว x 60 นิ้ว x 29 นิ้ว) และมีน้ำหนักถึง 1 ตัน

- **การเชื่อมต่อ:** ในยุคนี้ยังไม่มีมาตรฐานการเชื่อมต่อที่ชัดเจน การเชื่อมต่อระหว่าง HDD กับคอมพิวเตอร์เมนเฟรมจะเป็นแบบเฉพาะทางและซับซ้อนมาก



รูปที่ 7- 71 แผ่นจานแม่เหล็กและชุดจานหมุนขนาด 24 นิ้ว เครื่อง IBM 305



รูปที่ 7- 72 ลักษณะภายนอกของไดรฟ์จัดเก็บข้อมูล IBM 350 และระบบควบคุมเครื่อง IBM 305

9.4.1 ยุคคอมพิวเตอร์ส่วนบุคคลและมาตรฐาน (1980s - 2000s)



รูปที่ 7- 73 ลักษณะภายนอกของฮาร์ดดิสก์ขนาด 5.25 นิ้ว และฮาร์ดดิสก์ขนาด 3.5 นิ้ว

HDD เริ่มมีขนาดเล็กลงและกลายเป็นอุปกรณ์หลักสำหรับคอมพิวเตอร์ส่วนบุคคล

การพัฒนา: ขนาดของ HDD เริ่มลดลงอย่างรวดเร็ว โดยมีขนาดมาตรฐานที่ 5.25 นิ้ว และ 3.5 นิ้ว ตามมา

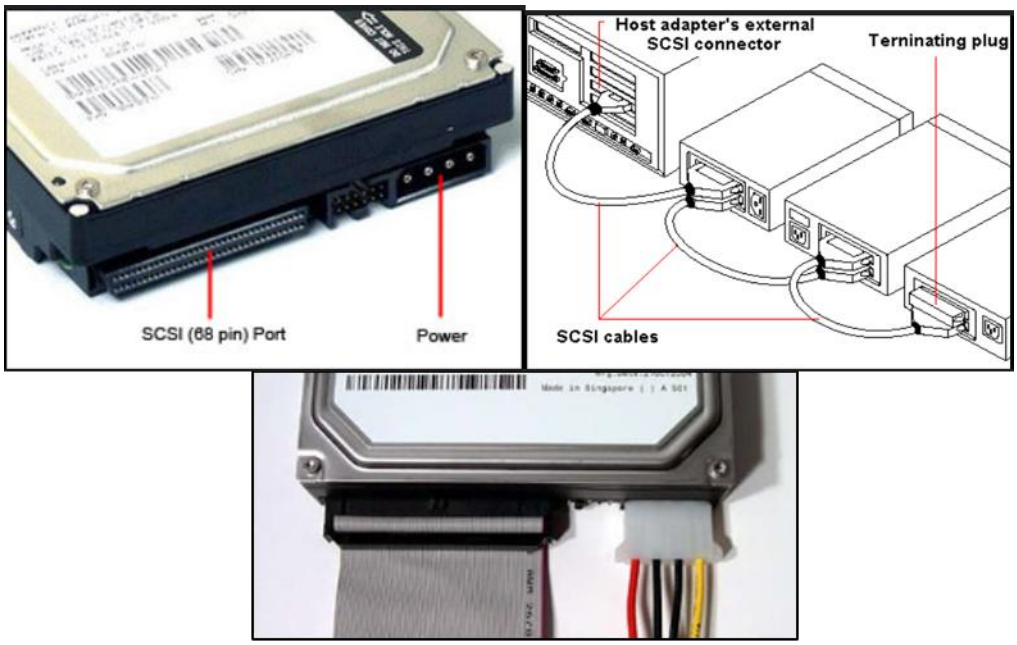
ความจุ: ความจุเพิ่มขึ้นอย่างก้าวกระโดดจากระดับ MB ไปสู่ระดับ GB และเริ่มมีการใช้เทคโนโลยี **MR head (Magnetoresistive head)** เพื่อเพิ่มความหนาแน่นในการบันทึกข้อมูล

ความเร็ว: ความเร็วรอบหมุนของจานเริ่มเข้าสู่มาตรฐานที่ 5,400 RPM และ 7,200 RPM

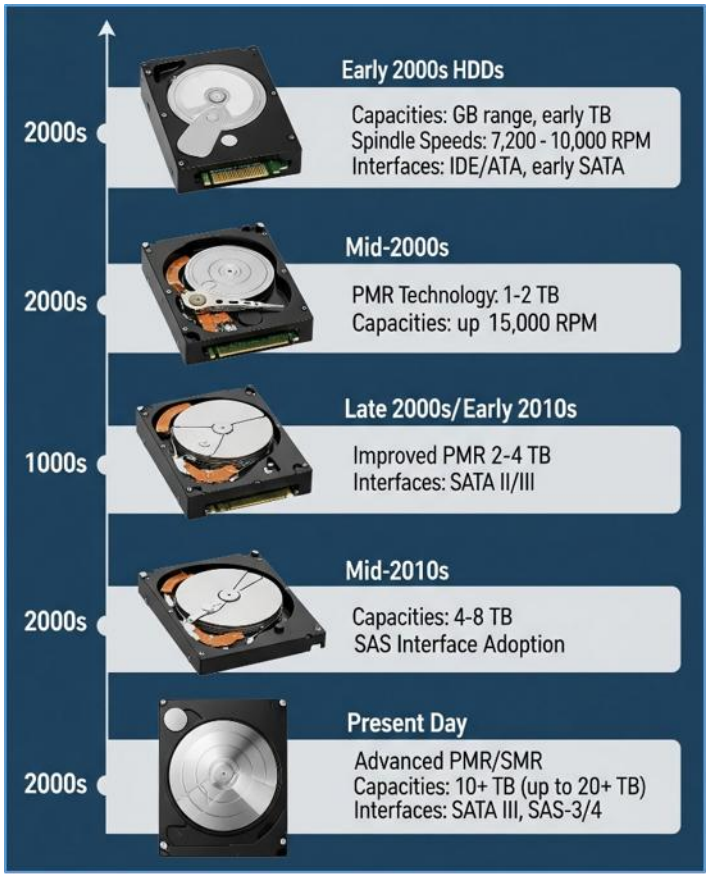
การเชื่อมต่อ: มีการพัฒนาอินเทอร์เฟซมาตรฐาน เช่น **IDE (Integrated Drive Electronics)** และ **SCSI (Small Computer System Interface)** ทำให้การเชื่อมต่อ HDD กับเมนบอร์ดทำได้ง่ายขึ้น .

พอร์ต IDE: ใช้สายแพ (Ribbon Cable) ที่มี 40 หรือ 80 เส้น ใช้สำหรับทั่วไปในคอมพิวเตอร์ส่วนบุคคล

พอร์ต SCSI: ใช้สายแพที่มีจำนวนขามากกว่า (เช่น 50, 68 หรือ 80 ขา) ใช้ในเซิร์ฟเวอร์หรือเวิร์กสเตชันที่ต้องการประสิทธิภาพสูง และรองรับการเชื่อมต่ออุปกรณ์หลายตัวพร้อมกัน



รูปที่ 7- 74 ลักษณะพอร์ตและการเชื่อมต่อฮาร์ดดิสก์แบบอินเทอร์เฟซ SCSI



รูปที่ 7- 75 ไทม์ไลน์วิวัฒนาการและเทคโนโลยีของฮาร์ดดิสก์ไดรฟ์ตั้งแต่ปี 2000 ถึงปัจจุบัน นำแคชขนาดใหญ่มาใช้เพื่อเพิ่มความเร็วในการเข้าถึงข้อมูล

ขนาด: ขนาดมาตรฐานที่นิยมในปัจจุบันคือ 3.5 นิ้ว สำหรับ Desktop และ 2.5 นิ้วสำหรับ Notebook และ Server

การเชื่อมต่อ: มาตรฐานการเชื่อมต่อที่ครองตลาดคือ **SATA (Serial ATA)** ซึ่งเป็นวิวัฒนาการของ IDE และ **SAS (Serial Attached SCSI)**

9.4.2 ยุคปัจจุบันและอนาคต (2000s - ปัจจุบัน) HDD ยังคงเป็นตัวเลือกหลักสำหรับงานที่ต้องการความจุสูง แต่เทคโนโลยีก็พัฒนาไปเพื่อรองรับความต้องการที่หลากหลายมากขึ้น

การพัฒนา: ความจุเพิ่มขึ้นอย่างต่อเนื่องด้วยเทคโนโลยีใหม่ๆ เช่น **PMR (Perpendicular Magnetic Recording)** และ **SMR (Shingled Magnetic Recording)** ที่ช่วยให้สามารถบันทึกข้อมูลได้หนาแน่นขึ้นในพื้นที่เท่าเดิม

ความจุ: HDD สำหรับผู้บริโภคทั่วไปมีความจุสูงถึง 20 TB และสำหรับองค์กรอาจสูงถึง 28 TB

ความเร็ว: ความเร็วรอบหมุนสูงสุดที่ 15,000 RPM (สำหรับ Enterprise HDD) และมีการ

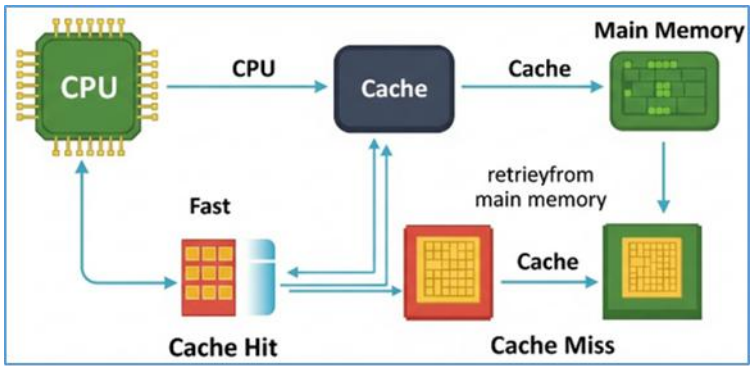


รูปที่ 7- 76 การเปรียบเทียบพอร์ตและลักษณะขั้วต่อสายสัญญาณของฮาร์ดดิสก์แบบ IDE (PATA) และ SATA



รูปที่ 7- 77 ลักษณะสายสัญญาณและหัวล็อกสำหรับเชื่อมต่อฮาร์ดดิสก์มาตรฐาน SATA II และ SATA III

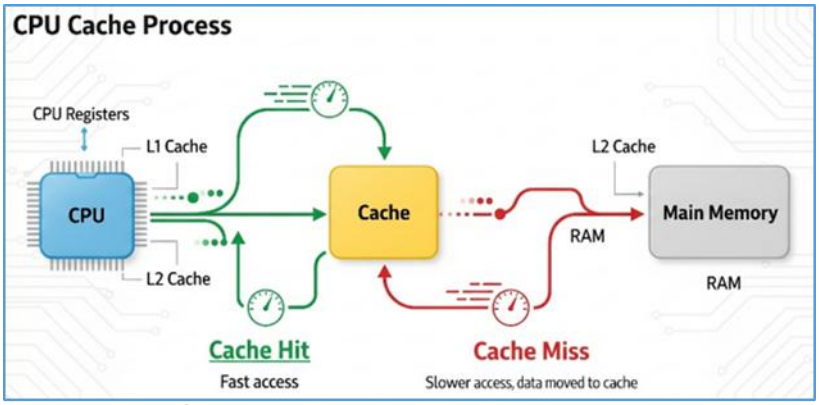
10. การทำงานของหน่วยความจำแคช (CACHE MEMORY)



รูปที่ 7- 78 กลไกการทำงานของหน่วยความจำแคชและการเกิด Cache Hit และ Cache Miss

หน่วยความจำแคชเป็นองค์ประกอบที่สำคัญมากในโครงสร้างระบบคอมพิวเตอร์ ทำหน้าที่เป็นหน่วยความจำขนาดเล็กที่มีความเร็วสูงมาก สร้างขึ้นจากชิป SRAM (Static RAM) ซึ่งเร็วกว่าหน่วยความจำหลักที่ใช้ DRAM เป็นอย่างมาก เนื่องจากมีขนาดเล็กกว่าและมีราคาแพงกว่า ทำให้สามารถเก็บข้อมูลได้จำกัด

วัตถุประสงค์หลักของหน่วยความจำแคช คือการลดช่องว่างความเร็วระหว่าง CPU กับ หน่วยความจำหลัก (Main Memory) โดยการเก็บสำเนาของข้อมูลและคำสั่งที่ CPU มีการเรียกใช้งานบ่อย ๆ หรือมีแนวโน้มจะใช้ในลำดับถัดไป เมื่อ CPU ต้องการข้อมูลใด ๆ จะตรวจสอบที่หน่วยความจำแคชก่อนเป็นอันดับแรก การทำเช่นนี้ช่วยให้ CPU สามารถเข้าถึงข้อมูลที่ต้องการได้ทันทีโดยไม่ต้องเสียเวลารอการถ่ายโอนข้อมูลจากหน่วยความจำหลัก ทำให้การประมวลผลเป็นไปอย่างรวดเร็วและมีประสิทธิภาพมากขึ้นการทำงานของหน่วยความจำแคชมี 2 แบบหลัก ดังนี้



รูปที่ 7- 79 ขั้นตอนกระบวนการประมวลผลของหน่วยความจำแคชในสถานะ Cache Hit และ Cache Miss

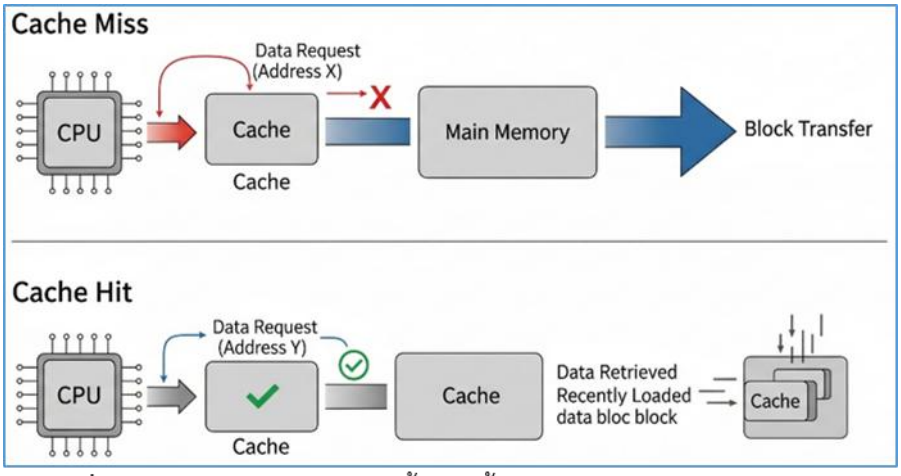
10.1 Cache Hit (พบข้อมูลในแคช):

เมื่อ CPU ต้องการข้อมูลในตำแหน่งใด และตรวจสอบแล้ว พบข้อมูลนั้นอยู่ในหน่วยความจำแคช นี่คือสถานการณ์ที่ต้องการ เพราะ CPU สามารถอ่านข้อมูลนั้นไปประมวลผลได้ทันทีโดยไม่ต้องรอจึงทำให้การทำงานเร็วขึ้น

10.2 Cache Miss (ไม่พบข้อมูลในแคช):

เมื่อ CPU ต้องการข้อมูลในตำแหน่งใด แต่ตรวจสอบแล้ว ไม่พบข้อมูลนั้นในหน่วยความจำแคช สถานการณ์นี้เรียกว่า "Cache Miss" ซึ่งจะทำให้ วงจรควบคุม (Cache Controller) ต้องเริ่มกระบวนการถ่ายโอนข้อมูล โดยจะไปดึงข้อมูลทั้ง "บล็อก" (Block) จากหน่วยความจำหลักมาเก็บไว้ในหน่วยความจำแคช หลังจากที่ข้อมูลถูกถ่ายโอนมาแล้ว CPU จึงจะสามารถเข้าถึงและประมวลผลข้อมูลที่ต้องการได้ ซึ่งกระบวนการนี้จะใช้เวลานานกว่าแบบ Cache Hit เนื่องจากต้องมีการเข้าถึงหน่วยความจำหลักซึ่งมีความเร็วต่ำกว่า

10.3 การโอนย้ายข้อมูลระหว่างหน่วยความจำแคชกับหน่วยความจำหลัก

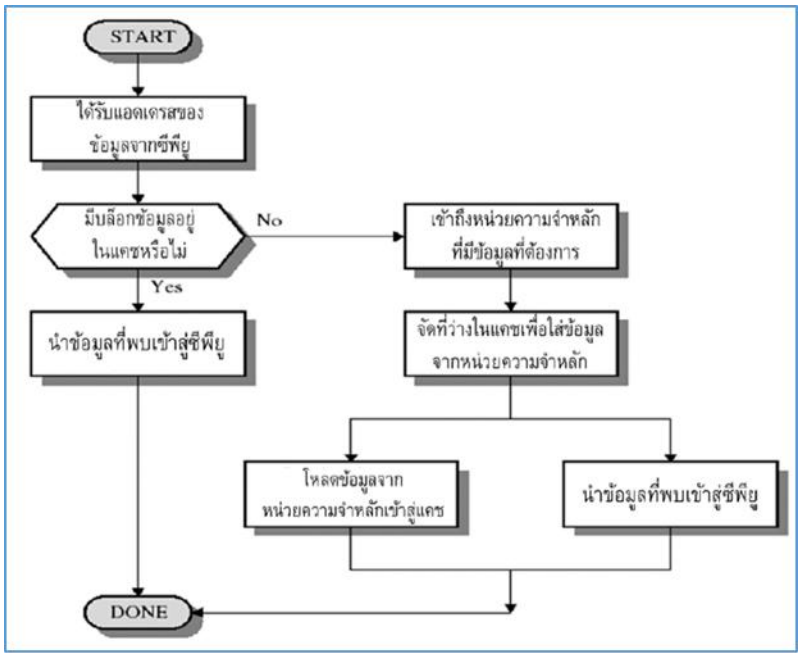


รูปที่ 7- 80 แผนภาพจำลองการร้องขอข้อมูลในสภาวะ Cache Miss และ Cache Hit ของ CPU

กระบวนการนี้เป็นหัวใจสำคัญของการทำงานของหน่วยความจำแคช เมื่อ CPU ต้องการเข้าถึงข้อมูล จะมีการตรวจสอบตามขั้นตอนดังนี้:

10.3.1หน่วยความจำหลัก (Main Memory): จะถูกแบ่งออกเป็นส่วนย่อยๆ ที่เรียกว่า **บล็อก (Block)** ซึ่งแต่ละบล็อกจะมีขนาดที่แน่นอนและมีแอดเดรสที่ระบุตำแหน่งเฉพาะ (เช่น มีแอดเดรสจำนวน n เส้น ทำให้สามารถอ้างอิงตำแหน่งได้ถึง 2^n ตำแหน่ง)

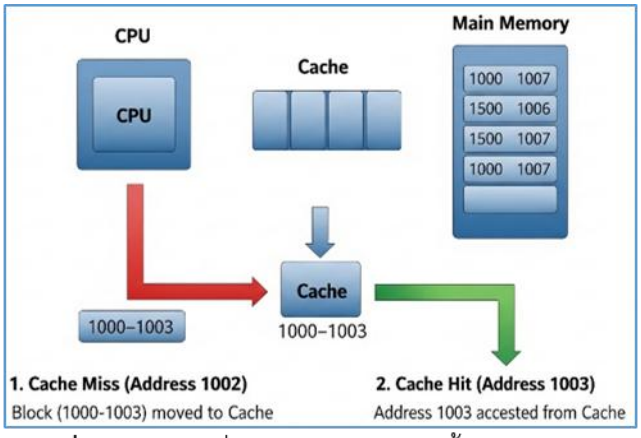
10.3.2 หน่วยความจำแคช (Cache Memory): จะถูกแบ่งออกเป็น **แถว (Cache Line)** แต่ละแถวในแคชจะมีขนาดเท่ากับหนึ่งบล็อกของหน่วยความจำหลัก และมี **Tag bit** หรือ **แท็กบิต** กำกับอยู่ ซึ่งแท็กบิตนี้จะทำหน้าที่เป็นป้ายบอกว่าข้อมูลที่อยู่ในแคชนั้นมาจากบล็อกใดในหน่วยความจำหลัก เพื่อให้ CPU สามารถตรวจสอบได้ว่าข้อมูลที่ต้องการอยู่ในแคชหรือไม่



รูปที่ 7- 81 ผังงาน (Flowchart) แสดงขั้นตอนและกระบวนการทำงานของหน่วยความจำแคช

เมื่อเกิด **Cache Miss** (CPU ไม่พบข้อมูลที่ต้องการในแคช) วงจรควบคุม (Cache Controller) จะทำงานโดยการไปดึงข้อมูลทั้งบล็อกจากหน่วยความจำหลักที่ตำแหน่งนั้นมาใส่ไว้ในหน่วยความจำแคชทั้งบล็อก และทำการอัปเดตแท็กบิตของแถวในแคชให้ตรงกับตำแหน่งแอดเดรสของข้อมูลที่ถูกนำมา

ตัวอย่างประกอบ: หน่วยความจำหลักมีแอดเดรส 1000 – 1007และอหน่วยความจำแคชมีเพียง 1 แถว และแต่ละบล็อกมีขนาด 4 เวิร์ด (สมมติให้เวิร์ดละ 1 ไบต์)



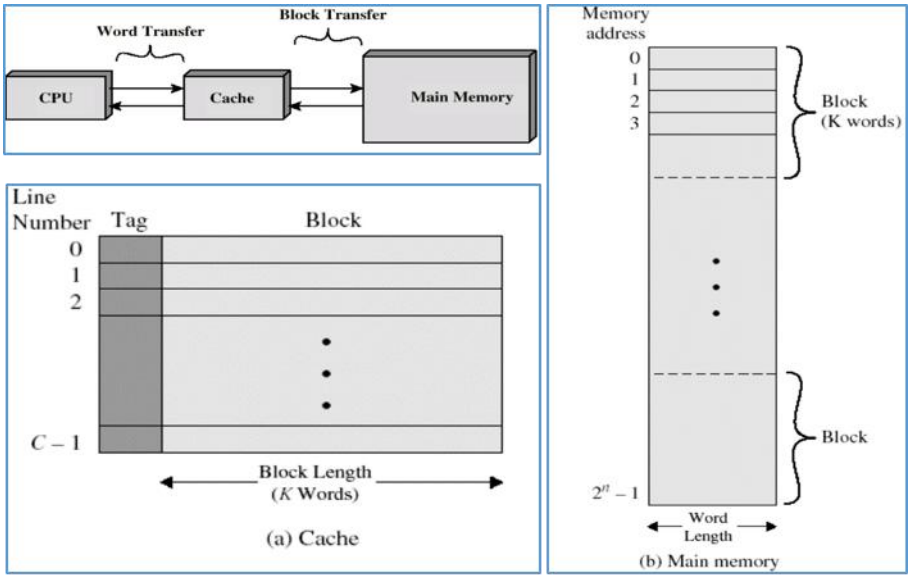
รูปที่ 7- 82 ตัวอย่างการจัดสรรบล็อกข้อมูลแอดเดรสในสภาวะ Cache Miss และ Cache Hit

ขั้นตอนที่ 1: CPU ต้องการอ่านข้อมูลจากแอดเดรส 1002 แต่ข้อมูลยังไม่มีอยู่ในแคช (เกิด Cache Miss)

1.1 วงจรควบคุมจะมองว่าแอดเดรส 1002 อยู่ในบล็อกใดของหน่วยความจำหลัก

1.2 ในตัวอย่างนี้ แอดเดรส 1002 อยู่ในบล็อกที่ประกอบด้วยข้อมูลตั้งแต่แอดเดรส 1000, 1001, 1002 และ 1003

1.3 วงจรควบคุมจะทำการ **โอนย้ายข้อมูลทั้งบล็อกนี้** จากหน่วยความจำหลัก (ข้อมูลจากแอดเดรส 1000 ถึง 1003) มาเก็บไว้ในแถวเดียวของหน่วยความจำแคช และตั้งค่า **Tag bit** ให้สอดคล้องกับตำแหน่งบล็อกนี้ในหน่วยความจำหลัก



รูปที่ 7- 83 โครงสร้างสถาปัตยกรรมและการแมปตำแหน่งข้อมูลระหว่าง Cache และ Main Memory

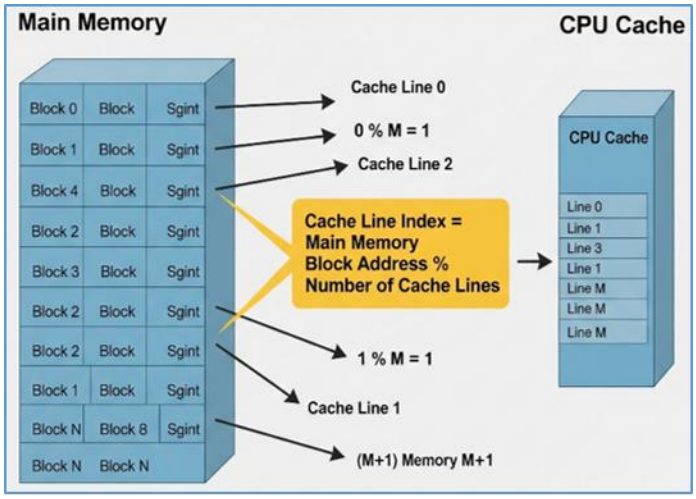
ขั้นตอนที่ 2: เมื่อ CPU ต้องการอ่านข้อมูลจากแอดเดรส 1003 ในภายหลัง CPU จะตรวจสอบที่หน่วยความจำแคชก่อน เนื่องจากข้อมูลจากบล็อกเดียวกับแอดเดรส 1003 (คือบล็อกที่มีแอดเดรส 1000-1003) ได้ถูกโหลดเข้ามาในแคชแล้วในขั้นตอนที่ 1 CPU จึง **พบข้อมูลที่ต้องการในแคช (เกิด Cache Hit)** และสามารถอ่านข้อมูลจากแอดเดรส 1003 ได้ทันทีโดยไม่ต้องไปที่หน่วยความจำหลัก

ตัวอย่างนี้แสดงให้เห็นว่าการดึงข้อมูลจากหน่วยความจำหลักไม่ได้ดึงมาแค่ข้อมูลที่ต้องการในขณะนั้น แต่จะดึงมาทั้งบล็อกเพื่อเพิ่มโอกาสในการเกิด Cache Hit ในการเข้าถึงข้อมูลที่อยู่ใกล้เคียงกันในอนาคต ทำให้การทำงานโดยรวมของระบบเร็วขึ้นอย่างมีนัยสำคัญ

10.4 ฟังก์ชั่นในการเชื่อมโยงข้อมูล (MAPPING FUNCTION) : ทำหน้าที่คอยจัดการการทำงานต่าง ๆ ระหว่าง CPU หน่วยความจำหลัก และหน่วยความจำแคช การเชื่อมโยงข้อมูลจะมี 3 คุณสมบัติ คือ

10.4.1 การวางข้อมูล (Placement) นำบล็อกของข้อมูลวางลงในแคช

10.4.1.1 การเชื่อมโยงแบบโดยตรง (Direct Mapping) วิธีนี้เป็นวิธีที่ง่ายที่สุด โดยแต่ละบล็อกในหน่วยความจำหลักจะมีตำแหน่งที่กำหนดไว้ตายตัวในแคชเพียงที่เดียวเท่านั้น การระบุตำแหน่งจะใช้สูตรทางคณิตศาสตร์ง่ายๆ คือ **ตำแหน่งแคช = ที่อยู่บล็อกในหน่วยความจำหลัก ระบุ จำนวนแถวในแคช** หากแคชมีขนาด 8 แถว บล็อกที่ 9 จากหน่วยความจำหลักจะถูกวางที่แถวที่ 1 ข้อเสียของวิธีนี้คือ หากบล็อกหลายบล็อกจากหน่วยความจำหลักถูกแมปมาที่แถวเดียวกันในแคช จะเกิดการแย่งพื้นที่กัน ทำให้ประสิทธิภาพลดลง เปรียบเสมือนการจองห้องพักในโรงแรมที่ระบุห้องไว้ตายตัว โดยใช้สูตรคำนวณจากตำแหน่งแอดเดรสของบล็อกข้อมูลในหน่วยความจำหลัก



รูปที่ 7- 84 หลักการคำนวณดัชนีและการแมปตำแหน่งข้อมูลแบบตรง (Direct Mapping) ของแคช

ข้อดี: ออกแบบและควบคุมได้ง่าย

ข้อเสีย: อาจเกิดปัญหา Cache Miss ได้บ่อย หาก CPU มีการเรียกใช้ข้อมูลจากหลายบล็อกที่ถูกแมปไปยังตำแหน่งเดียวกันในแคช

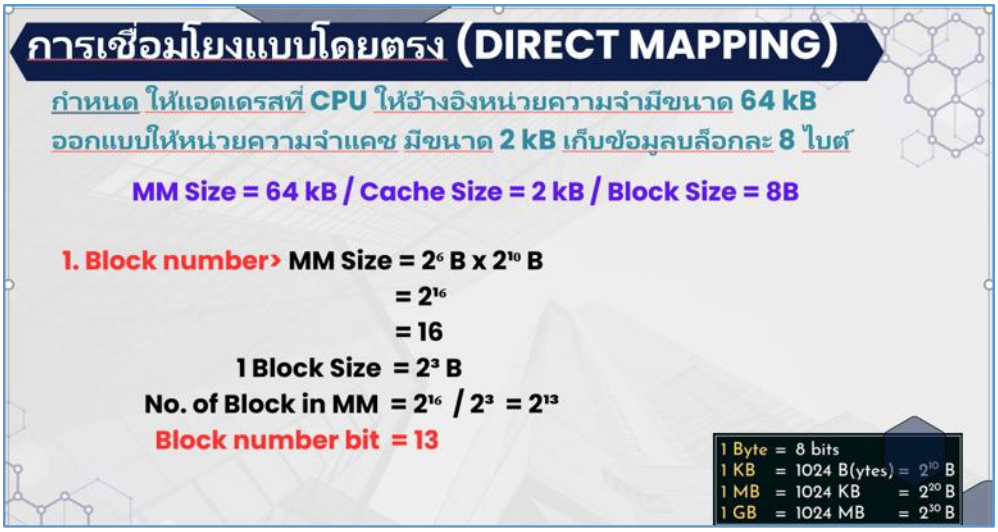
ตัวอย่าง: สมมติว่ามี Main Memory 8 บล็อก และ Cache 4 บรรทัด

- Main Memory Block 0 จะถูกแมปไปที่ Cache Line 0 (0 mod 4 = 0)
- Main Memory Block 4 จะถูกแมปไปที่ Cache Line 0 (4 mod 4 = 0)
- Main Memory Block 1 จะถูกแมปไปที่ Cache Line 1 (1 mod 4 = 1)
- Main Memory Block 5 จะถูกแมปไปที่ Cache Line 1 (5 mod 4 = 1)

ดังนั้น หากข้อมูลจาก Block 0 ถูกโหลดอยู่ใน Cache Line 0 และต่อมา CPU ต้องการข้อมูลจาก Block 4 ซึ่งต้องถูกวางในตำแหน่งเดียวกัน จะทำให้เกิดการเขียนทับข้อมูลเดิมและเกิด Cache Miss ขึ้น

ตัวอย่างการคำนวน: การเชื่อมโยงแบบ **Direct Mapping** เป็นวิธีการจัดการแคชที่ง่ายที่สุด โดยแต่ละบล็อกของหน่วยความจำหลัก (Main Memory) จะสามารถถูกแมปไปยังบล็อกที่เฉพาะเจาะจงเพียงบล็อกเดียวในหน่วยความจำแคช (Cache Memory) ได้เท่านั้น ในการคำนวณ เราจะแบ่งแอดเดรสของหน่วยความจำหลักออกเป็น 3 ส่วนหลัก ได้แก่:

1. **Block Offset (จำนวนบิตสำหรับระบุตำแหน่งข้อมูลภายในบล็อก)**: ใช้สำหรับระบุตำแหน่งของข้อมูล (ไบต์) ภายในบล็อกนั้นๆ
2. **Cache Index (จำนวนบิตสำหรับระบุตำแหน่งบล็อกในแคช)**: ใช้สำหรับระบุว่าบล็อกข้อมูลจากหน่วยความจำหลักจะถูกนำไปเก็บไว้ที่ตำแหน่งใดในแคช
3. **Tag (จำนวนบิตสำหรับระบุบล็อก)**: ใช้สำหรับระบุว่าข้อมูลที่อยู่ในแคชบล็อกนั้นมาจากบล็อกใดของหน่วยความจำหลัก



รูปที่ 7- 85 ตัวอย่างการคำนวณจำนวนบิตของบล็อกข้อมูลในการเชื่อมโยงแบบโดยตรง (Direct Mapping)

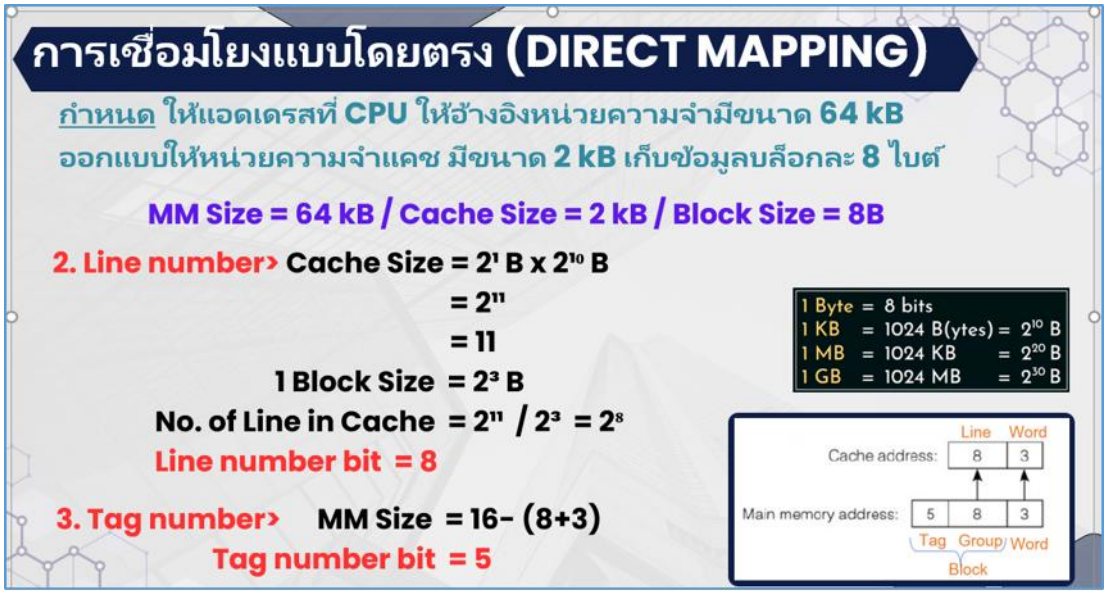
การคำนวณตามโจทย์

โจทย์: กำหนดให้ Main Memory Size = 64 kB, Cache Size = 2 kB, และ Block Size = 8 B

1. การหาจำนวนบิตทั้งหมดของแอดเดรสในหน่วยความจำหลัก (Total Address Bits)

- ขนาดของหน่วยความจำหลัก (MM Size) = 64 Kbyte และ (1 Kbyte =1024 Byte และ 2^{10} Byte)
- จากนั้น 64 Kbyte = 64×2^{10} Byte = $2^6 \times 2^{10}$ Byte = 2^{16} Byte

ดังนั้น จำนวนบิตของแอดเดรสทั้งหมดคือ 16 บิต (เพราะ 2^{16} คือขนาดพื้นที่ที่สามารถอ้างอิงได้ด้วย 16 บิต)



รูปที่ 7- 86 ตัวอย่างการคำนวณบิตสำหรับ Line number และ Tag number ในการเชื่อมโยงแบบโดยตรง

2. การคำนวณ Block Offset

ขนาดของบล็อก (Block Size) = 8 Byte (1 Byte = 8 Bit)

8 Byte=2^3 Bit

*ดังนั้น บิตสำหรับ Block Offset คือ **3 บิต** (เพื่อใช้ในการระบุตำแหน่งภายในบล็อกที่มีขนาด 8 ไบต์)*

3. การคำนวณ Cache Index

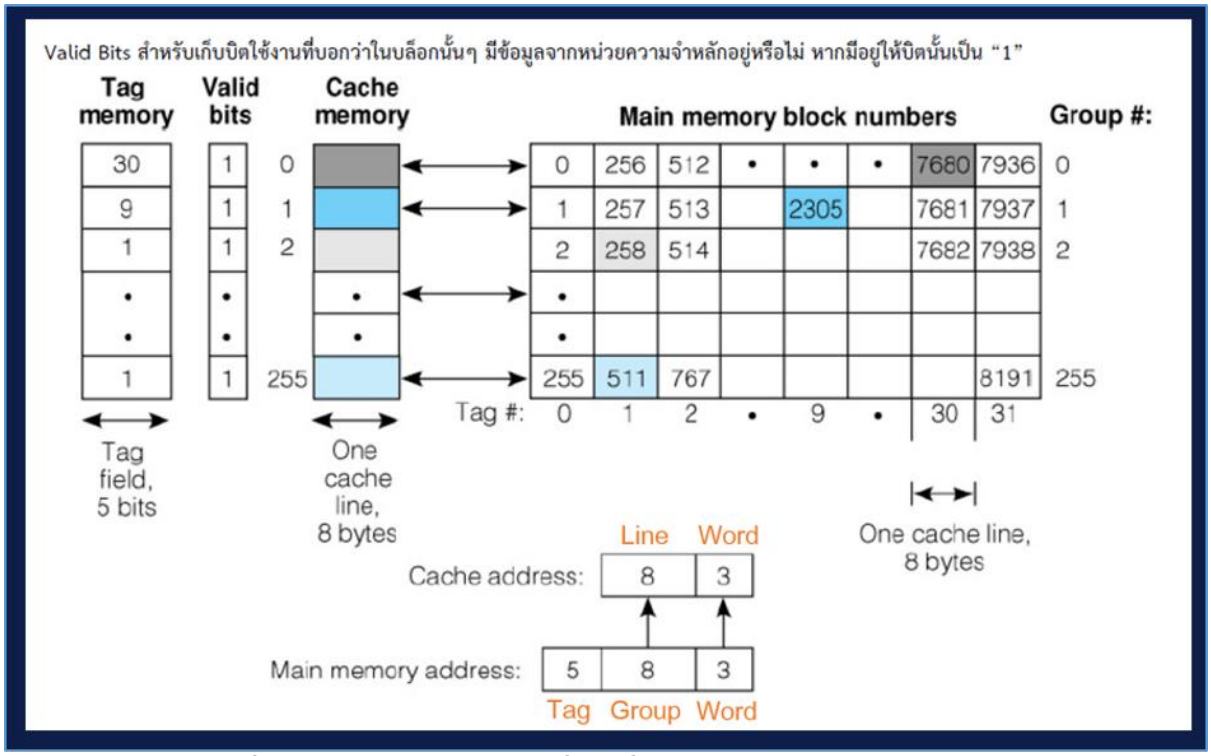
- ขนาดของแคช (Cache Size) = 2 kByte=2048 Byte
- ขนาดของบล็อก (Block Size) = 8 Byte
- จำนวนบล็อกทั้งหมดในแคช = Block Size หารด้วย Cache Size

2048 Byte หารด้วย 8 Byte = เท่ากับ 256 บล็อก และ 256 = 2^8

*ดังนั้น จำนวนบิตสำหรับ Cache Index คือ **8 บิต** (เพื่อใช้ในการระบุตำแหน่งของบล็อกในแคชทั้ง 256 บล็อก)*

4. การคำนวณ Tag

- จำนวนบิตทั้งหมดของแอดเดรส = **16 บิต**
- จำนวนบิตสำหรับ Block Offset = **3 บิต**
- จำนวนบิตสำหรับ Cache Index = **8 บิต**
- จำนวนบิตสำหรับ Tag = (จำนวนบิตทั้งหมด) ลบ (Block Offset) ลบ (Cache Index)
- จำนวนบิตสำหรับ Tag = 16–3–8=5 บิต
- ดังนั้น จำนวนบิตสำหรับ Tag คือ **5 บิต**



รูปที่ 7- 87 โครงสร้างตารางและการตรวจสอบข้อมูลด้วย Valid Bit ในระบบ Direct Mapping Cache

สรุปผลการคำนวณ แอดเดรสของหน่วยความจำหลักจะถูกแบ่งออกเป็น 3 ส่วนดังนี้:

1.จำนวน Tag: 5 บิต **2.จำนวน Cache Index:** 8 บิต **3.จำนวน Block Offset:** 3 บิต

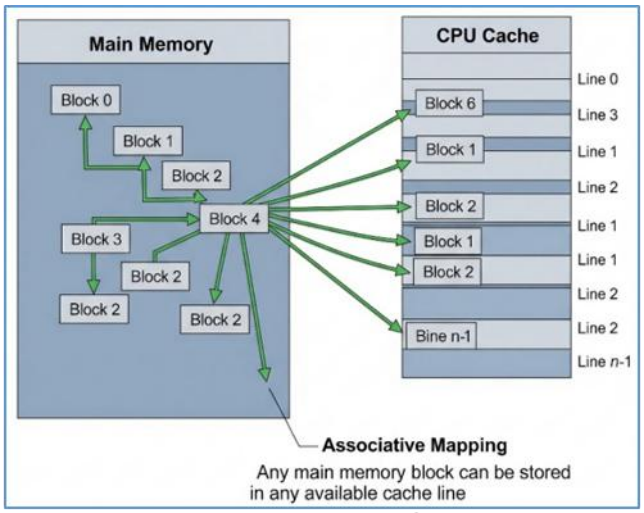
สรุปข้อดีการเชื่อมโยงแบบ Direct Mapping

เป็นวิธีการออกแบบระบบที่ง่ายที่สุด การอ้างถึงหน่วยความจำหลักทำได้ง่ายด้วยการใช้ Tag

สรุปข้อเสียการเชื่อมโยงแบบ Direct Mapping

มีการกำหนด Cache Line ที่แน่นอน ในกรณีข้อมูลอยู่ในกลุ่มเดียวกัน และมีการเรียกใช้งานบ่อยๆ ทำให้ต้องมีการโอนย้ายข้อมูลระหว่างหน่วยความจำแคช และหน่วยความจำหลักบ่อยขึ้นหาก CPU ต้องการข้อมูลในบล็อก 512 และมีข้อมูลอยู่ในแคช ต่อมาต้องการข้อมูลในบล็อก 7680 จะไม่พบข้อมูลในแคช ระบบต้องสลับข้อมูลจากบล็อก 7680 มาแทนที่บล็อก 512 ที่อยู่ในแคช

10.5 การเชื่อมโยงแบบสัมพันธ์ (Associative Mapping)



รูปที่ 7- 88 หลักการแมปตำแหน่งข้อมูลแบบเกี่ยวพัน (Associative Mapping) ของหน่วยความจำแคช

วิธีนี้มีความยืดหยุ่นสูงที่สุด เพราะบล็อกใดๆ จากหน่วยความจำหลักสามารถถูกวางไว้ในตำแหน่งใดก็ได้ที่ว่างอยู่ในแคช ทำให้ไม่มีปัญหาการแย่งชิงตำแหน่งเหมือนวิธี Direct Mapping แต่ข้อเสียคือการค้นหาข้อมูลจะซับซ้อนและต้องใช้ฮาร์ดแวร์ที่ซับซ้อนกว่า เนื่องจาก CPU ต้องตรวจสอบทุกแถวในแคชเพื่อหาข้อมูลท ซึ่งทำให้มีค่าใช้จ่ายสูงและใช้พลังงานมากกว่า เปรียบเหมือนการจอดรถในลานจอดที่ไม่มีการระบุช่องจอด สามารถจอดตรงไหนก็ได้ที่ว่าง

ข้อดี: ลดโอกาสเกิด Cache Miss ได้มาก เนื่องจากไม่มีข้อจำกัดในการวางตำแหน่ง

ข้อเสีย: มีความซับซ้อนในการออกแบบวงจรควบคุมและต้องใช้ฮาร์ดแวร์ที่ราคาแพงเพื่อตรวจสอบ Tag Bit ของทุกบรรทัดในแคชพร้อมกัน เพื่อหาว่าข้อมูลที่ต้องการอยู่ตำแหน่งใด

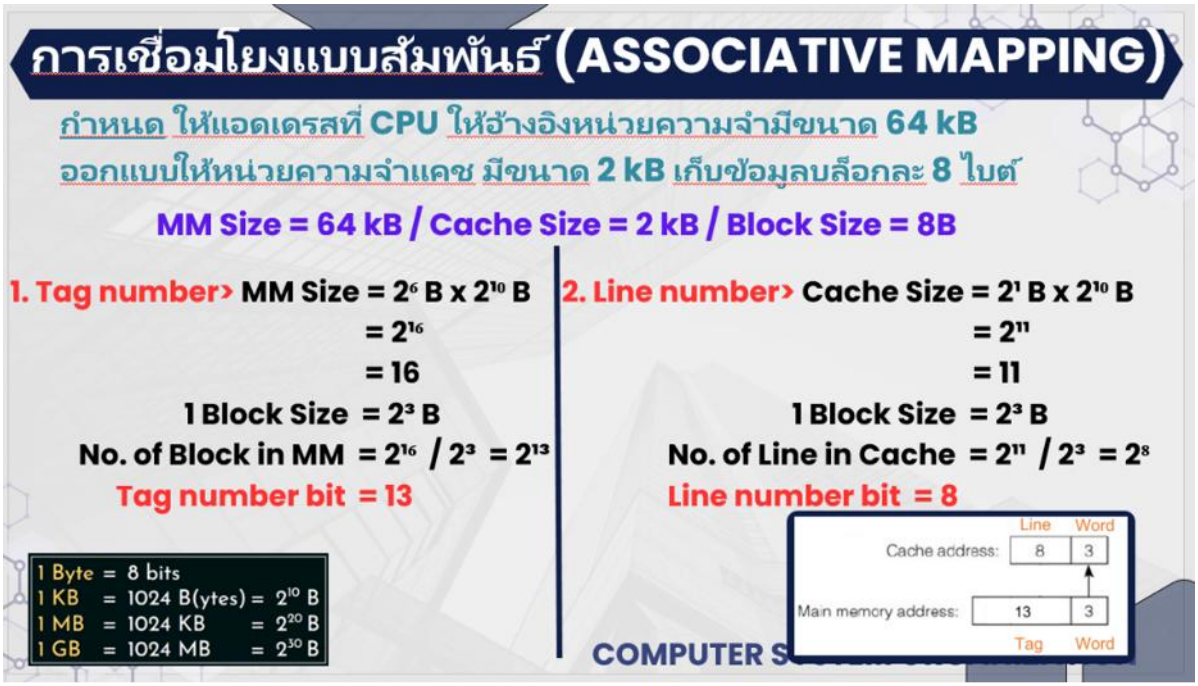
ตัวอย่าง: สมมติว่ามี Main Memory 8 บล็อก และ Cache 4 บรรทัด

CPU ต้องการข้อมูล Block 0 สามารถนำไปวางที่ Cache Line 0 ได้ และต่อมา CPU ต้องการข้อมูล Block 5 ซึ่งไม่มีอยู่ในแคช สามารถนำไปวางที่ Cache Line 1 ได้ (หรือบรรทัดอื่นที่ว่าง) หลังจากนั้น เมื่อ CPU ต้องการข้อมูล Block 0 อีกครั้ง ก็จะหาและพบได้ทันทีในบรรทัดที่ 0 โดยไม่ถูกเขียนทับจาก Block 5

10.5.1 หลักการคำนวณการเชื่อมโยงแบบสัมพันธ์ (Associative Mapping)

การเชื่อมโยงแบบ **Associative Mapping** เป็นวิธีการจัดการแคชที่ยืดหยุ่นที่สุด โดยบล็อกของหน่วยความจำหลักสามารถถูกนำไปเก็บไว้ที่ตำแหน่งใดก็ได้ในหน่วยความจำแคช การระบุตำแหน่งจึงไม่จำเป็นต้องใช้ **Cache Index** เหมือนในแบบ Direct Mapping ในการคำนวณ เราจะแบ่งแอดเดรสของหน่วยความจำหลักออกเป็น 2 ส่วนหลัก ได้แก่ :

1. **Block Offset (จำนวนบิตสำหรับระบุตำแหน่งข้อมูลภายในบล็อก)**: ใช้สำหรับระบุตำแหน่งของข้อมูล (ไบต์) ภายในบล็อกนั้นๆ
2. **Tag (จำนวนบิตสำหรับระบุบล็อก)**: ใช้สำหรับระบุว่าข้อมูลที่อยู่ในแคชมาจากบล็อกใดของหน่วยความจำหลัก



รูปที่ 7- 89 ตัวอย่างการคำนวณจำนวนบิตในการเชื่อมโยงแบบสัมพันธ์ (Associative Mapping)

โจทย์: กำหนดให้ Main Memory Size = 64 kB, Cache Size = 2 kB, และ Block Size = 8 B

1. การหาจำนวนบิตทั้งหมดของแอดเดรสในหน่วยความจำหลัก (Total Address Bits)

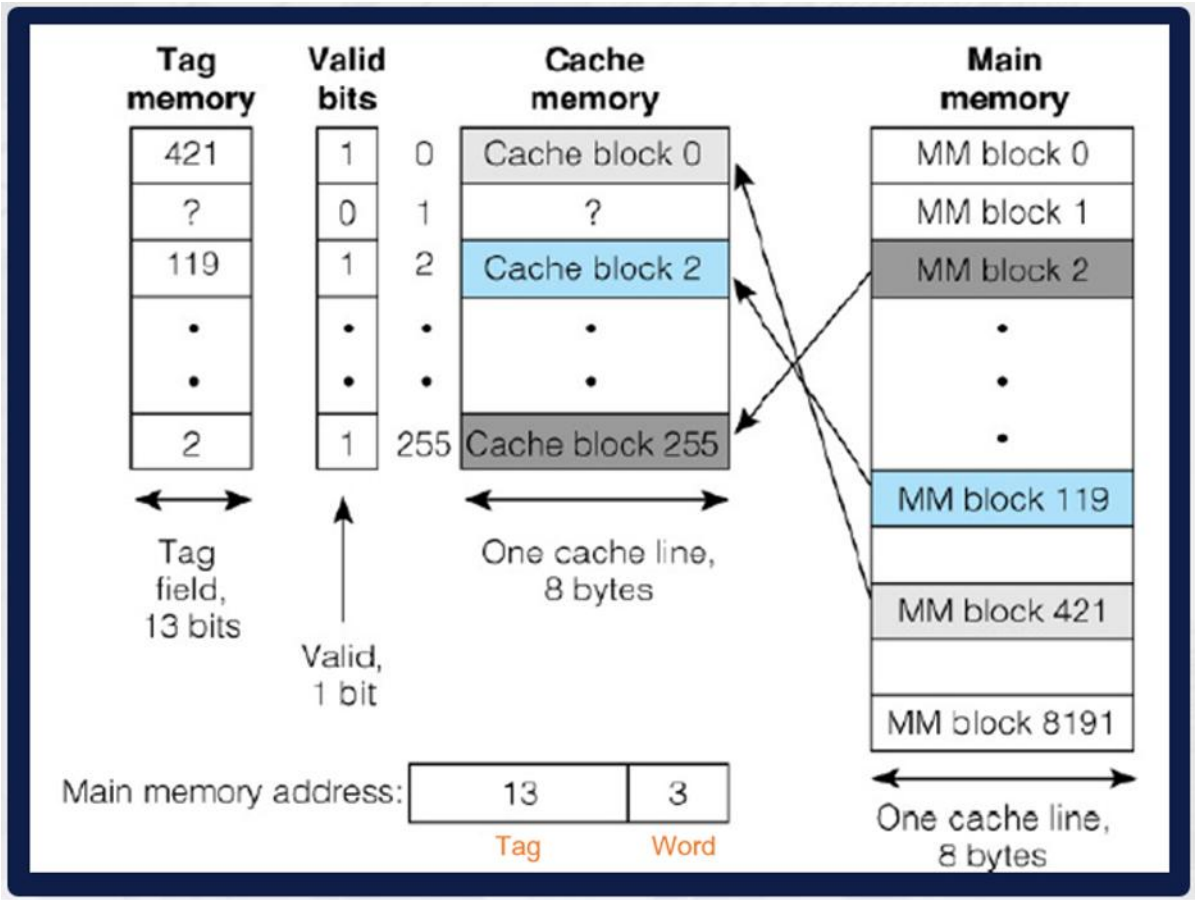
- ขนาดของหน่วยความจำหลัก (MM Size) = 64 kByte
- จากค่าคงที่ (1 kByte=2^{10} Byte)
- 64 kByte = 64×2^{10} Byte = 2^6×2^{10} Byte = 2^{16} Byte
- ดังนั้น จำนวนบิตของแอดเดรสทั้งหมดคือ **16 บิต**

2. การคำนวณ Block Offset

- ขนาดของบล็อก (Block Size) = 8 Byte
- จากค่าคงที่ (8 Byte = 2^3 Byte)
- ดังนั้น จำนวนบิตสำหรับ Block Offset คือ **3 บิต**

3. การคำนวณ Tag: ในระบบ Associative Mapping จำนวนบิตสำหรับ Tag จะเท่ากับ จำนวนบิตของ *แอดเดรสทั้งหมด* **ลบด้วย** *จำนวนบิตของ Block Offset*

- จำนวนบิตทั้งหมดของแอดเดรส = **16 บิต**
- จำนวนบิตสำหรับ Block Offset = **3 บิต**
- จำนวนบิตสำหรับ Tag = 16−3=13 บิต



รูปที่ 7- 90 โครงสร้างตารางและการตรวจสอบข้อมูลด้วย Valid Bit ใน ระบบ Associative Mapping Cache

ดังนั้น จำนวนบิตสำหรับ Tag คือ 13 บิต ซึ่งถูกต้องตามที่นักศึกษาคำนวณมาในส่วนของ "Tag number"

สรุปข้อดีการเชื่อมโยงแบบ **(Associative Mapping)**

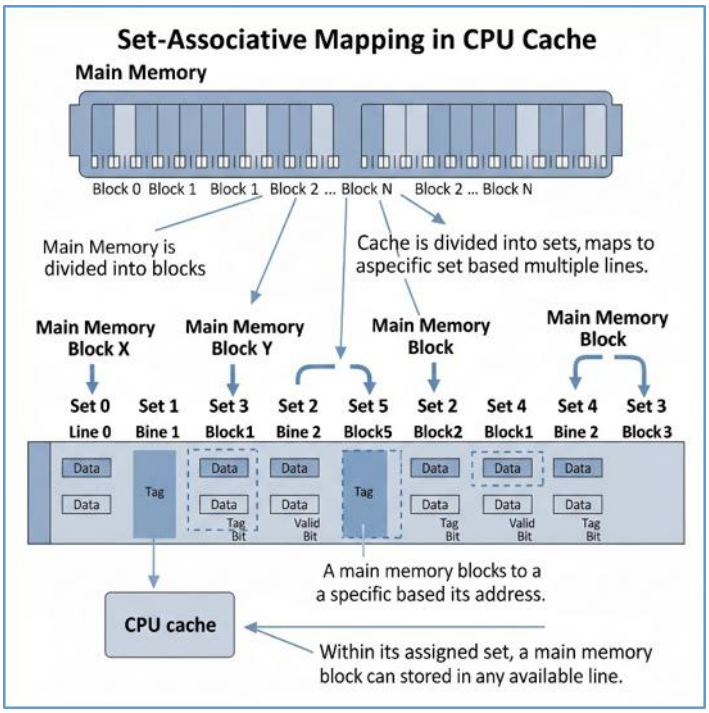
เป็นวิธีการแก้ปัญหาที่เกิดขึ้นกับวิธีการเชื่อมโยงแบบตรง โดยให้บล็อกของข้อมูลจากหน่วยความจำหลักมาอยู่ในตำแหน่งใดในหน่วยความจำแคชก็ได้ ทำให้การจัดการของหน่วยความจำแคชทำได้เต็มประสิทธิภาพมากขึ้น

สรุปข้อเสียการเชื่อมโยงแบบ **(Associative Mapping)**

เมื่อระบบทำงานจะต้องนำ Tag มาเปรียบเทียบค่าที่ต้องการทั้งหมด ทำให้ระบบทำงานได้ช้า หากในแคชไม่มีข้อมูล ระบบจะอ่านข้อมูลจากบล็อกใหม่ไปแทนที่ข้อมูลที่ไม่ได้ใช้แล้ว หรืออยู่ในแคชนานที่สุด ขึ้นอยู่กับการคำนวนอัลกอริทึมที่นำมาใช้

10.6 การเชื่อมโยงแบบกลุ่มสัมพันธ์ (Set-Associative Mapping)

วิธีนี้เป็นการผสมผสานข้อดีของสองวิธีแรกเข้าด้วยกัน โดยจะแบ่งแคชออกเป็นกลุ่มย่อยๆ (Set) แต่ละกลุ่มจะมีหลายแถว (Line) บล็อกข้อมูลจากหน่วยความจำหลักจะถูกกำหนดให้ต้องไปวางอยู่ในกลุ่มใดกลุ่มหนึ่งเป็นการเฉพาะ (เหมือน Direct Mapping) แต่ภายในกลุ่มนั้น บล็อกข้อมูลสามารถถูกวางได้ในแถวใดก็ได้ที่ว่างอยู่ (เหมือน Associative Mapping) ทำให้เกิดความสมดุลระหว่างความยืดหยุ่นและความเร็วการค้นหา



รูปที่ 7- 91 หลักการเชื่อมโยงแบบกึ่งสัมพันธ์ (Set-Associative Mapping) ของหน่วยความจำแคช

ข้อดี: ให้สมดุลระหว่างความยืดหยุ่นในการวางข้อมูลกับความซับซ้อนของฮาร์ดแวร์

ข้อเสีย: ซับซ้อนกว่า Direct Mapping แต่มีประสิทธิภาพสูงกว่า

ตัวอย่าง: สมมติว่ามี Main Memory 8 บล็อก และ Cache 4 บรรทัด (แบ่งเป็น 2 Set, แต่ละ Set มี 2 บรรทัด)

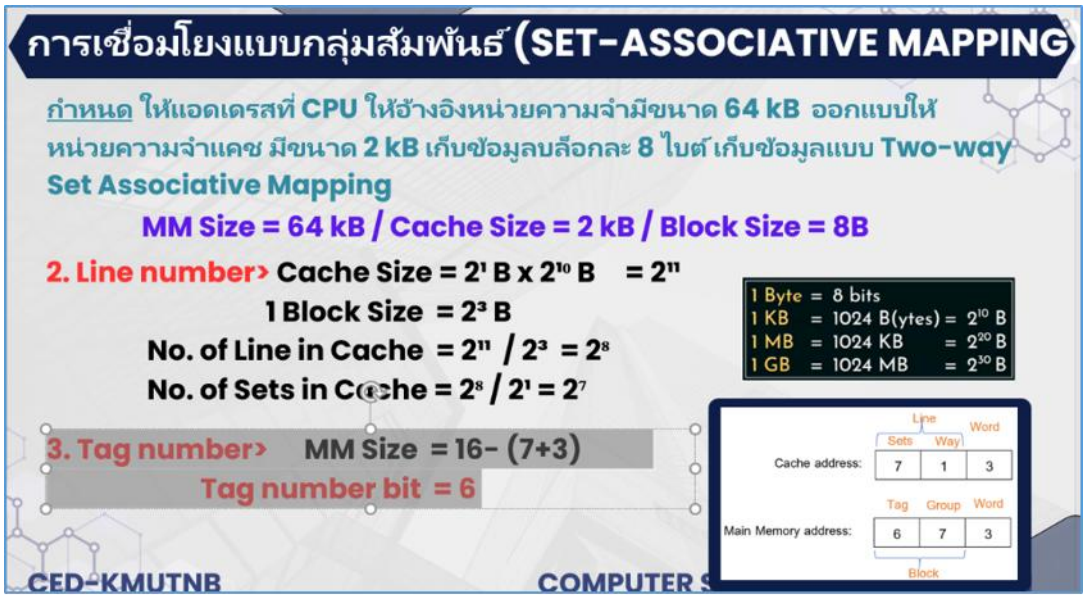
- Main Memory Block 0 และ 4 จะถูกแมปไปที่ Set 0
- Main Memory Block 1 และ 5 จะถูกแมปไปที่ Set 1

ถ้า CPU ต้องการข้อมูล Block 0 จะนำไปวางในบรรทัดใดก็ได้ใน Set 0 (เช่น บรรทัด 0) และหากต่อมาต้องการข้อมูล Block 4 ซึ่งก็ถูกแมปไปที่ Set 0 เช่นกัน ก็สามารถนำไปวางในบรรทัด 1 ของ Set 0 ได้ โดยไม่เกิดปัญหาการเขียนทับกับ Block 0

10.7 หลักการคำนวณการเชื่อมโยงแบบกลุ่มสัมพันธ์ (Set-Associative Mapping)

การเชื่อมโยงแบบ **Set-Associative Mapping** เป็นวิธีที่อยู่กึ่งกลางระหว่าง Direct Mapping และ Associative Mapping โดยจะแบ่งแคชออกเป็นกลุ่มย่อยๆ ที่เรียกว่า **Set** และแต่ละกลุ่มจะมีจำนวนบล็อกคงที่ ซึ่งเรียกว่า **"ทาง" (Way)** หรือ **"สาย" (Line)** บล็อกข้อมูลจากหน่วยความจำหลักสามารถถูกนำไปเก็บไว้ในบล็อกใดก็ได้ภายใน Set ที่ถูกกำหนดไว้เท่านั้น ในการคำนวณ จะแบ่งแอดเดรสของหน่วยความจำหลักออกเป็น 3 ส่วนหลัก ได้แก่:

1. **Block Offset (จำนวนบิตสำหรับระบุตำแหน่งข้อมูลภายในบล็อก)**: ใช้สำหรับระบุตำแหน่งของข้อมูล (ไบต์) ภายในบล็อกนั้นๆ
2. **Set Index (จำนวนบิตสำหรับระบุตำแหน่ง Set ในแคช)**: ใช้สำหรับระบุว่าบล็อกข้อมูลจากหน่วยความจำหลักจะถูกนำไปเก็บไว้ใน Set ใดของแคช
3. **Tag (จำนวนบิตสำหรับระบุบล็อก)**: ใช้สำหรับระบุว่าข้อมูลที่อยู่ในแคชบล็อกนั้นมาจากบล็อกใดของหน่วยความจำหลัก



รูปที่ 7- 92 ตัวอย่างการคำนวณจำนวนบิตในการเชื่อมโยงแบบกลุ่มสัมพันธ์ (Set-Associative Mapping)

โจทย์: กำหนดให้ Main Memory Size = 64 kB, Cache Size = 2 kB, และ Block Size = 8 B โดยใช้การเชื่อมโยงแบบ **Two-way Set Associative Mapping**

1. การหาจำนวนบิตทั้งหมดของแอดเดรสในหน่วยความจำหลัก (Total Address Bits)

- ขนาดของหน่วยความจำหลัก (MM Size) = 64 kByte (ค่าคงที่ 1 kB = 2^{10} Byte)
- 64 kByte = 64×2^{10} Byte = $2^6 \times 2^{10}$ Byte = 2^{16} Byte
- ดังนั้น จำนวนบิตของแอดเดรสทั้งหมดคือ **16 บิต**

2. การคำนวณ Block Offset : ขนาดของบล็อก (Block Size) = 8 Byte (ค่าคงที่ 8 Byte = 2^3 Byte)

- ดังนั้น จำนวนบิตสำหรับ Block Offset คือ **3 บิต**

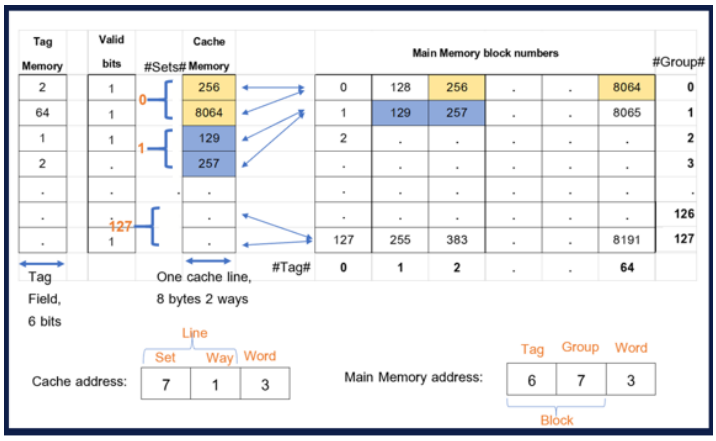
3. การคำนวณ Set Index : ขนาดของแคช (Cache Size) = 2 kByte = 2048 Byte

- ขนาดของบล็อก (Block Size) = 8 Byte
- จำนวนบล็อกทั้งหมดในแคช = Cache Size หารด้วย Block Siz
  - = 2048 Byte หารด้วย 8 Byte

  เท่ากับ = 256 บล็อก
- โจทย์กำหนดให้เป็น **Two-way** หมายถึง แต่ละ Set มี 2 บล็อก
- จำนวน Set ทั้งหมดในแคช = (จำนวนบล็อกทั้งหมดในแคช) หารด้วย (จำนวนทาง (Way))
  - = จำนวน 256 หาร 2 Way =128 Sets
  - และ 128 = ทำยกกำลังสอง จึงได้ 2^7
- ดังนั้น จำนวนบิตสำหรับ Set Index คือ **7 บิต**

4. การคำนวณ Tag

- จำนวนบิตทั้งหมดของแอดเดรส = **16 บิต**
- จำนวนบิตสำหรับ Block Offset = **3 บิต**
- จำนวนบิตสำหรับ Set Index = **7 บิต**
- จำนวนบิตสำหรับ Tag = (จำนวนบิตทั้งหมด) ลบด้วย (Block Offset) ลบด้วย (Set Index)
- จำนวนบิตสำหรับ Tag = 16 – 3 – 7 = 6 บิต
- ดังนั้น จำนวนบิตสำหรับ Tag คือ **6 บิต**



รูปที่ 7- 93 โครงสร้างตารางและการตรวจสอบข้อมูลด้วย Valid Bit ในระบบ Set-Associative Mapping Cache

สรุปผลการคำนวณ แอดเดรสของหน่วยความจำหลักจะถูกแบ่งออกเป็น 3 ส่วนดังนี้:

- **1. Tag:** 6 บิต **2. Set Index:** 7 บิต **3. Block Offset:** 3 บิต

สรุปข้อดีการเชื่อมโยงแบบกลุ่มสัมพันธ์ (Set-Associative Mapping)

เป็นวิธีการนำแนวคิดทั้ง 2 วิธีมารวมกัน ทำให้ข้อมูลที่อยู่ในหน่วยความจำต่างบล็อกกัน แต่อยู่ในกลุ่มเดียวกันมาเก็บในหน่วยความจำแคชพร้อมกันได้

สรุปข้อเสียการเชื่อมโยงแบบกลุ่มสัมพันธ์ (Set-Associative Mapping)

หากกลุ่มของข้อมูลในแคชใหญ่ขึ้น อัตราการพบข้อมูลจะมากขึ้น แต่การทำงานของระบบจะซับซ้อนมากขึ้น

10.8 การแทนที่ข้อมูล (Cache Replacement Policy) บล็อกที่จะนำข้อมูลไปแทนที่ เมื่อไม่มีการพบข้อมูลในแคช

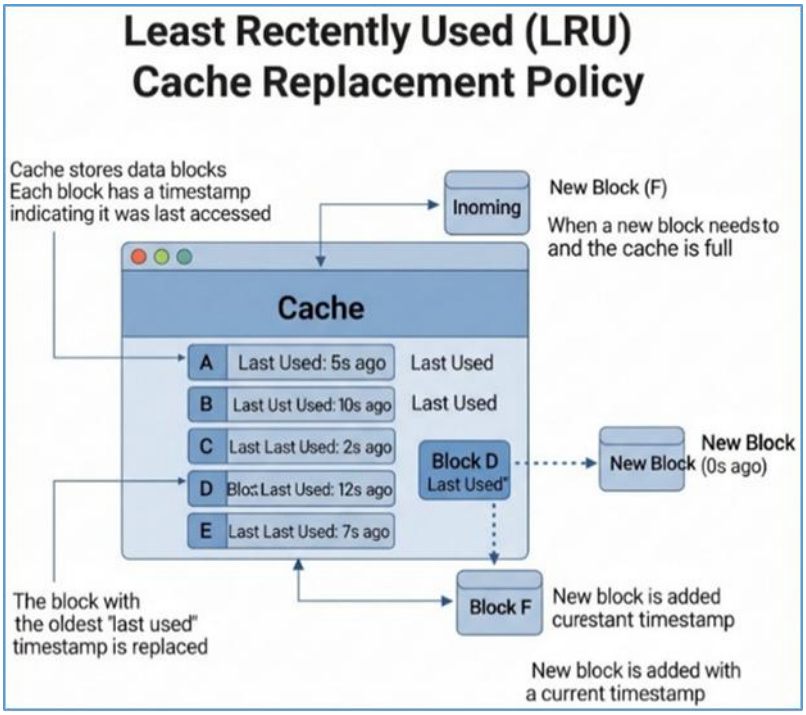
เมื่อ CPU ต้องการข้อมูลจากหน่วยความจำหลักแต่ไม่พบในหน่วยความจำแคช (Cache Miss) ระบบจะต้องนำข้อมูลบล็อกใหม่จากหน่วยความจำหลักมาเก็บไว้ในแคช ในกรณีที่แคชเต็ม จะต้องมีนโยบายในการเลือกบล็อกข้อมูลในแคชเพื่อนำบล็อกใหม่เข้ามาแทนที่ ซึ่งเราจะเรียกนโยบายนี้ว่า **“Cache Replacement Policy”** โดยมีวิธีการที่สำคัญดังนี้

10.8.1. วิธีแบบใช้งานน้อยที่สุด (Least Recently Used: LRU)

วิธีนี้จะเลือกบล็อกข้อมูลที่ไม่ได้ถูกใช้งานมานานที่สุดแล้วเพื่อนำออกไป หลักการคือระบบจะติดตามว่าบล็อกใดในแคชถูกใช้งานครั้งสุดท้ายเมื่อใด และจะเลือกบล็อกที่ไม่ได้ใช้งานนานที่สุดเพื่อนำออกไป ซึ่งเป็นวิธีที่มีประสิทธิภาพสูงในการจัดการแคช

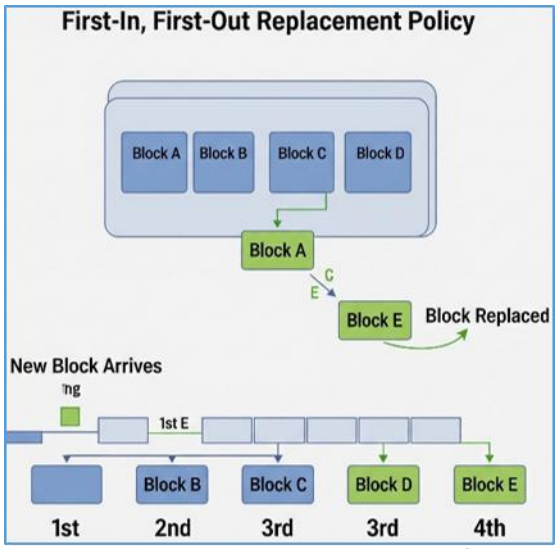
ตัวอย่าง: สมมติว่าแคชมี 3 บล็อก และมีการเรียกข้อมูลตามลำดับ: A, B, C, A, D, B, E

1. **A, B, C**: แคชว่างอยู่ จึงนำ A, B, C เข้ามาตามลำดับ
2. **A**: A ถูกเรียกใช้ ทำให้ A เป็นข้อมูลที่ถูกใช้ล่าสุด
3. **D**: D ไม่อยู่ในแคช (Cache Miss) จะต้องนำ D เข้ามาแทนที่ข้อมูลที่ถูกใช้งานน้อยที่สุด นั่นคือ **B** (A และ C เพิ่งถูกใช้ไป)
4. **B**: B ถูกเรียกอีกครั้ง แต่ไม่อยู่ในแคช (Cache Miss) ต้องนำ B เข้ามาแทนที่ข้อมูลที่ถูกใช้งานน้อยที่สุด นั่นคือ **C**
5. **E**: E ไม่อยู่ในแคช (Cache Miss) ต้องนำ E เข้ามาแทนที่ข้อมูลที่ถูกใช้งานน้อยที่สุด นั่นคือ **A** (B และ D เพิ่งถูกใช้ไป)



รูปที่ 7- 94 กลไกการสับเปลี่ยนบล็อกข้อมูลในแคชด้วยนโยบาย Least Recently Used (LRU)

10.8.2 วิธีแบบมาก่อนออกก่อน (First-in First-out: FIFO)



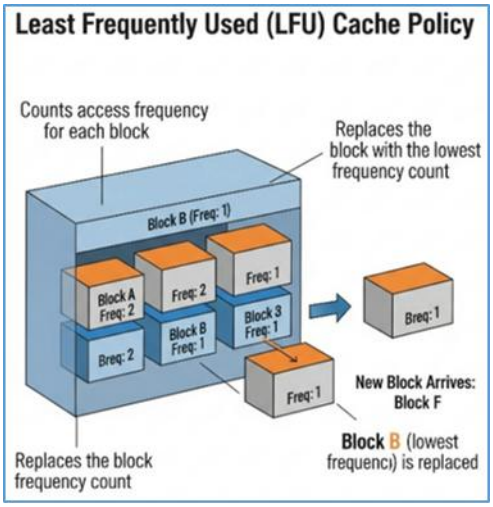
รูปที่ 7- 95 กลไกการสับเปลี่ยนบล็อกข้อมูลแคชด้วยนโยบาย First-In, First-Out (FIFO)

เป็นวิธีที่ง่ายที่สุด โดยจะเลือกบล็อกข้อมูลที่เข้ามาในแคชเป็นอันแรกเพื่อนำออกไปก่อนเสมอ โดยไม่สนใจว่าข้อมูลนั้นถูกใช้งานบ่อยแค่ไหน หลักการทำงานคล้ายกับแถวรอคิว ข้อมูลที่เข้ามาแรกสุดจะถูกนำออกไปแรกสุดเช่นกัน

ตัวอย่าง: สมมติว่าแคชมี 3 บล็อก และมีการเรียกข้อมูลตามลำดับ: A, B, C, A, D, B, E

1. **A, B, C**: แคชว่างอยู่ จึงนำ A, B, C เข้ามาตามลำดับ
2. **A**: ข้อมูล A ถูกเรียกอีกครั้ง ซึ่งอยู่ในแคชแล้ว (Cache Hit)
3. **D**: ข้อมูล D ไม่อยู่ในแคช (Cache Miss) ต้องนำ D เข้ามาแทนที่ข้อมูลที่เข้ามาแรกสุด นั่นคือ **A** (ซึ่งเข้ามาแรกสุดในบรรดา A, B, C)
4. **B**: ข้อมูล B ถูกเรียกอีกครั้ง ซึ่งอยู่ในแคชแล้ว (Cache Hit)
5. **E**: ข้อมูล E ไม่อยู่ในแคช (Cache Miss) ต้องนำ E เข้ามาแทนที่ข้อมูลที่เข้ามาเป็นอันดับถัดไป คือ **C**

10.8.3 วิธีแบบใช้งานถี่น้อยที่สุด (Least Frequently Used: LFU)



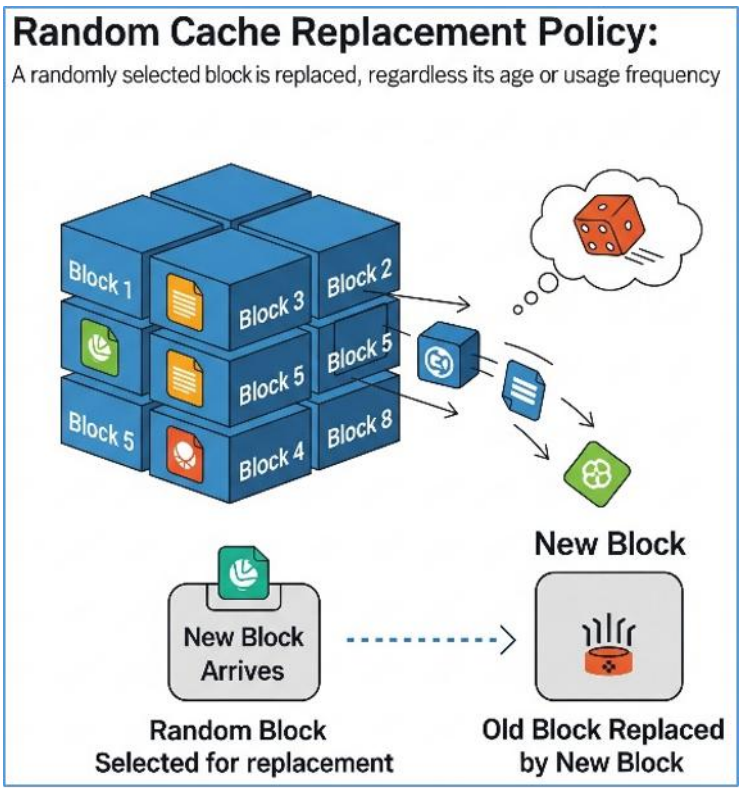
รูปที่ 7- 96 กลไกการสับเปลี่ยนบล็อกข้อมูลในแคชด้วยนโยบาย Least Frequently Used (LFU)

วิธีนี้จะเลือกบล็อกข้อมูลที่มีความถี่ในการใช้งานน้อยที่สุดเพื่อนำออกไปจากแคช หลักการคือจะนับจำนวนครั้งที่บล็อกข้อมูลแต่ละบล็อกถูกเรียกใช้งาน และเมื่อแคชเต็มก็จะเลือกบล็อกที่มีจำนวนการใช้งานน้อยที่สุดเพื่อนำออกไป **ตัวอย่าง:** สมมติว่าแคชมี 3 บล็อก และมีการเรียกข้อมูลตามลำดับ: A, A, B, C, D, A, B, E

1. **A, A, B**: แคชว่างอยู่ นำ A, B เข้ามา โดย A มีการใช้งาน 2 ครั้ง, B มีการใช้งาน 1 ครั้ง
2. **C**: C ไม่อยู่ในแคช (Cache Miss) แคชเต็มแล้ว ต้องแทนที่บล็อกที่มีความถี่น้อยที่สุด นั่นคือ **B** (ความถี่ 1 ครั้ง) ดังนั้น C จึงเข้ามาแทนที่ B
3. **D**: D ไม่อยู่ในแคช (Cache Miss) แคชเต็มแล้ว ต้องแทนที่บล็อกที่มีความถี่น้อยที่สุด นั่นคือ **C** (ความถี่ 1 ครั้ง)
4. **A, B**: A และ B ถูกเรียกใช้ A มีความถี่เพิ่มเป็น 3 ครั้ง ส่วน B เข้ามาใหม่ มีความถี่ 1 ครั้ง
5. **E**: E ไม่อยู่ในแคช (Cache Miss) ต้องแทนที่บล็อกที่มีความถี่น้อยที่สุด นั่นคือ **B** (ความถี่ 1 ครั้ง)

10.8.4 วิธีแบบสุ่ม (Random)

วิธีนี้จะไม่มีการคำนวณหรือติดตามข้อมูลใด ๆ ทั้งสิ้น เมื่อเกิด Cache Miss และแคชเต็ม ระบบจะสุ่มเลือกบล็อกข้อมูลขึ้นมาหนึ่งบล็อกเพื่อนำออกไป วิธีนี้เป็นวิธีที่เรียบง่ายที่สุด แต่ประสิทธิภาพอาจไม่ดีเท่าวิธีอื่น ๆ เนื่องจากไม่มีการพิจารณาถึงพฤติกรรมการใช้งานข้อมูลเลย

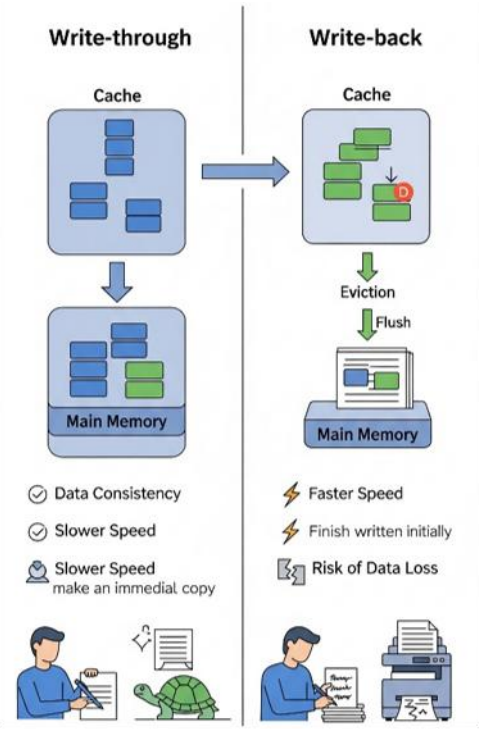


รูปที่ 7- 97 กลไกการสับเปลี่ยนบล็อกข้อมูลในแคชด้วยนโยบายการสุ่ม (Random Cache Replacement Policy)

ตัวอย่าง: สมมติว่าแคชมี 3 บล็อก และมีการเรียกข้อมูลตามลำดับ: A, B, C, A, D, B, E

1. **A, B, C**: แคชว่างอยู่ จึงนำ A, B, C เข้ามาตามลำดับ
2. **A**: A ถูกเรียกใช้ (Cache Hit)
3. **D**: D ไม่อยู่ในแคช (Cache Miss) ระบบสุ่มเลือกบล็อกที่จะนำออกไป สมมติว่าสุ่มได้ **A** ดังนั้น D จึงเข้ามาแทนที่ A
4. **B**: B ถูกเรียกใช้ (Cache Hit)
5. **E**: E ไม่อยู่ในแคช (Cache Miss) ระบบสุ่มเลือกบล็อกที่จะนำออกไปอีกครั้ง สมมติว่าสุ่มได้ **C** ดังนั้น E จึงเข้ามาแทนที่ C

แต่ละวิธีมีข้อดีและข้อเสียที่แตกต่างกันไป การเลือกใช้วิธีใดขึ้นอยู่กับลักษณะการทำงานของโปรแกรมและความต้องการของระบบเป็นหลัก



รูปที่ 7- 98 กระบวนการเขียนข้อมูลซ้ำและการจัดการสภาวะไม่สอดคล้องแบบ Write-through และ Write-back

10.9 การเขียนข้อมูล (Write Policy)

ในการจัดการหน่วยความจำแคช (Cache Memory) การเขียนข้อมูล (Write Policy) เป็นกลไกสำคัญที่กำหนดว่าเมื่อมีการเขียนข้อมูลใหม่ลงในหน่วยความจำแคชแล้ว จะมีการอัปเดตข้อมูลในหน่วยความจำหลัก (Main Memory) อย่างไร เพื่อให้ข้อมูลทั้งสองส่วนมีความสอดคล้องกันนโยบายการเขียนข้อมูลมี 2 แบบ

10.9.1. การเขียนทันที (Write-through)

หลักการทำงาน: นโยบาย Write-through คือ การที่ CPU เขียนข้อมูลลงในแคชและหน่วยความจำหลักพร้อมกันในทันที เมื่อข้อมูลในแคชมีการเปลี่ยนแปลง ข้อมูลนั้นจะถูกส่งไปอัปเดตในหน่วยความจำหลักทันทีเช่นกัน

ข้อดี: ความสอดคล้องกันของข้อมูล: ข้อมูลในแคชและหน่วยความจำหลักจะตรงกันเสมอ ทำให้สามารถกู้คืนข้อมูลได้ง่ายหากเกิดความล้มเหลวของแคช

ข้อเสีย: เกิดความช้า: การส่งข้อมูลไปยังหน่วยความจำหลักทุกครั้งที่มีการเขียนข้อมูลใหม่ ทำให้เกิดการใช้บัสระบบ (System Bus) บ่อยครั้ง ซึ่งอาจทำให้การทำงานช้าลง

ตัวอย่างเปรียบเทียบ: การที่เราจดบันทึกข้อมูลสำคัญลงบนกระดาษทด (แคช) และถ่ายสำเนา (หน่วยความจำหลัก) ทันทีทุกครั้งที่เราแก้ไขข้อมูลบนกระดาษทดนั้นทำให้ข้อมูลบนกระดาษทดและสำเนาตรงกันเสมอ

10.9.2 การเขียนทีหลัง (Write-back)

หลักการทำงาน: นโยบาย Write-back จะเขียนข้อมูลลงในแคชเท่านั้น และจะยังไม่เขียนกลับไปที่หน่วยความจำหลัก โดยจะมีการตรวจสอบสถานะของข้อมูลในแคชด้วย **Dirty Bit** หากข้อมูลในแคชมีการเปลี่ยนแปลง (Dirty Bit ถูกตั้งค่าเป็น 1) ระบบจะเขียนข้อมูลกลับไปยังหน่วยความจำหลักก็ต่อเมื่อข้อมูลในแคชบล็อกนั้นจะต้องถูกแทนที่ด้วยข้อมูลใหม่จากหน่วยความจำหลัก หรือเมื่อโปรแกรมจบการทำงานเท่านั้น

ข้อดี: ความเร็ว: การเขียนข้อมูลลงในแคชเพียงอย่างเดียวทำให้ไม่ต้องเข้าถึงหน่วยความจำหลักบ่อยครั้ง ส่งผลให้การทำงานมีความเร็วสูงขึ้น

ข้อเสีย: ความเสี่ยงต่อข้อมูล: หากเกิดความล้มเหลวของแคช (Cache Failure) ข้อมูลที่อยู่ในแคชแต่ยังไม่ได้เขียนกลับไปยังหน่วยความจำหลักจะสูญหาย ทำให้ไม่สามารถกู้คืนได้

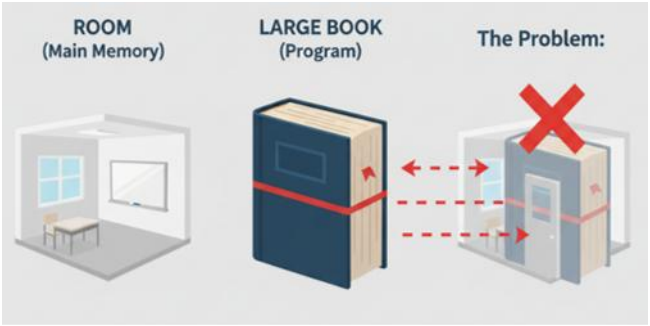
ตัวอย่างเปรียบเทียบ: การที่เราจดบันทึกข้อมูลสำคัญลงบนกระดาษทด (แคช) และแก้ไขข้อมูลไปเรื่อย ๆ แต่จะยังไม่ถ่ายสำเนา (หน่วยความจำหลัก) จนกว่าเราจะจดข้อมูลใหม่จนต้องทิ้งกระดาษทดแผ่นเดิมไปก่อน หรือเมื่อจดบันทึกเสร็จสิ้นแล้วจึงค่อยนำไปถ่ายสำเนาเป็นฉบับสุดท้าย

11.หลักการทำงานของหน่วยความจำเสมือน (Virtual Memory)

หน่วยความจำเสมือนเป็นเทคนิคการจัดการหน่วยความจำที่ช่วยให้โปรแกรมสามารถทำงานได้แม้ว่าขนาดของโปรแกรมจะใหญ่กว่าขนาดของหน่วยความจำหลักที่มีอยู่จริง หลักการคือระบบปฏิบัติการ (Operating System) จะทำหน้าที่แบ่งโปรแกรมออกเป็นส่วนๆ และจัดเก็บเพียงบางส่วนที่จำเป็นต้องใช้งานในขณะนั้นไว้ใน **หน่วยความจำหลัก (Main Memory)** ส่วนที่เหลือจะถูกเก็บไว้ใน **หน่วยความจำสำรอง (Secondary Memory)** เช่น ฮาร์ดดิสก์ เมื่อโปรแกรมต้องการส่วนใดที่ยังไม่ได้อยู่ในหน่วยความจำหลัก ระบบปฏิบัติการจะทำการสลับข้อมูล (Swap) ส่วนนั้นจากฮาร์ดดิสก์เข้ามาในหน่วยความจำหลัก และอาจจะต้องสลับส่วนที่ไม่ใช้งานออกไปเพื่อเคลียร์พื้นที่ ซึ่งจะเกิดขึ้นอย่างอัตโนมัติ ทำให้ผู้ใช้และโปรแกรมไม่จำเป็นต้องสนใจว่าข้อมูลอยู่ที่ใด

11.1 การเปรียบเทียบการทำงาน (Virtual Memory)

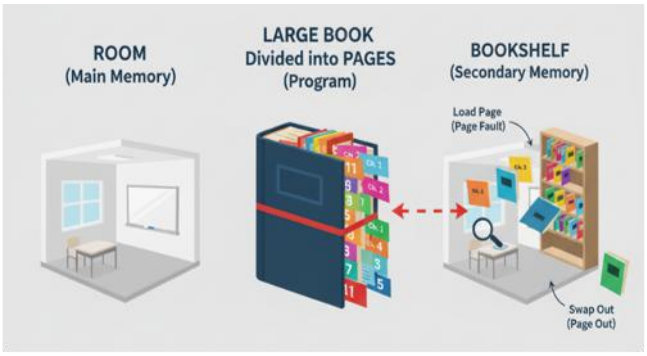
กรณีที่ 1 การทำงานแบบไม่มีหน่วยความจำเสมือน (No Virtual Memory)



รูปที่ 7- 99 ปัญหาข้อจำกัดของพื้นที่เมื่อรันโปรแกรมขนาดใหญ่โดยไม่มีหน่วยความจำเสมือน

เปรียบเทียบห้องเรียนหนึ่ง “ห้อง” (เปรียบเสมือน หน่วยความจำหลัก) และมีหนังสือเรียนขนาดใหญ่มากเล่มหนึ่ง (เปรียบเสมือน โปรแกรม) การทำงานจะเริ่มได้ก็ต่อเมื่อเราสามารถนำหนังสือทั้งเล่มเข้ามาในห้องได้ทั้งหมดเท่านั้น

กรณีที่ 2 การทำงานแบบมีหน่วยความจำเสมือน (With Virtual Memory)

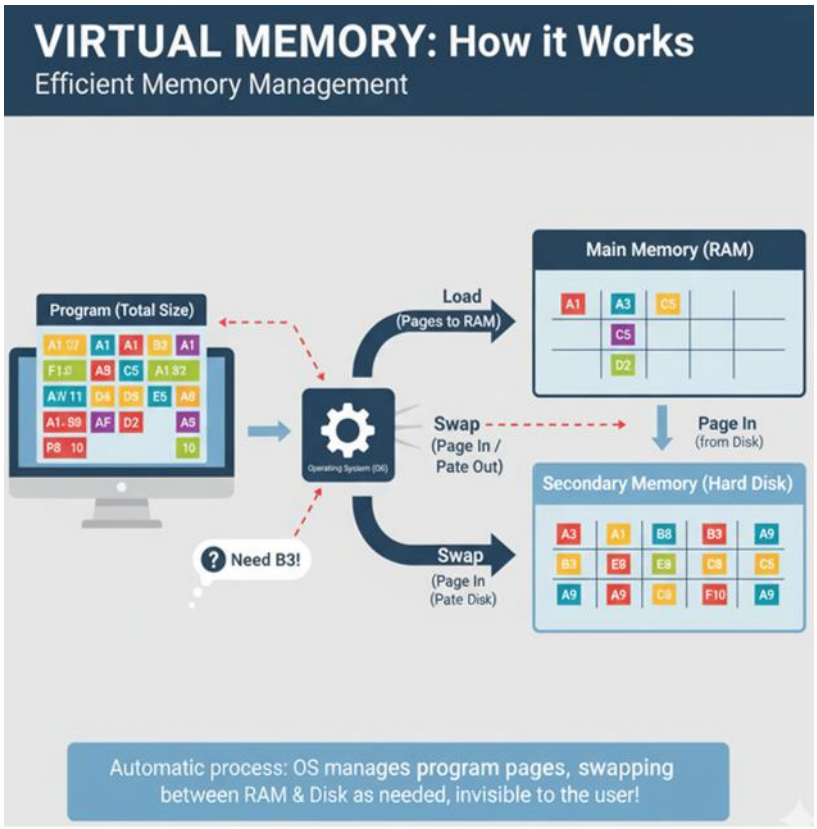


รูปที่ 7- 100 แนวคิดการจัดการหน้าข้อมูล (Paging) และการสลับหน้าในระบบหน่วยความจำเสมือน

ในทางตรงกันข้าม ถ้าเราใช้แนวคิดของหน่วยความจำเสมือน เราจะทำ การแบ่งหน้า (Paging) หนังสือออกเป็นบทๆ (เปรียบเสมือน เพจ) และเราจะนำเฉพาะบทที่จำเป็นต้องใช้เข้ามาไว้ในห้องเรียน (หน่วยความจำหลัก) ในขณะนั้น ส่วนบทที่เหลือจะยังคงอยู่บนชั้นวางหนังสือขนาดใหญ่ที่อยู่นอกห้อง (เปรียบเสมือน หน่วยความจำสำรอง/ฮาร์ดดิสก์)

สรุปการทำงาน: "เราสามารถเริ่มการเรียนการสอน (รันโปรแกรม) ได้ทันที โดยนำเฉพาะหน้าที่จำเป็นเข้ามาในห้องเรียนก่อน และถ้าต้องการหน้าอื่นเพิ่มเติม เราก็จะเดินออกไปหยิบมาเพิ่ม (Page Fault) ทำให้เราสามารถทำงานกับหนังสือขนาดใหญ่ได้ แม้ห้องเรียนจะมีขนาดจำกัด"

11.2 ข้อดีและข้อเสียของหน่วยความจำเสมือน (Virtual Memory)



รูปที่ 7- 101 กลไกการทำงานและการสลับหน้าข้อมูล (Swapping) ในระบบหน่วยความจำเสมือน

11.2.1 ข้อดี: 1. ใช้หน่วยความจำหลักอย่างคุ้มค่า: ช่วยให้ระบบสามารถรันโปรแกรมขนาดใหญ่ได้มากกว่าความจุของ RAM จริง

2.Multiprogramming ได้ดีขึ้น: ทำให้สามารถรันโปรแกรมหลายๆ โปรแกรมพร้อมกันได้มากขึ้น เพราะแต่ละโปรแกรมไม่จำเป็นต้องใช้พื้นที่ RAM ทั้งหมดตลอดเวลา

3.ความยืดหยุ่น: โปรแกรมเมอร์ไม่ต้องกังวลเรื่องขนาดของหน่วยความจำหลักขณะพัฒนาโปรแกรม

11.2.2 ข้อเสีย: 1.ประสิทธิภาพที่ลดลง: การสลับข้อมูลระหว่างหน่วยความจำหลักและหน่วยความจำสำรอง (ฮาร์ดดิสก์) นั้นใช้เวลามาก เนื่องจากความเร็วในการเข้าถึงข้อมูลของฮาร์ดดิสก์ช้ากว่า RAM มาก ซึ่งอาจทำให้โปรแกรมทำงานช้าลงอย่างเห็นได้ชัด

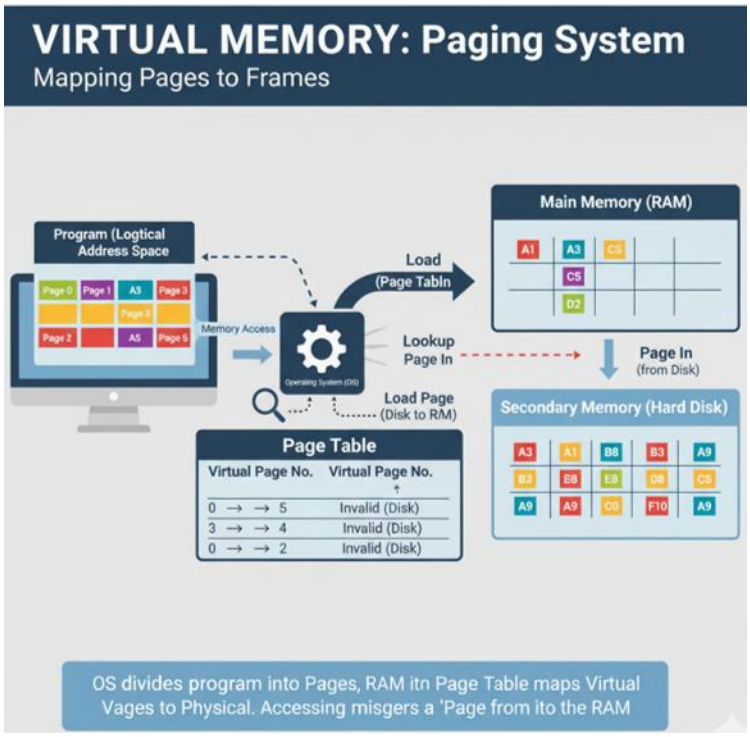
2.การเกิด Page Fault: หากมีการอ้างถึงหน้าที่ไม่ได้อยู่ในหน่วยความจำหลักบ่อยๆ จะทำให้เกิด **การผิดเพจ (Page Fault)** ซึ่งจะทำให้ระบบต้องเสียเวลาโหลดหน้าจากหน่วยความจำสำรองเข้ามาบ่อยครั้ง ทำให้โปรแกรมทำงานช้าลง

11.3 การทำงานหน่วยความจำเสมือนระบบเพจ (Paging System)

Paging เป็นเทคนิคหลักที่ใช้ในการจัดการหน่วยความจำเสมือน โดยมีหลักการแบ่งพื้นที่ของหน่วยความจำออกเป็นส่วนๆ ดังนี้:

- **โปรแกรม (Process):** จะถูกแบ่งออกเป็นส่วนๆ ที่มีขนาดเท่ากันเรียกว่า **เพจ (Page)**

- **หน่วยความจำหลัก (Main Memory):** จะถูกแบ่งออกเป็นส่วนๆ ที่มีขนาดเท่ากันเรียกว่า **เฟรม (Frame)**



รูปที่ 7- 102 ความแตกต่างของขนาดพื้นที่แอดเดรสระหว่างหน่วยความจำเสมือนและหน่วยความจำจริง

ระบบเพจเป็นวิธีการจัดการหน่วยความจำเสมือนที่นิยมใช้กัน โดยระบบปฏิบัติการจะแบ่งหน่วยความจำเสมือนออกเป็นส่วนย่อยๆ ที่เรียกว่า เพจ (Page) และแบ่งหน่วยความจำหลักออกเป็นส่วนย่อยๆ ที่มีขนาดเท่ากันเรียกว่า เฟรม (Frame) เมื่อโปรแกรมต้องการใช้งานข้อมูล ระบบปฏิบัติการจะโหลดเฉพาะเพจที่จำเป็นเข้ามาไว้ในเฟรมของหน่วยความจำหลัก และสร้าง ตารางเพจ (Page Table) เพื่อทำหน้าที่บันทึกว่าแต่ละเพจของโปรแกรมถูกจัดเก็บไว้ที่เฟรมใดในหน่วยความจำหลัก หากโปรแกรมพยายามเข้าถึงเพจที่ยังไม่ได้ถูกโหลดเข้าหน่วยความจำหลัก จะเกิดสิ่งที่เรียกว่า การผิดเพจ (Page Fault) ในกรณีนี้ ระบบปฏิบัติการจะต้องหยุดการทำงานชั่วคราวเพื่อค้นหาเพจที่ต้องการจากหน่วยความจำสำรองมาเก็บไว้ในหน่วยความจำหลัก

11.3.1 การเปรียบเทียบการทำงานของระบบ Paging System

กรณีนักศึกษาไปนั่งอ่านหนังสือห้องสมุด มีหนังสือหลายเล่ม ซึ่งแต่ละเล่มมีขนาดใหญ่มากจนไม่สามารถนำมาวางบนโต๊ะอ่านหนังสือเล็กๆ ของเราได้ทั้งหมด

11.3.1.1 การจัดระเบียบข้อมูล

- **หนังสือขนาดใหญ่ (โปรแกรม):** เราจะแบ่งหนังสือออกเป็นบทๆ แต่ละบทมีขนาดเท่ากันหมด (เปรียบเสมือน **เพจ** หรือ **Page**)
- **โต๊ะอ่านหนังสือ (หน่วยความจำหลัก):** เราจะแบ่งพื้นที่บนโต๊ะออกเป็น



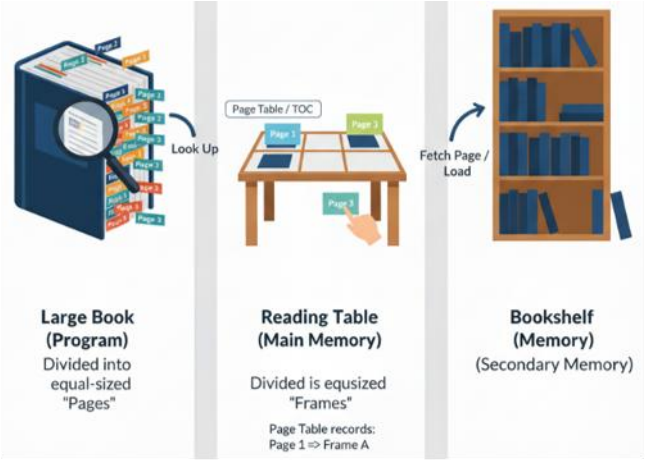
รูปที่ 7- 103 แบบจำลองเปรียบเทียบองค์ประกอบของระบบหน้าข้อมูล (Paging System Analogy)

ช่องๆ หรือส่วนๆ ที่มีขนาดเท่ากันพอดีกับแต่ละบทของหนังสือ (เปรียบเสมือน **เฟรม** หรือ **Frame**)

- **ชั้นวางหนังสือในห้องสมุด (หน่วยความจำสำรอง/ฮาร์ดดิสก์):** คือที่เก็บหนังสือทั้งเล่มไว้ ซึ่งบทต่างๆ จะถูกจัดเรียงตามลำดับอย่างเป็นระเบียบ

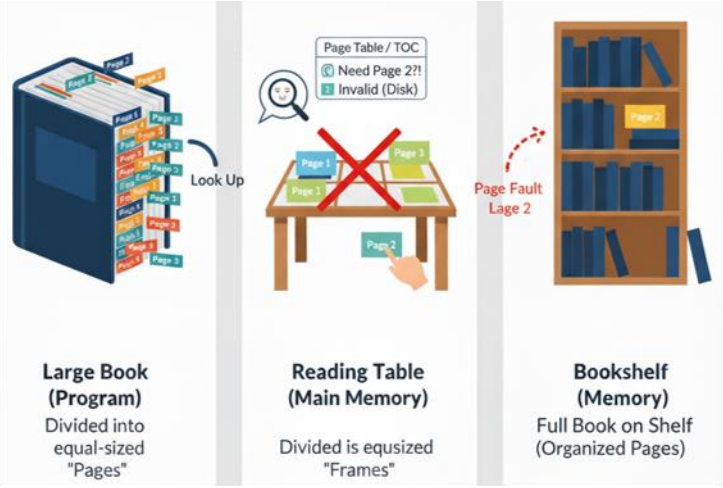
11.3.2. การทำงานปกติ (ไม่มี Page Fault)

เมื่อเราต้องการจะอ่านหนังสือเล่มนี้ เราไม่ได้ขนหนังสือทั้งเล่มมาวางบนโต๊ะ แต่เราจะดูสารบัญ (เปรียบเสมือน **ตารางเพจ** หรือ **Page Table**) เพื่อหาว่าบทที่เราต้องการอยู่ตรงไหน



รูปที่ 7- 104 แบบจำลองเปรียบเทียบกระบวนการค้นหาและดึงหน้าข้อมูลผ่านตารางสารบัญ (Page Table)

- **กระบวนการ:** เราจะหยิบบทที่เราจะอ่าน (Page) จากชั้นวางหนังสือ (หน่วยความจำสำรอง) มาวางบนช่องว่างบนโต๊ะ (Frame) เพื่อทำการอ่าน
- **ตารางเพจ (Page Table):** ทำหน้าที่บันทึกบทที่ 1 วางอยู่ที่ช่อง A, บทที่ 3 วางอยู่ที่ช่อง B เป็นต้น



รูปที่ 7- 105 แบบจำลองเปรียบเทียบสภาวะไม่พบหน้าข้อมูลในหน่วยความจำหลัก (Page Fault)

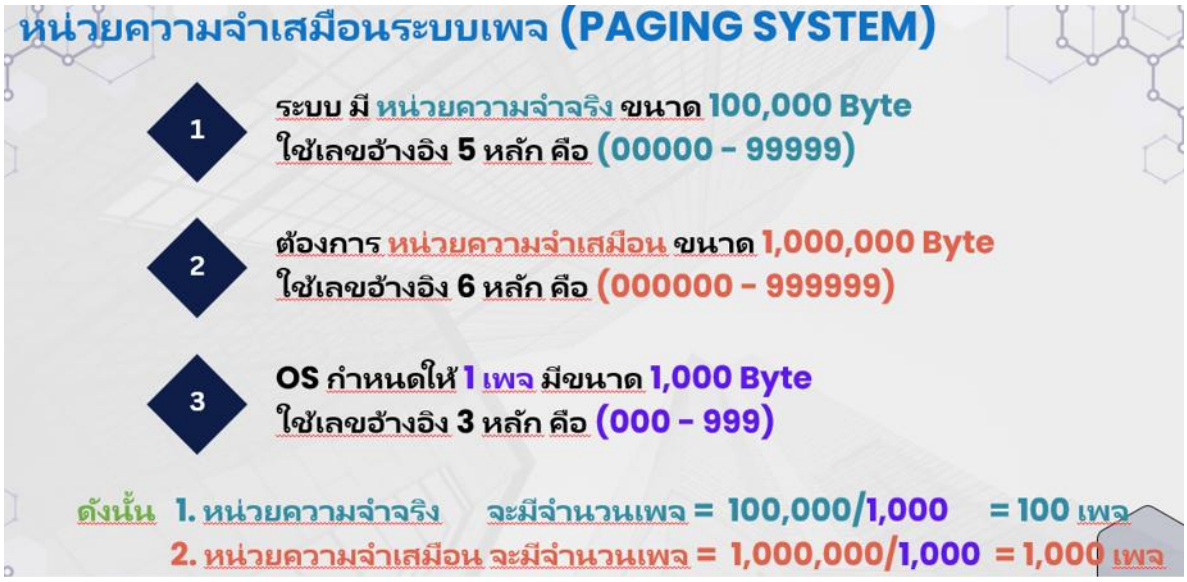
3. การเกิด Page Fault

เมื่อเราอ่านบทที่ 1 จบ แล้วต้องการจะอ่านบทที่ 2 ต่อ แต่พบว่าบทที่ 2 ยังไม่ได้ถูกนำมาวางบนโต๊ะ (ยังไม่ได้ถูกโหลดเข้าหน่วยความจำหลัก) นี่คือสิ่งที่เรียกว่า **การผิดเพจ (Page Fault)**

- **กระบวนการ:** เราจะหยุดการอ่านชั่วคราว แล้วเดินไปที่ชั้นวางหนังสือ (หน่วยความจำสำรอง) เพื่อหยิบบทที่ 2 มาวางบนโต๊ะ
- **ผลลัพธ์:** การกระทำนี้เสียเวลาเล็กน้อย แต่เราก็สามารถอ่านบทต่อไปได้ และระบบก็ยังคงทำงานต่อไปได้

สรุป: Paging System จึงเป็นวิธีการที่ช่วยให้เราสามารถทำงานกับโปรแกรมที่มีขนาดใหญ่กว่าหน่วยความจำหลักได้ โดยอาศัยการแบ่งข้อมูลและสลับเปลี่ยนไปมาอย่างชาญฉลาด ซึ่งช่วยเพิ่มประสิทธิภาพในการใช้งานทรัพยากรของระบบคอมพิวเตอ

11.4 การคำนวนเพื่อสร้าง หน่วยความจำเสมือน



รูปที่ 7- 106 กระบวนการแปลงแอดเดรสเสมือนเป็นแอดเดรสจริงและการทำงานของ Page System

การทำงานของหน่วยความจำเสมือนจากโจทย์

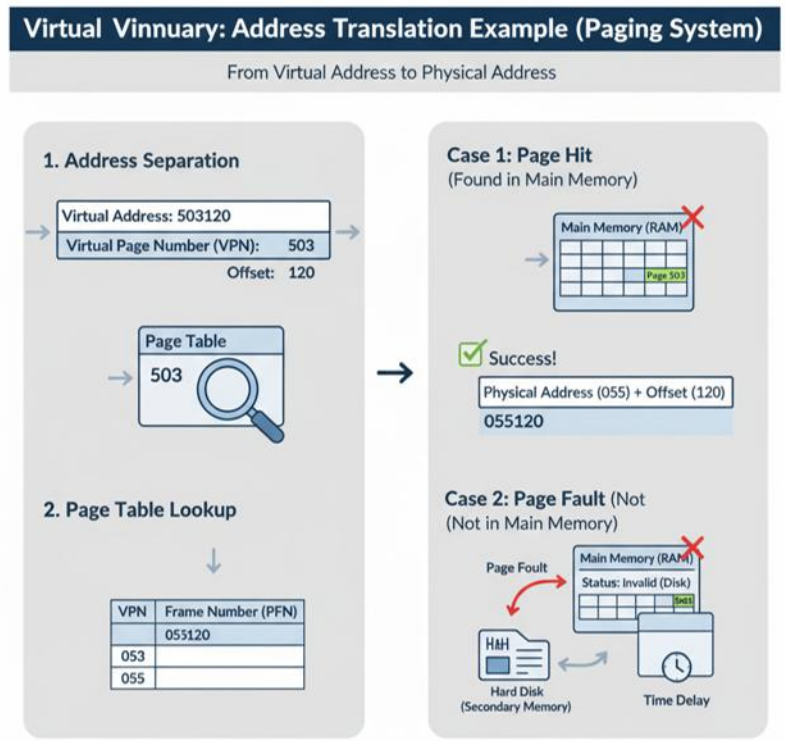
- **ที่อยู่เสมือน (Virtual Address):** มีขนาด 6 หลัก (000000 - 999999) ซึ่งหมายถึงโปรแกรมสามารถใช้ที่อยู่ได้ถึง 1,000,000 ไบต์ แต่ในความเป็นจริงแล้วหน่วยความจำหลักมีขนาดเพียง 100,000 ไบต์
- **ที่อยู่จริง (Physical Address):** มีขนาด 5 หลัก (00000 - 99999) ซึ่งเป็นขนาดที่แท้จริงของหน่วยความจำหลัก
- **ขนาดเพจ (Page Size):** 1,000 ไบต์
- **หน่วยความจำจริง:** มี 100 เฟรม (หรือเพจ) เนื่องจาก 100,000/1,000=100
- **หน่วยความจำเสมือน:** มี 1,000 เพจ เนื่องจาก 1,000,000/1,000=1,000

ดังนั้นแล้ว ถึงแม้ว่าโปรแกรมจะมีขนาดใหญ่ถึง 1,000,000 ไบต์ แต่ระบบจะจัดเก็บโปรแกรมลงในหน่วยความจำหลักที่มีขนาด 100,000 ไบต์ โดยใช้เทคนิคการแบ่งโปรแกรมเป็น **เพจ** และนำไปจัดเก็บในส่วนของหน่วยความจำจริงที่เรียกว่า **เฟรม**

ตัวอย่างการแปลงที่อยู่ (Address Translation)

สมมติว่า CPU ต้องการเข้าถึงข้อมูลที่ที่อยู่เสมือน 503120 ในระบบเพจ

1. **การแยกที่อยู่:** ที่อยู่เสมือนจะถูกแบ่งเป็น 2 ส่วน
    - **หมายเลขเพจเสมือน (Virtual Page Number - VPN):** ที่อยู่ 3 หลักแรก คือ 503
    - **ออฟเซ็ต (Offset):** ที่อยู่ 3 หลักหลัง คือ 120



รูปที่ 7- 107 ขั้นตอนการแยกแอดเดรสและการเปรียบเทียบสภาวะ Page Hit และ Page Fault ในระบบเพจ

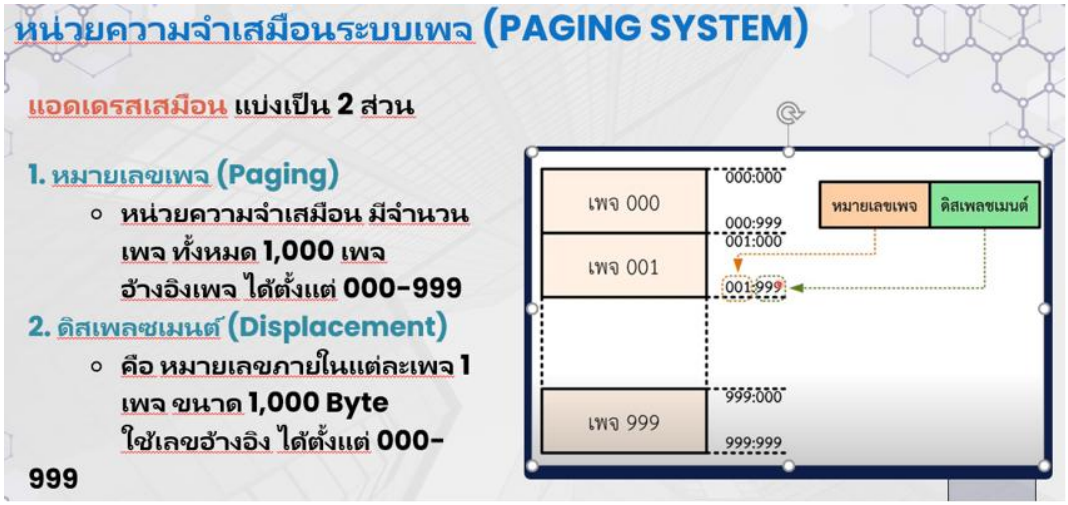
2. **การค้นหาในตารางเพจ (Page Table):** ระบบปฏิบัติการจะใช้ VPN (503) เพื่อไปค้นหาใน **ตารางเพจ (Page Table)** . ตารางนี้จะบอกว่าเพจหมายเลข 503 ถูกเก็บไว้ที่เฟรมหมายเลขอะไรในหน่วยความจำจริง

- **กรณีที่ 1: เพจอยู่ที่หน่วยความจำหลัก (Page Hit)** สมมติว่าตารางเพจบอกว่าเพจ 503 อยู่ที่เฟรมหมายเลข 055 **ที่อยู่จริง** คือ 055 (หมายเลขเฟรม) + 120 (ออฟเซ็ต) = **055120**
- **กรณีที่ 2: เพจไม่ได้อยู่ที่หน่วยความจำหลัก (Page Fault)** ถ้าตารางเพจบอกว่าเพจ 503 ยังไม่ได้ถูกโหลดเข้าหน่วยความจำหลัก (อาจจะอยู่บนฮาร์ดดิสก์) จะเกิด **การผิดเพจ (Page Fault)** ระบบปฏิบัติการจะต้องหยุดการทำงานชั่วคราวเพื่อนำเพจ 503 จากฮาร์ดดิสก์มาโหลดไว้ในเฟรมที่ว่างในหน่วยความจำหลักก่อน

สรุป

การแปลงที่อยู่แบบนี้ทำให้โปรแกรมสามารถมองเห็นที่อยู่ได้ถึง 1,000,000 ไบต์ ซึ่งเป็น **หน่วยความจำเสมือน** ในขณะที่ระบบคอมพิวเตอร์ทำงานอยู่บน **หน่วยความจำจริง** ขนาด 100,000 ไบต์ อย่างมีประสิทธิภาพ

11.5 หลักการของหน่วยความจำเสมือนระบบเพจ



รูปที่ 7- 108 โครงสร้างและการแบ่งส่วนแอดเดรสเสมือน (Virtual Address Components) ในระบบเพจ

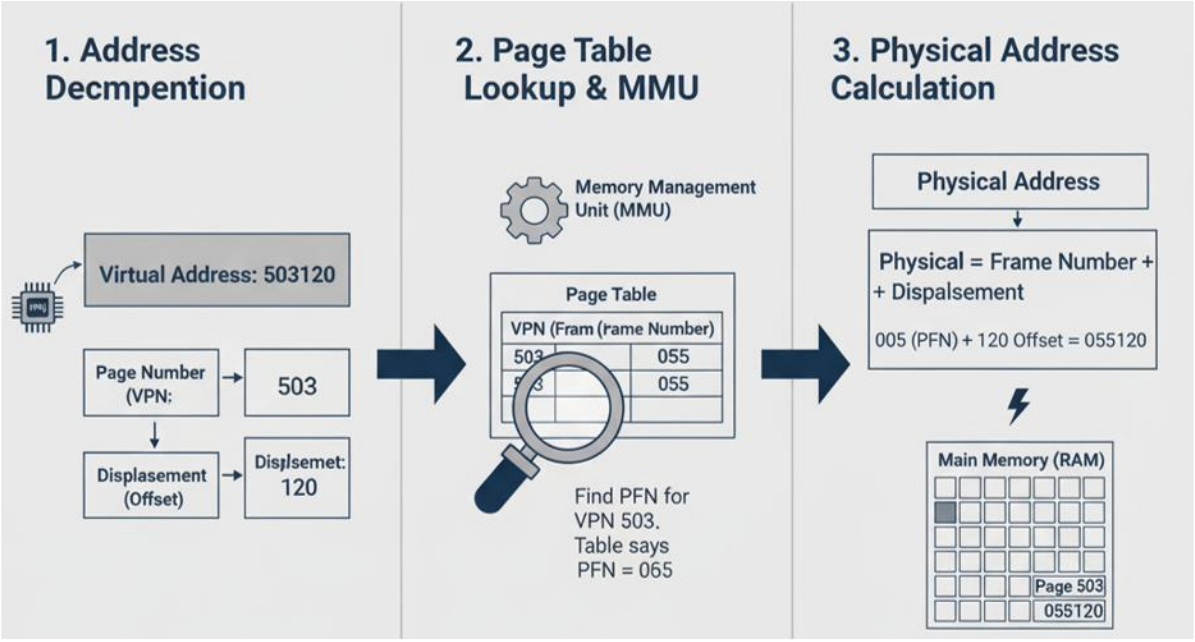
ระบบเพจเป็นเทคนิคที่ใช้จัดการหน่วยความจำเสมือน (Virtual Memory) โดยจะแบ่งหน่วยความจำออกเป็นส่วนย่อยๆ ที่มีขนาดคงที่ เพื่อให้โปรแกรมขนาดใหญ่สามารถทำงานบนหน่วยความจำจริง (Main Memory) ที่มีขนาดเล็กกว่าได้ แอดเดรสเสมือน (Virtual Address) จะถูกแบ่งออกเป็น 2 ส่วนหลัก คือ:

- **หมายเลขเพจ (Paging):** เป็นตัวระบุว่าข้อมูลนั้นอยู่ในเพจที่เท่าไหร่ของหน่วยความจำเสมือน ในกรณีนี้คือมีทั้งหมด 1,000 เพจ อ้างอิงได้ตั้งแต่เพจ 000 ถึง 999
- **ดิสเพลซเมนต์ (Displacement):** คือค่าที่ใช้ระบุตำแหน่งของข้อมูลภายในเพจนั้นๆ เนื่องจากแต่ละเพจมีขนาด 1,000 ไบต์ จึงสามารถใช้เลขอ้างอิงได้ตั้งแต่ 000 ถึง 999

11.5.1การทำงานของระบบเพจ :การทำงานของระบบเพจคือการแปลงที่อยู่เสมือน (Virtual Address) ที่ CPU สร้างขึ้นไปเป็นที่อยู่จริง (Physical Address) ในหน่วยความจำหลัก โดยมีกระบวนการดังนี้:

1. **CPU สร้างที่อยู่เสมือน:** เมื่อ CPU ต้องการเข้าถึงข้อมูล จะสร้างที่อยู่เสมือนขึ้นมา ซึ่งประกอบด้วย **หมายเลขเพจ** และ **ดิสเพลซเมนต์**
    - **การแปลงที่อยู่ (Address Translation):** ระบบปฏิบัติการจะใช้ **ตารางเพจ (Page Table)** เพื่อหาว่าเพจเสมือนที่ CPU ต้องการนั้นถูกจัดเก็บไว้ที่เฟรม (Frame) ใดในหน่วยความจำหลัก และทำหน้าที่เป็นสารบัญที่เก็บข้อมูลการจับคู่ระหว่างหมายเลขเพจเสมือนกับหมายเลขเฟรมจริง
2. **การเข้าถึงที่อยู่จริง:** เมื่อได้หมายเลขเฟรมแล้ว ระบบจะนำหมายเลขเฟรมนั้นมารวมกับค่า **ดิสเพลซเมนต์** เพื่อสร้างเป็นที่อยู่จริง ซึ่งเป็นที่อยู่ของข้อมูลในหน่วยความจำหลักนั่นเอง
3. **กรณี Page Fault:** หากระบบตรวจสอบในตารางเพจแล้วพบว่าเพจที่ต้องการยังไม่ได้ถูกโหลดเข้ามาในหน่วยความจำหลัก จะเกิด **Page Fault** ซึ่งระบบปฏิบัติการจะต้องหยุดการทำงานชั่วคราวเพื่อไปโหลดเพจนั้นจากหน่วยความจำสำรอง (ฮาร์ดดิสก์) เข้ามาในหน่วยความจำหลักก่อน

ตัวอย่างการทำงานสมมติว่า CPU ต้องการเข้าถึงข้อมูลที่ **ที่อยู่เสมือน** 503120



รูปที่ 7- 109 ขั้นตอนการแปลงแอดเดรสเสมือนเป็นแอดเดรสจริงโดยหน่วยจัดการหน่วยความจำ (MMU)

1.**การแยกส่วนประกอบ:**

1.1 หมายเลขเพจ (Page Number): คือส่วน 3 หลักแรกของที่อยู่เสมือน นั่นคือ **503**

1.2 ดิสเพลซเมนต์ (Displacement): คือส่วน 3 หลักหลังของที่อยู่เสมือน นั่นคือ **120**

2.การค้นหาในตารางเพจ: ระบบจะนำ **หมายเลขเพจ 503** ไปตรวจสอบใน **ตารางเพจ**

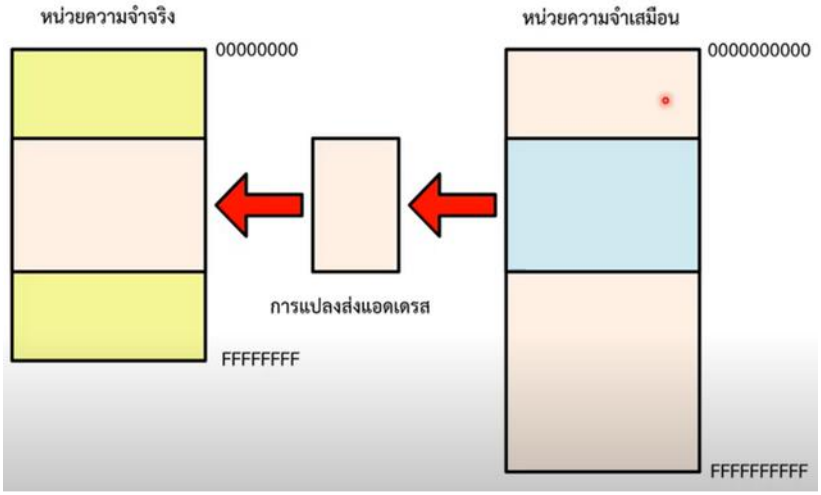
2.1 สมมติว่าตารางเพจระบุ เพจหมายเลข 503 จัดเก็บอยู่ที่ **เฟรมหมายเลข 055** ในหน่วยความจำหลัก

3.การคำนวณที่อยู่จริง:

- ระบบจะนำ **หมายเลขเฟรม** ที่หาได้คือ **055** มารวมกับ **ดิสเพลซเมนต์** คือ **120**
- ดังนั้น **ที่อยู่จริง** (Physical Address) คือ **055120**

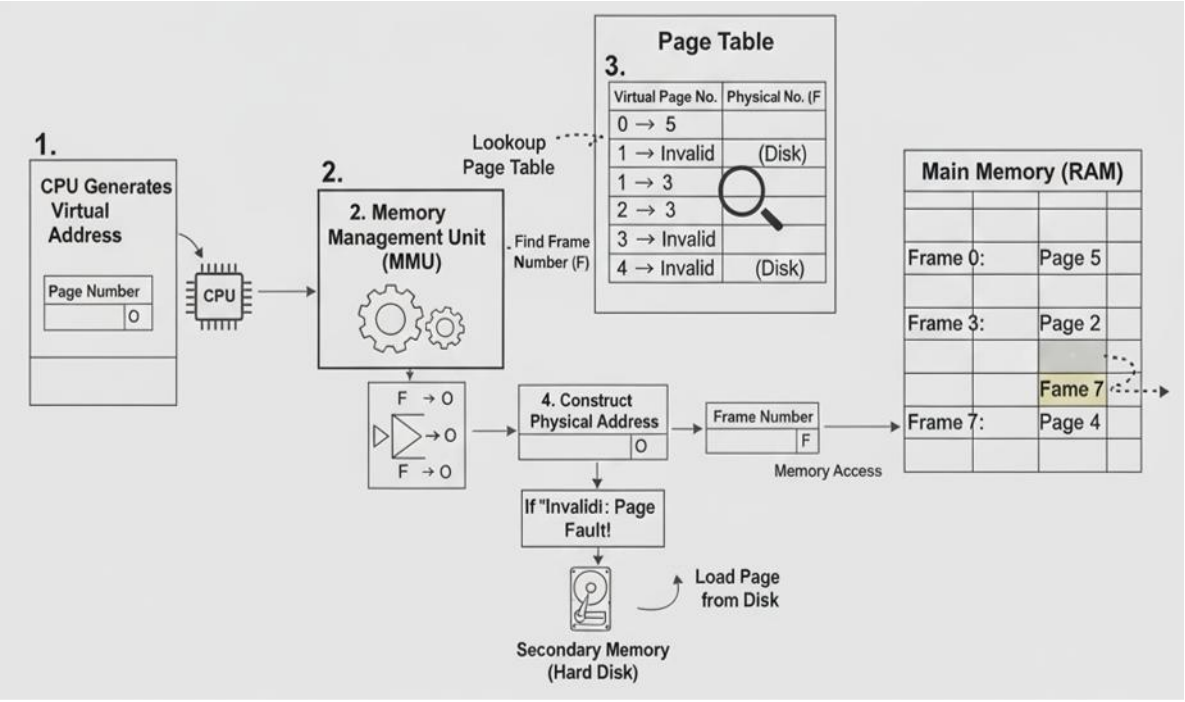
กระบวนการนี้จะเกิดขึ้นอย่างรวดเร็วโดยหน่วยประมวลผลการจัดการหน่วยความจำ (Memory Management Unit - MMU) ซึ่งเป็นส่วนประกอบของฮาร์ดแวร์ที่ทำหน้าที่แปลงที่อยู่โดยเฉพาะ

11.5.2. การแปลงส่งแอดเดรส (Address Mapping) : การที่ขนาดของโปรแกรมมีขนาดโตกว่าขนาดของหน่วยความจำ ดังนั้น แอดเดรสที่อ้างอิงในโปรแกรม จะแตกต่างกับแอดเดรสของหน่วยความจำจริง โดยแอดเดรสในหน่วยความจำจริง เรียกว่า แอดเดรสจริง (Real Address หรือ Physical Address) และ แอดเดรสที่อ้างถึงภายในโปรแกรม เรียกว่า แอดเดรสเสมือน (Virtual Address หรือ Logical Address)



รูปที่ 7- 110 กระบวนการเชื่อมโยงและแปลงส่งแอดเดรสระหว่างหน่วยความจำเสมือนและหน่วยความจำจริง

11.5.3 การแปลงส่งแอดเดรส (Address Mapping) แบบ Dynamic Address Translation - DAT

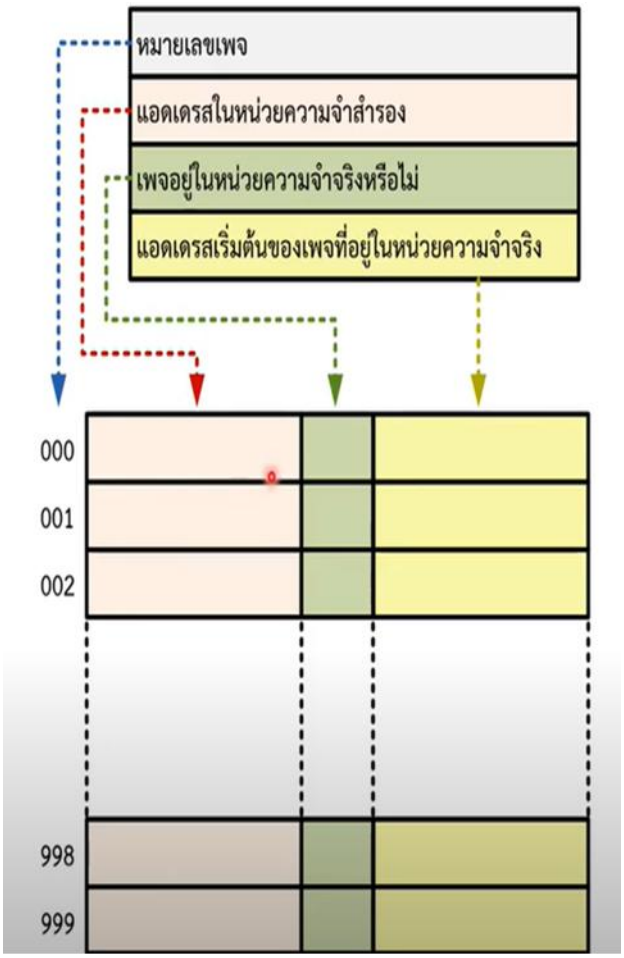


รูปที่ 7- 111 ผังจำลองขั้นตอนการแปลงที่อยู่และกลไกการเกิด Page Fault โดยหน่วย MMU

การทำงานของระบบเพจจะใช้เทคนิคที่เรียกว่า **Dynamic Address Translation (DAT)** หรือ **Memory Management Unit (MMU)** ในการแปลงแอดเดรส ซึ่งเป็นกระบวนการที่ฮาร์ดแวร์จะทำหน้าที่แปลง **แอดเดรสเสมือน (Virtual Address)** ที่ CPU สร้างขึ้นให้เป็น **แอดเดรสจริง (Physical Address)** ที่ใช้ในการเข้าถึงหน่วยความจำหลัก โดยใช้ **ตารางเพจ (Page Table)** เป็นข้อมูลอ้างอิง

ขั้นตอนการทำงานของ DAT

1. **CPU สร้างแอดเดรสเสมือน:** CPU จะสร้างแอดเดรสเสมือนที่ประกอบด้วย **หมายเลขเพจ (Page Number)** และ **ส่วนชดเชยในเพจ (Offset)**
2. **ค้นหาในตารางเพจ:** MMU จะใช้หมายเลขเพจเพื่อค้นหาในตารางเพจของโปรเซส
3. **หาหมายเลขเฟรม:** ตารางเพจจะให้ข้อมูลว่าเพจที่ต้องการถูกเก็บอยู่ที่ **หมายเลขเฟรม (Frame Number)** ใดในหน่วยความจำหลัก
4. **สร้างแอดเดรสจริง:** MMU จะนำหมายเลขเฟรมที่ได้มาต่อกับส่วนชดเชย (Offset) เพื่อสร้างเป็น **แอดเดรสจริง (Physical Address)** ที่สามารถใช้เข้าถึงข้อมูลในหน่วยความจำหลักได้จริง



รูปที่ 7- 112 โครงสร้างและการจัดสรรฟิลด์ข้อมูลภายในตารางเพจ (Page Table Entry Fields)

<u>ตัวอย่างตาราง</u>

ระบบ OS จะสร้างตารางหน้า (Page Table) ให้กับแต่ละโปรเซสไว้ โดยจะบอกข้อมูลดังนี้

1.บอกตำแหน่งที่อยู่ในหน่วยความจำสำรอง

2.บอกว่า หน้าดังกล่าวอยู่ในหน่วยความจำจริงหรือไม่

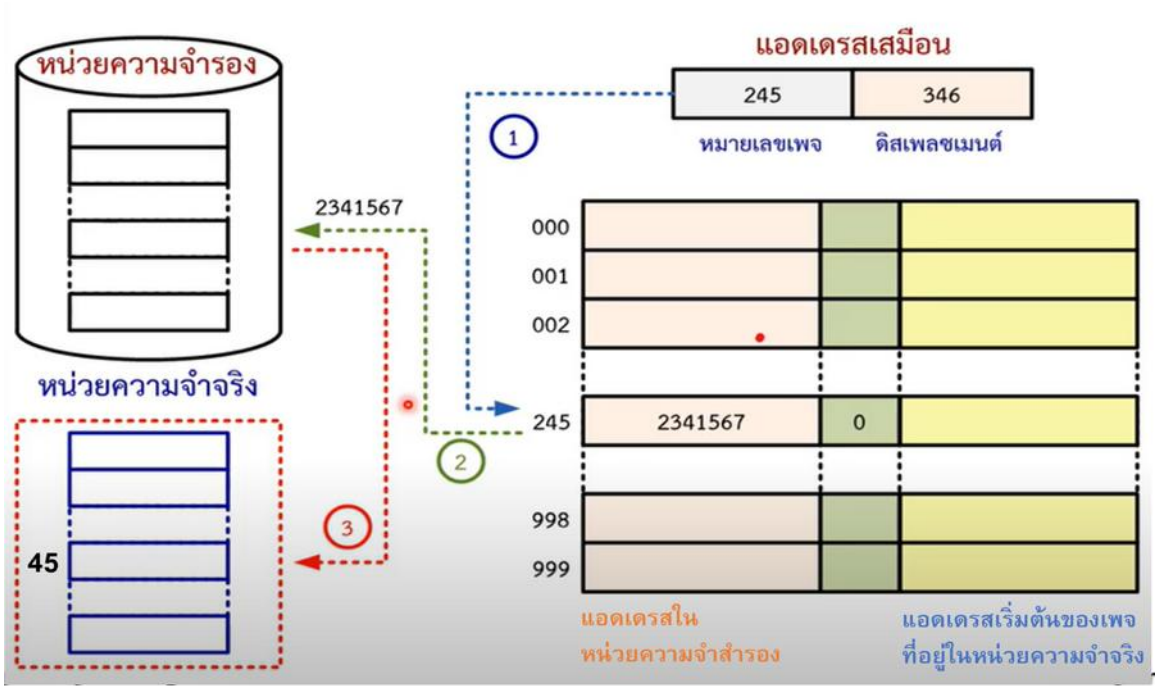
0 บอกว่า เพจนั้นไม่ได้อยู่ในหน่วยความจำจริง

1 บอกว่า เพจนั้นอยู่ในหน่วยความจำจริง และถ้าพยายามเข้าถึงข้อมูลของหน้าที่กำหนดเป็น 0 จะทำให้เกิดปัญหาการผิดเพจ (Page Fault) ได้

3.บอกตำแหน่งเริ่มต้นของเพจที่อยู่ในหน่วยความจำจริง (ตามด้วยค่า displacement เริ่มจาก 000)

11.6 การบริหารการจัดการหน่วยความจำเสมือน (Virtual Memory)

ตัวอย่างที่ 1 การหาค่าแอดเดรสในหน่วยความจำจริง

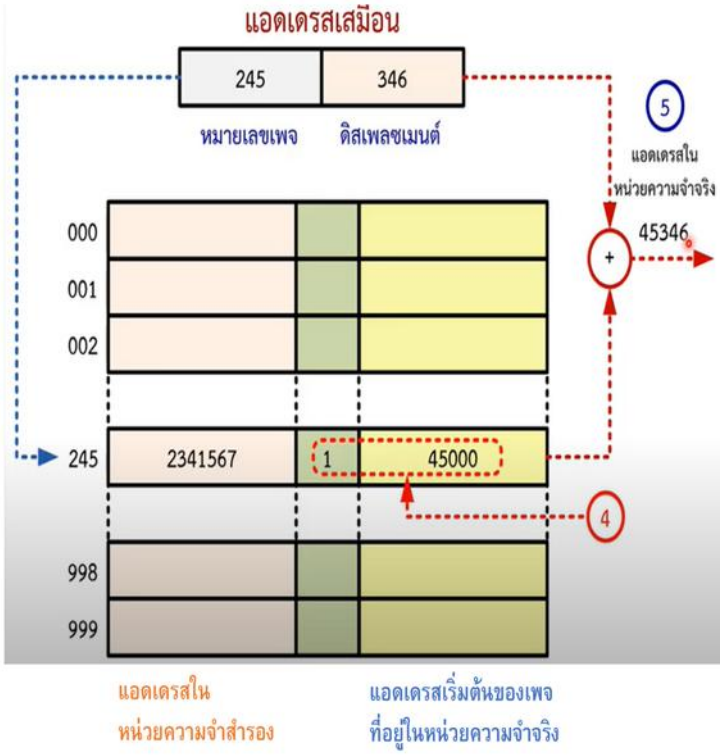


รูปที่ 7- 113 ผังจำลองขั้นตอนการเกิดสภาวะ Page Fault เมื่อเรียกใช้เพจที่ไม่ได้อยู่ในหน่วยความจำหลัก

จากรูปที่ 7 – 112 การหาค่าแอดเดรสในหน่วยความจำจริง (Physical Address) จะมีขั้นตอนดังนี้:

1. **CPU สร้างที่อยู่เสมือน (Virtual Address):** สมมติว่า CPU ต้องการเข้าถึงข้อมูลที่ตำแหน่ง **0100101011111000** ซึ่งเป็นที่อยู่เสมือนที่มีขนาด 16 บิต
2. **แบ่งส่วนที่อยู่เสมือน:** ที่อยู่เสมือนนี้จะถูกแบ่งเป็น 2 ส่วน คือ
   - 2.1 **หมายเลขเพจ (Virtual Page Number):** บิต 8 บิตแรก คือ **01001010**
   - 2.2 **ออฟเซ็ต (Offset):** บิต 8 บิตหลัง คือ **11111000**
3. **ค้นหาในตารางเพจ (Page Table):** ระบบจะใช้ **หมายเลขเพจ (01001010)** ไปค้นหาใน **ตารางเพจ** เพื่อหาว่าเพจนี้ถูกเก็บไว้ที่เฟรม (Frame) ใดในหน่วยความจำจริง
4. **ได้หมายเลขเฟรม (Frame Number):** เมื่อพบในตารางเพจแล้ว จะได้หมายเลขเฟรมที่เกี่ยวข้องออกมา เช่น ในภาพคือ **11011001**
5. **สร้างที่อยู่จริง (Physical Address):** นำ **หมายเลขเฟรม (11011001)** มาต่อเข้ากับ **ออฟเซ็ต (11111000)** ที่อยู่จริงที่ได้คือ **1101100111111000** ซึ่งเป็นที่อยู่ข้อมูลที่ต้องการในหน่วยความจำจริง

ตัวอย่างที่ 2 การหาค่าแอดเดรสในหน่วยความจำจริง



รูปที่ 7- 114 ผังจำลองขั้นตอนการคำนวณและแปลงที่อยู่ข้อมูลในสภาวะ Page Hit

จากภาพการทำงานของระบบเพจจะเริ่มต้นจาก **CPU** และสิ้นสุดที่การเข้าถึงหน่วยความจำจริง โดยมีขั้นตอนดังนี้

1. **CPU Generates Virtual Address:** CPU สร้างที่อยู่เสมือนขึ้นมา ซึ่งประกอบด้วย หมายเลขเพจและออฟเซ็ต

2. **Page Table:** หมายเลขเพจจะถูกส่งไปยังตารางเพจ เพื่อหาว่าเพจนั้นถูกเก็บไว้ที่เฟรมใดในหน่วยความจำหลัก

3. **MMU (Memory Management Unit):** หน่วยนี้จะตรวจสอบว่าเพจที่ต้องการอยู่ในหน่วยความจำหลักหรือไม่

3.1 ถ้าใช่ (Page Hit): MMU จะใช้ข้อมูลจากตารางเพจเพื่อสร้างที่อยู่จริง และส่งคำสั่งให้เข้าถึงหน่วยความจำจริงได้ทันที

3.2 ถ้าไม่ใช่ (Page Fault): MMU จะแจ้งให้ระบบปฏิบัติการ (OS) ทราบว่าเกิด Page Fault

4 **Operating System (OS):** เมื่อเกิด Page Fault ระบบปฏิบัติการเข้าควบคุมการทำงานเพื่อแก้ไขปัญหานี้

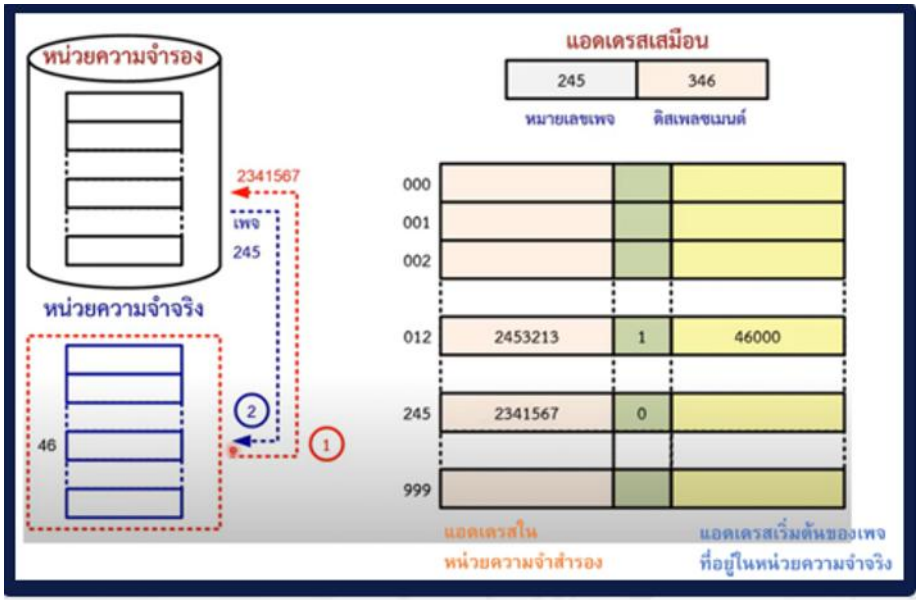
4.1 หาพื้นที่ว่าง: OS จะหาเฟรมว่างในหน่วยความจำหลัก (หากไม่มี จะต้องมีการเลือกเพจเก่าเพื่อสลับออก)

4.2 Load Page: OS จะไปโหลดเพจที่ต้องการจากหน่วยความจำสำรอง (Secondary Storage) เข้ามาเก็บในเฟรมที่หาได้

4.3 Update Page Table: OS จะอัปเดตตารางเพจเพื่อให้การอ้างอิงที่อยู่ของเพจใหม่ถูกต้อง

5 **Access Physical Memory:** เมื่อเพจที่ต้องการถูกโหลดเข้ามาในหน่วยความจำหลักเรียบร้อยแล้ว การเข้าถึงที่อยู่จริงก็จะเกิดขึ้นได้ในที่สุด

ตัวอย่างที่ 3 การหาค่าแอดเดรสในหน่วยความจำจริง (กรณีหน่วยความจำจริงเต็ม)

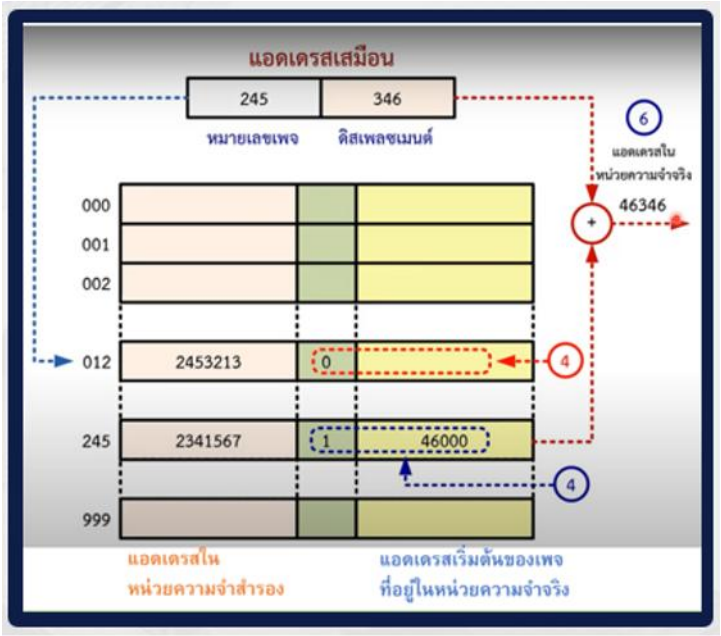


รูปที่ 7- 115 ผังจำลองขั้นตอนการโหลดและปรับปรุงสถานะตารางเพจหลังเกิด Page Fault

จากภาพเป็นการอธิบายสถานการณ์ที่หน่วยความจำหลัก (Main Memory) ถูกใช้งานจนเต็มแล้ว ซึ่งการจะนำเพจใหม่เข้ามาได้นั้น จำเป็นต้องมีการนำเพจเก่าออกไปเพื่อเคลียร์พื้นที่ . นี่คือขั้นตอนที่เรียกว่า **การสับเปลี่ยนหน้า (Page Replacement)** โดยมีหลักการทำงานดังนี้

- **Page Fault (การผิดเพจ):** เมื่อ CPU ต้องการเข้าถึงข้อมูลที่อยู่ในเพจที่ยังไม่ได้ถูกโหลดเข้าหน่วยความจำหลัก ระบบปฏิบัติการจะเกิด Page Fault ขึ้น
- **ตรวจสอบหน่วยความจำจริง:** ระบบจะตรวจสอบว่ามีเฟรม (Frame) ว่างในหน่วยความจำหลักหรือไม่
- **การสับเปลี่ยนเพจ:** เมื่อพบว่าหน่วยความจำหลักเต็ม (ไม่มีเฟรมว่าง) ระบบจะเรียกใช้อัลกอริทึมการสับเปลี่ยนหน้า (Page Replacement Algorithm) เพื่อเลือกเพจที่เหมาะสมที่จะนำออกไปจากหน่วยความจำหลัก
- **กระบวนการสลับข้อมูล:**
    1. ระบบจะนำเพจที่ถูกเลือกออกจากหน่วยความจำหลัก (หากเพจนั้นมีการแก้ไขข้อมูล จะต้องเขียนข้อมูลนั้นกลับไปที่หน่วยความจำสำรองก่อน)
    2. จากนั้น ระบบจะโหลดเพจที่ต้องการจากหน่วยความจำสำรองเข้ามาแทนที่ในเฟรมที่ว่างลง

ตัวอย่างที่ 4 การหาค่าแอดเดรสในหน่วยความจำจริง (กรณีหน่วยความจำจริงเต็ม)



รูปที่ 7- 116 ตัวอย่างการคำนวณและแปลงที่อยู่ข้อมูลใน สภาวะปกติ (Page Hit) หลังการปรับปรุงตารางเพจ

จากรูปที่ 7-116 การทำงานในแต่ละขั้นสามารถอธิบายได้ดังนี้

1. **CPU Generates Virtual Address:** ขั้นตอนแรก CPU จะสร้าง **ที่อยู่เสมือน (Virtual Address)** ที่ประกอบด้วย **หมายเลขเพจ (Page Number)** และ **ออฟเซ็ต (Offset)**
2. **Virtual Page Number:** ระบบจะนำหมายเลขเพจเสมือนนี้ไปค้นหาใน **ตารางเพจ (Page Table)**
3. **Page Fault Check:**

   3.1 ถ้าเพจอยู่บนหน่วยความจำหลัก (Page Hit): ตารางเพจจะให้ **หมายเลขเฟรม (Frame Number)** ที่เพจนั้นถูกเก็บไว้ ระบบจะนำหมายเลขเฟรมมารวมกับออฟเซ็ตเพื่อสร้าง **ที่อยู่จริง (Physical Address)** และเข้าถึงข้อมูลได้ทันที

   3.2 ถ้าเพจไม่ได้อยู่บนหน่วยความจำหลัก (Page Fault): ตารางเพจจะแจ้งว่าเพจที่ต้องการยังไม่ได้ถูกโหลดเข้ามา

4 **Process Page Fault:**

   4.1 เลือกเพจที่จะสับเปลี่ยน: ระบบจะเลือกเพจที่อยู่ในหน่วยความจำหลักเพื่อนำออกไป (ตาม Page Replacement Algorithm)

   4.2 โหลดเพจใหม่: ระบบจะโหลดเพจที่ต้องการจากหน่วยความจำสำรอง (ฮาร์ดดิสก์) เข้ามาแทนที่

   4.3 อัปเดตตารางเพจ: ระบบจะอัปเดตตารางเพจเพื่อให้การอ้างอิงที่อยู่ของเพจใหม่ถูกต้อง

5 **Access Physical Memory:** หลังจากโหลดเพจใหม่เข้ามาแล้ว ระบบจะสามารถเข้าถึงข้อมูลที่ที่อยู่จริงได้ตามปกติ

11.6 การบริหารหน่วยความจำเสมือน (Virtual Memory) แบบการสับเปลี่ยนหน้า (PAGE REPLACEMENT ALGORITHMS)

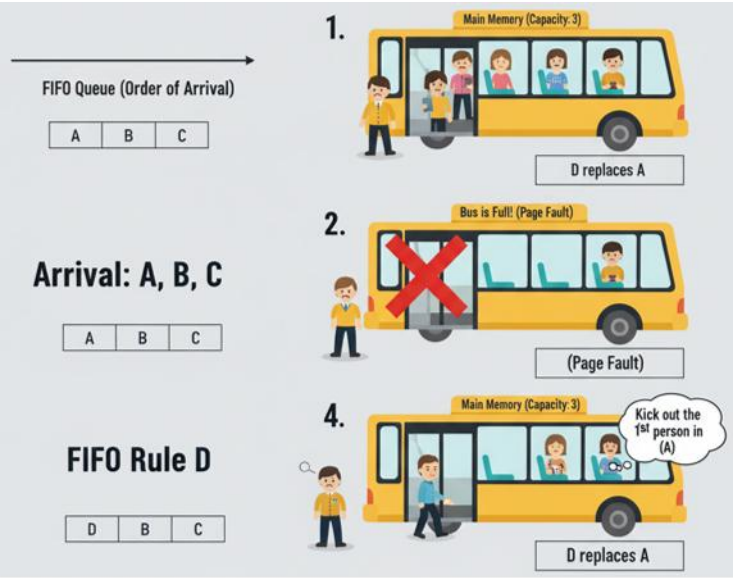
การสับเปลี่ยนหน้าคือวิธีการที่ระบบปฏิบัติการนำมาใช้เพื่อแก้ปัญหา **การผิดเพจ (Page Fault)** เมื่อหน่วยความจำหลัก (Main Memory) เต็ม โดยมีหน้าที่หลักคือการเลือกเพจ (Page) ที่อยู่ในหน่วยความจำหลักอยู่แล้วเพื่อสลับออกไปเก็บไว้ในหน่วยความจำสำรอง (Harddisk) และนำเพจใหม่ที่ต้องการเข้ามาแทนที่ . อัลกอริทึมเหล่านี้มีความสำคัญอย่างยิ่งต่อประสิทธิภาพของระบบ มีวิธีการสับเปลี่ยน 3 วิธี

11.6.1. วิธีสับเปลี่ยนแบบมาก่อน-ออกก่อน (First-in First-out : FIFO)

- **หลักการ:** อัลกอริทึมนี้จะเลือกเพจที่ถูกโหลดเข้ามาในหน่วยความจำหลักเป็นเวลานานที่สุด (มาก่อน) เพื่อสลับออกไปก่อน หลักการนี้คล้ายกับการเข้าคิว
- **คำอธิบาย:** ระบบจะสร้างคิวขึ้นมาเพื่อบันทึกว่าแต่ละเพจเข้ามาในหน่วยความจำเมื่อไหร่ เมื่อเกิด Page Fault และหน่วยความจำเต็ม ระบบจะสลับเพจที่อยู่หน้าสุดของคิวออกไปเพื่อเอาเพจใหม่เข้ามาแทนที่
- **ข้อดี:** เข้าใจง่ายและนำไปใช้งานได้ง่าย
- **ข้อเสีย:** ไม่ได้พิจารณาว่าเพจที่ถูกสลับออกไปนั้นอาจจะเป็นเพจที่ถูกใช้งานบ่อย ทำให้เกิด Page Fault ซ้ำๆ ได้

ตัวอย่างเปรียบเทียบ: สมมติว่ามีรถเมล์ (หน่วยความจำหลัก) ที่จอดรับผู้โดยสาร (เพจ) ได้ 3 คน

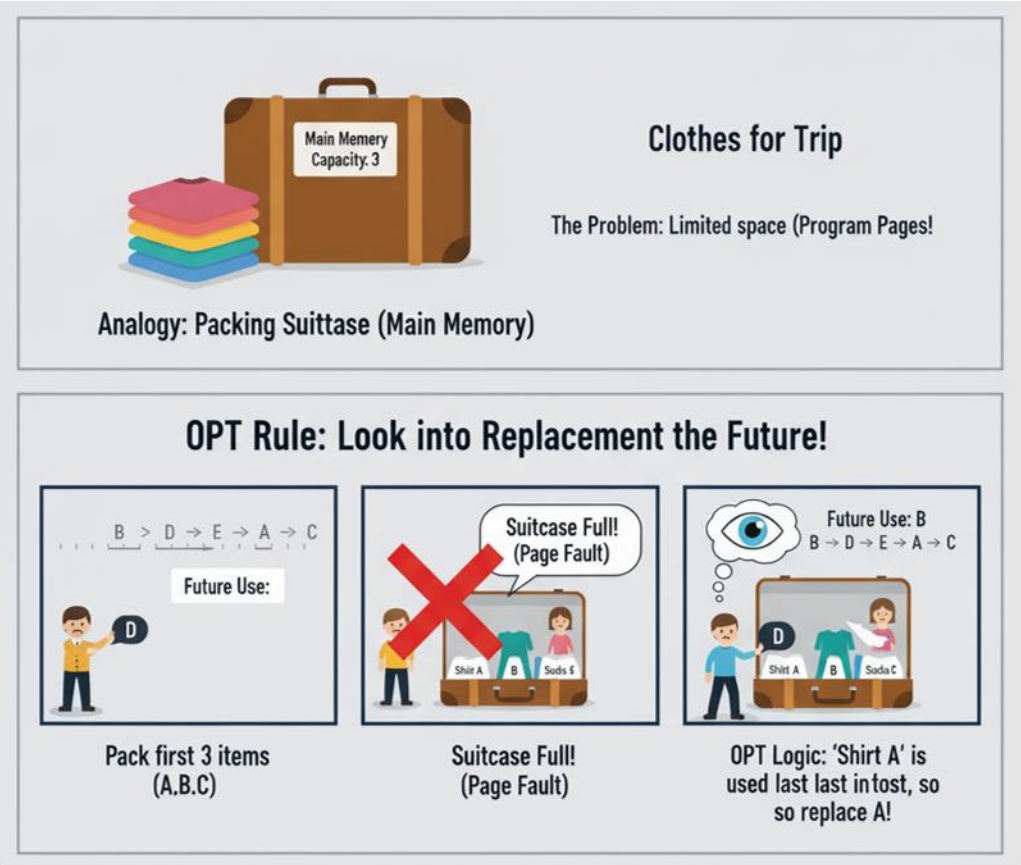
- คนแรกมา (A) ขึ้นรถ
- คนที่สองมา (B) ขึ้นรถ
- คนที่สามมา (C) ขึ้นรถ
- คนที่สี่มา (D) แต่รถเต็มแล้ว ระบบ FIFO จะไล่ **คนแรก (A)** ที่ขึ้นรถมานานที่สุดลง เพื่อให้ **คนสี่ (D)** ขึ้นแทน



รูปที่ 7- 117 แผนภาพอุปมาอุปไมยหลักการสับเปลี่ยนเพจแบบ FIFO ด้วยการจำลองที่นั่งบนรถโดยสาร

11.6.2. วิธีสับเปลี่ยนแบบดีที่สุด (Optimal Page Replacement : OPT)

- **หลักการ:** อัลกอริทึมนี้จะเลือกเพจที่ **จะไม่ถูกเรียกใช้งานอีกเป็นเวลานานที่สุดในอนาคต** เพื่อสลับออกไป ซึ่งเป็นวิธีที่ดีที่สุดในเชิงทฤษฎี .
- **คำอธิบาย:** ระบบจะพิจารณาจากลำดับการเรียกใช้งานเพจในอนาคตทั้งหมด เพื่อให้แน่ใจว่าจะไม่เกิด Page Fault กับเพจที่ถูกเรียกใช้ในเร็วๆ นี้
- **ข้อดี:** เป็นวิธีที่ทำให้เกิด Page Fault น้อยที่สุด
- **ข้อเสีย:** ไม่สามารถนำไปใช้ได้จริงในทางปฏิบัติ เพราะระบบไม่สามารถรู้ลำดับการใช้งานเพจในอนาคตล่วงหน้าได้

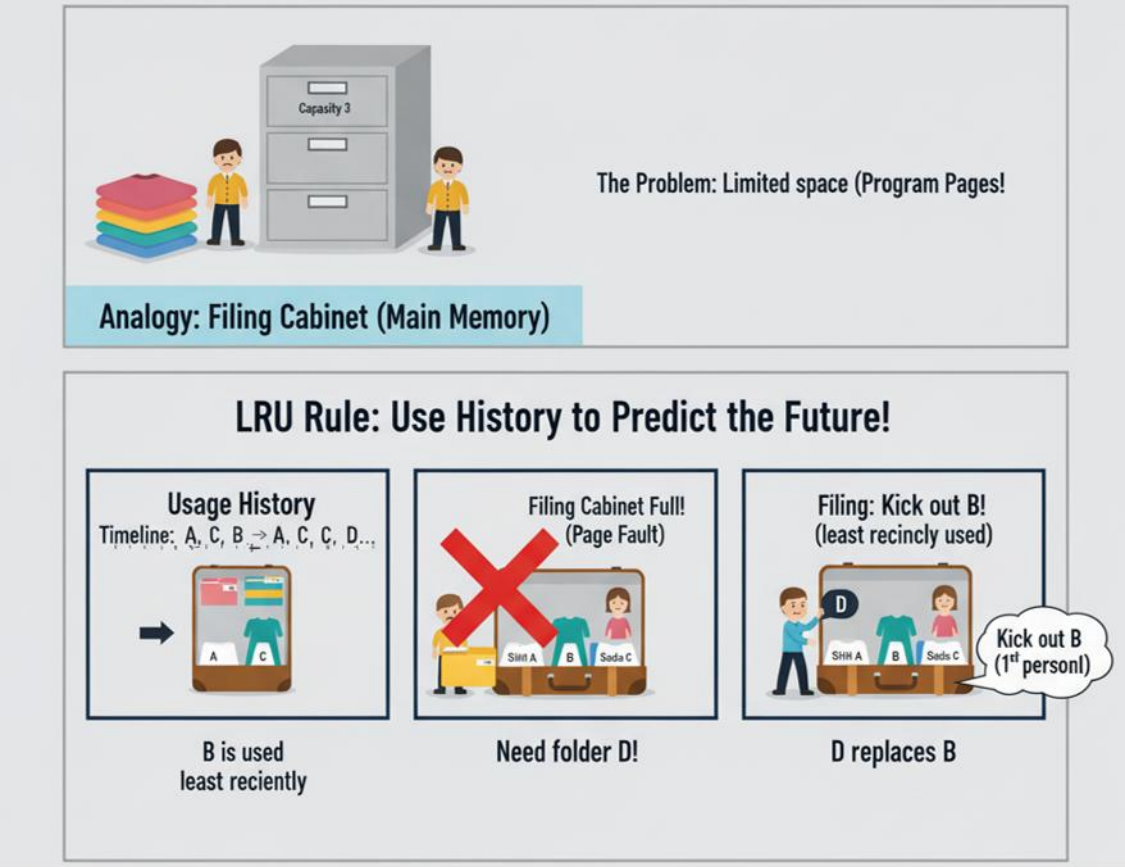


รูปที่ 7- 118 แผนภาพอุปมาอุปไมยหลักการสับเปลี่ยนเพจแบบ OPT ด้วยการจำลองการจัดกระเป๋าเดินทาง

ตัวอย่างเปรียบเทียบ: เหมือนเรากำลังเดินทางไปต่างจังหวัดแล้วต้องจัดกระเป๋าเสื้อผ้า (หน่วยความจำหลัก) แต่มีพื้นที่จำกัด เราจะเลือกเสื้อผ้าที่เราจะใส่เป็นลำดับแรกๆ ไว้ใกล้ตัว และเอาเสื้อผ้าที่เราจะใส่ตอนกลับบ้านไปไว้ล่างสุดของกระเป๋า

11.6.3. วิธีสับเปลี่ยนแบบใช้มานานที่สุด (Least Recently Used : LRU)

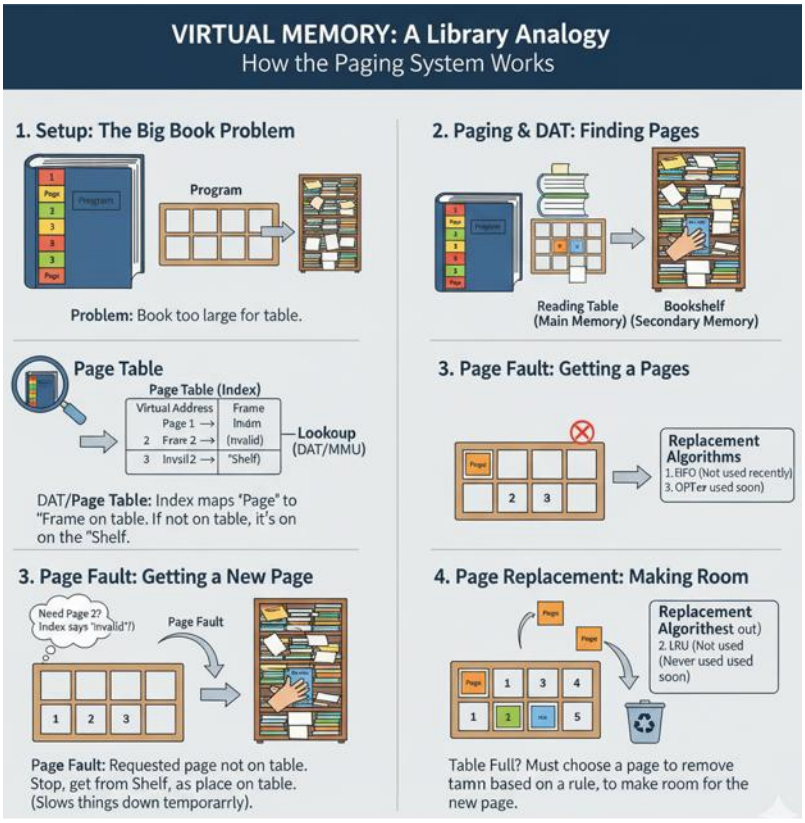
- **หลักการ:** อัลกอริทึมนี้จะเลือกเพจที่ **ถูกใช้งานล่าสุดเมื่อนานมาแล้วที่สุด** เพื่อสลับออกไป . หลักการนี้พิจารณาจากอดีต ซึ่งเป็นการคาดเดาว่าเพจที่ถูกใช้งานมานานแล้วน่าจะไม่ถูกใช้ในอนาคตอันใกล้นี้
- **คำอธิบาย:** ระบบจะเก็บข้อมูลการใช้งานล่าสุดของแต่ละเพจ เมื่อเกิด Page Fault จะเลือกสลับเพจที่มีเวลาการใช้งานล่าสุดนานที่สุดออกไป
- **ข้อดี:** เป็นอัลกอริทึมที่นิยมใช้และมีประสิทธิภาพสูง เนื่องจากเป็นการคาดเดาจากพฤติกรรมในอดีต ซึ่งมีแนวโน้มที่จะถูกต้อง
- **ข้อเสีย:** ค่อนข้างซับซ้อนในการนำไปใช้งาน เพราะต้องมีการเก็บบันทึกเวลาการใช้งานล่าสุดของทุกเพจ



รูปที่ 7- 119 แผนภาพอุปมาอุปไมยหลักการสับเปลี่ยนเพจแบบ LRU ด้วยการจำลองการจัดตู้เก็บเอกสาร

ตัวอย่างเปรียบเทียบ: เปรียบเหมือนการจัดระเบียบตู้เก็บเอกสาร เราจะเก็บเอกสารที่เราเพิ่งใช้งานไว้ด้านบนสุดและเลื่อนเอกสารที่เราไม่ได้ใช้มานานแล้วไปไว้ด้านล่างสุดของตู้ เมื่อตู้เต็มเราก็จะหยิบเอกสารที่อยู่ล่างสุดออกไปก่อนเพื่อสลับออกไปเก็บไว้ในหน่วยความจำสำรอง (Harddisk) และนำเพจใหม่ที่ต้องการเข้ามาแทนที่

ตัวอย่างเปรียบเทียบการทำงานให้เข้าใจง่าย สมมติว่า **โปรแกรม (Program)** เปรียบเสมือน **หนังสือ** เล่มใหญ่ ซึ่งมีเนื้อหามากเกินกว่าจะวางบนโต๊ะทำงานได้หมดในคราวเดียว **โต๊ะทำงาน (Main Memory)** ของเรามีขนาดจำกัด ในขณะที่ **ตู้หนังสือ (Secondary Memory)** มีพื้นที่มหาศาล



รูปที่ 7- 120 แผนภาพอุปมาอุปไมยหลักการสับเปลี่ยนเพจแบบ LRU ด้วยการจำลองการจัดตู้เก็บเอกสาร

- **หน่วยความจำเสมือน (Virtual Memory)** เปรียบได้กับการที่เราสามารถ "อ่าน" หนังสือเล่มนั้นได้ แม้ว่าจะไม่ได้เอาหนังสือทั้งเล่มมาวางบนโต๊ะ
- **เพจ (Page)** เปรียบได้กับ **หน้ากระดาษ** แต่ละหน้าของหนังสือ
- **เฟรม (Frame)** เปรียบได้กับ **พื้นที่ว่าง** บนโต๊ะทำงานที่สามารถวางหน้ากระดาษได้
- **ระบบเพจ (Paging System)** เปรียบได้กับการที่เมื่อเราจะอ่านหน้าใดหน้าหนึ่ง (Page) เราก็แค่หยิบหน้านั้นมาวางบนโต๊ะ (Frame)
- **การแปลงแอดเดรส (DAT)** เปรียบเสมือนการที่เรามี **ดัชนี (Page Table)** ที่บอกว่าหน้ากระดาษที่เราต้องการ (Virtual Address) ที่จริงแล้วอยู่บนโต๊ะทำงานตรงตำแหน่งไหน (Physical Address) และหากหน้าที่ต้องการยังอยู่ในตู้หนังสือ (Secondary Memory) เราก็จะหยิบหน้าใหม่มาวางบนโต๊ะแทนหน้าที่ไม่ได้ใช้แล้ว
- **Page Fault** ก็คือเหตุการณ์ที่เราต้องการอ่านหน้าหนังสือ แต่หน้านั้นยังอยู่ในตู้หนังสือ เราจึงต้องเสียเวลาไปหยิบมันมาก่อน
- **Page Replacement Algorithms** เปรียบได้กับกฎที่เราใช้ในการตัดสินใจว่า "จะเอาหน้าไหนออกไปจากโต๊ะ" เพื่อจะเอาหน้าใหม่เข้ามาแทนที่ เช่น
    - **FIFO (First-in First-out):** เอาหน้ากระดาษที่วางบนโต๊ะมานานที่สุดออกไปก่อน
    - **LRU (Least Recently Used):** เอาหน้ากระดาษที่ไม่ได้ถูกอ่านมานานที่สุดออกไป
    - **OPT (Optimal):** เอาหน้ากระดาษที่จะไม่ถูกอ่านในอนาคตอันใกล้ที่สุดออกไป (ซึ่งทำได้ยากในทางปฏิบัติ)

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 7 เรื่อง หน่วยความจำ (Memory Unit) จำนวน 48 ข้อ

1. หน้าที่หลักของ หน่วยความจำ (Memory Unit) คืออะไร?

ก. ควบคุมการไหลของข้อมูลในระบบบัส ข. ดำเนินการคำสั่งทางคณิตศาสตร์และตรรกะ

ค. เก็บชุดคำสั่งและข้อมูลที่ใช้ในการประมวลผล ง. แปลงที่อยู่เสมือนให้เป็นที่อยู่จริง

เฉลย: ค.

2. ในทางสถาปัตยกรรมคอมพิวเตอร์ ส่วนประกอบหลักของหน่วยความจำแบ่งออกเป็นกี่ประเภทตามหลักการทำงาน?

ก. หน่วยความจำหลัก (Main Memory) และ หน่วยความจำสำรอง (Secondary Memory)

ข. หน่วยควบคุม (Control Unit) และ หน่วย ALU

ค. หน่วยอินพุต และ หน่วยเอาต์พุต

ง. ฮาร์ดแวร์ และ ซอฟต์แวร์

เฉลย: ก.

3. ลำดับชั้นหน่วยความจำ (Memory Hierarchy) หน่วยความจำชนิดใดที่อยู่ชั้นบนสุด (เร็วที่สุด แต่มีความจุต่ำสุด)?

ก. Cache Memory (L1, L2) ข. Registers

ค. Main Memory (RAM) ง. Secondary Storage (Hard Disk)

เฉลย: ข.

4. หากเปรียบเทียบระหว่าง Cache Memory และ Main Memory (RAM) ข้อใดถูกต้อง?

ก. Cache มีความจุสูงกว่า และราคาถูกกว่า

ข. RAM มีความเร็วในการเข้าถึงเร็วกว่า Cache

ค. RAM มีความจุต่ำกว่า แต่ช้ากว่า Cache

ง. Cache มีความเร็วในการเข้าถึงเร็วกว่า แต่มีความจุต่ำกว่า

เฉลย: ง.

5. ตามหลักการออกแบบลำดับชั้นหน่วยความจำ การจัดเรียงหน่วยความจำจาก "ความเร็วสูง/ต้นทุนสูง" ไปยัง "ความเร็วต่ำ/ต้นทุนต่ำ" เป็นการดำเนินการเพื่อให้ได้คุณสมบัติใด?

ก. มีขนาดความจุรวมต่ำที่สุด ข. มีความซับซ้อนในการจัดการที่สูงที่สุด

ค. มีประสิทธิภาพรวมสูง ในต้นทุนที่เหมาะสม ง. สามารถลบเลือน (Volatile) ได้ทั้งหมด

เฉลย: ค.

6. คุณลักษณะของหน่วยความจำที่ควบคุมตำแหน่งการเข้าถึงระหว่าง หน่วยความจำภายใน (Internal Memory) และ หน่วยความจำภายนอก (External Memory) คืออะไร?

ก. ความจุ (Capacity)
ข. ตำแหน่ง (Location)
ค. การเข้าถึงข้อมูล (Access Method)
ง. การถ่ายโอนข้อมูล (Unit of Transfer)

เฉลย: ข.

7. การระบุปริมาณความจุของหน่วยความจำหลัก 1 MB (Megabyte) หมายถึงอะไร?

ก. 2^{20} Byte
ข. 2^{10} KiloByte
ค. 10^{6} Bit
ง. 2^{30} Byte

เฉลย: ก.

8. หาก Unit of Transfer ของหน่วยความจำคือแบบ บล็อก (Block) แสดงว่าการเข้าถึงข้อมูลมีลักษณะอย่างไร?

ก. ถ่ายโอนข้อมูลทีละ 1 บิต
ข. ถ่ายโอนข้อมูลทีละ 1 เวิร์ด (Word)
ค. ใช้กับการติดต่อระหว่าง CPU กับรีจิสเตอร์
ง. ใช้กับการติดต่อระหว่างหน่วยความจำที่อยู่คนละระดับชั้น (เช่น Cache กับ Main Memory)

เฉลย: ง.

9. การเข้าถึงข้อมูลแบบ Sequential Access (เช่น Magnetic Tape) ต่างจากการเข้าถึงแบบ Direct Access (เช่น Magnetic Disk) อย่างไร?

ก. Sequential Access มีความเร็วสูงกว่า
ข. Direct Access สามารถเริ่มอ่านจากตำแหน่งใดก็ได้โดยไม่ต้องไล่จากต้น
ค. Sequential Access ต้องอ่านข้อมูลตามลำดับจากต้นจนถึงตำแหน่งที่ต้องการ
ง. Direct Access ต้องใช้หัวอ่าน/เขียนแบบคงที่

เฉลย: ค.

10. การเข้าถึงข้อมูลแบบ Associative Access มีคุณสมบัติพิเศษอย่างไรเมื่อเทียบกับ Random Access?

ก. อ่านข้อมูลได้เร็วที่สุดในบรรดา Access Method ทั้งหมด
ข. เข้าถึงข้อมูลตามค่าของข้อมูลที่ต้องการค้นหา ไม่ใช่ตามที่อยู่ (Address)
ค. ใช้ได้เฉพาะกับหน่วยความจำสำรองเท่านั้น
ง. ต้องอ่านข้อมูลทีละบล็อกเท่านั้น

เฉลย: ข.

11. การติดต่อระหว่าง CPU กับหน่วยความจำหลัก (Main Memory) ต้องใช้บัสใดเป็นหลักในการระบุตำแหน่งข้อมูลที่ต้องการอ่าน/เขียน?

ก. Address Bus ข. Data Bus

ค. Control Bus ง. I/O Bus

เฉลย: ก.

12. หน่วยความจำประเภท Flash Memory มีคุณสมบัติเด่นที่ถูกนำมาใช้ทดแทน ROM คืออะไร?

ก. สามารถเขียนข้อมูลซ้ำได้หลายล้านครั้งอย่างรวดเร็ว

ข. มีความจุต่ำมาก แต่มีราคาแพง

ค. เป็นหน่วยความจำแบบลบเลือน (Volatile)

ง. สามารถเขียน/ลบข้อมูลด้วยไฟฟ้าได้ และยังคงข้อมูลไว้เมื่อไม่มีไฟเลี้ยง (Non-Volatile)

เฉลย: ง.

13. คุณสมบัติที่สำคัญของหน่วยความจำ RAM (Random Access Memory) คืออะไร?

ก. ไม่ลบเลือน (Non-Volatile)

ข. เข้าถึงข้อมูลที่ตำแหน่งใดก็ได้โดยใช้เวลาเท่ากัน (Random Access)

ค. จัดเก็บข้อมูลแบบ Sequential Access เท่านั้น

ง. มีความเร็วในการเข้าถึงช้ากว่าฮาร์ดดิสก์

เฉลย: ข.

14. ลักษณะการทำงานของหน่วยความจำ SRAM (Static RAM) คืออะไร?

ก. ต้องมีการรีเฟรชประจุไฟฟ้าอย่างสม่ำเสมอ

ข. ใช้ Capacitor ในการจัดเก็บข้อมูล

ค. ใช้ Flip-Flops (ทรานซิสเตอร์) ในการจัดเก็บข้อมูล และไม่ต้องรีเฟรช

ง. มีความจุสูงกว่าและราคาถูกกว่า DRAM

เฉลย: ค.

15. การทำงานของหน่วยความจำ DRAM (Dynamic RAM) จำเป็นต้องมีการดำเนินการใดอย่างสม่ำเสมอเพื่อรักษาข้อมูลไว้?

ก. การปรับปรุงแรงดันไฟฟ้า

ข. การทำ 2's Complement

ค. การจัดการ Page Fault

ง. การรีเฟรชประจุไฟฟ้า (Refresh) ในคาปาซิเตอร์

เฉลย: ก.

16. คำว่า SDRAM (Synchronous DRAM) ใน DDR SDRAM สื่อถึงคุณสมบัติใดในการทำงาน?

ก. เป็นหน่วยความจำแบบ Non-Volatile

ข. ทำงานแบบ Asynchronous กับสัญญาณนาฬิกาของ CPU

ค. ใช้ไฟฟ้าต่ำกว่า DRAM แบบเก่า

ง. ทำงานประสาน (Synchronous) กับสัญญาณนาฬิกาของ CPU

เฉลย: ง.

17. ความแตกต่างหลักระหว่าง DDR3 และ DDR4 RAM ในแง่ของประสิทธิภาพคืออะไร?

ก. DDR3 ใช้แรงดันไฟฟ้าต่ำกว่า

ข. DDR4 มีอัตราการถ่ายโอนข้อมูล (Data Transfer Rate) ที่สูงกว่า

ค. DDR4 ใช้ช่องเสียบ (Socket) เดียวกับ DDR3

ง. DDR3 มี Pin Connection มากกว่า

เฉลย: ข.

18. คุณสมบัติใดที่ทำให้โมดูลหน่วยความจำถูกเรียกว่า DIMM (Dual In-line Memory Module)?

ก. มีแถวของ Pin Connection เพียงแถวเดียว

ข. ต้องมีการตรวจสอบ Parity bit เสมอ

ค. มี Pin Connection ทั้งสองด้านของแผงวงจร และแต่ละด้านมีการเชื่อมต่อเป็นอิสระต่อกัน

ง. ใช้ได้เฉพาะกับหน่วยความจำแบบ SRAM เท่านั้น

เฉลย: ค.

19. NVDIMM (Non-volatile Dual inline memory module) รวมคุณสมบัติใดไว้ด้วยกัน?

ก. DRAM และ SSD

ข. RAM และ EEPROM

ค. SRAM และ Flash Memory

ง. DRAM และ Non-Volatile Storage (เช่น Flash Memory)

เฉลย: ง. (DRAM + Non-Volatile Storage, มักเป็น Flash)

20. ในหน่วยความจำแสง (Optical Memory) เช่น CD-ROM การอ่านข้อมูลทำได้อย่างไร?

ก. โดยการเหนี่ยวนำแม่เหล็ก

ข. โดยการวัดความต้านทานไฟฟ้า

ค. โดยการตรวจจับการเปลี่ยนแปลงของประจุไฟฟ้า

ง. โดยการใช้เลเซอร์อ่านการสะท้อนของแสงจากหลุม (Pits) และพื้นผิว (Lands)

เฉลย: ง.

21. หน่วยความจำสำรองประเภท เทปแม่เหล็ก (Magnetic Tape) เหมาะสมกับงานประเภทใดมากที่สุด?

ก. การเข้าถึงข้อมูลแบบสุ่ม (Random Access)

ข. การสำรองข้อมูลขนาดใหญ่และการเก็บถาวร (Archival) ที่เน้น Sequential Access

ค. การจัดเก็บแคช L1

ง. การทำงานแบบ Write-through

เฉลย: ข.

22. แผ่นฟลอปปี้ดิสก์ (Floppy Disk) จัดเป็นหน่วยความจำประเภทใด?

ก. หน่วยความจำแสง (Optical Memory)

ข. หน่วยความจำหลักแบบลบเลือน (Volatile Main Memory)

ค. หน่วยความจำสำรองแบบแม่เหล็ก (Magnetic Secondary Memory)

ง. หน่วยความจำแคชแบบ L1

เฉลย: ค.

23. จานแม่เหล็ก (Magnetic Disk) ใช้หลักการใดในการจัดเก็บข้อมูล?

ก. การเขียน/ลบโดยใช้แสงเลเซอร์ ข. การจัดเก็บประจุไฟฟ้าในคาปาซิเตอร์

ค. การจัดเก็บข้อมูลด้วยทรานซิสเตอร์ ง. การบันทึกข้อมูลโดยใช้สื่อแม่เหล็กบนแผ่นจานหมุน

เฉลย: ง.

23. จานแม่เหล็ก (Magnetic Disk) ใช้หลักการใดในการจัดเก็บข้อมูล?

ก. การบันทึกข้อมูลโดยใช้สื่อแม่เหล็กบนแผ่นจานหมุน

ข. การเขียน/ลบโดยใช้แสงเลเซอร์

ค. การจัดเก็บประจุไฟฟ้าในคาปาซิเตอร์

ง. การจัดเก็บข้อมูลด้วยทรานซิสเตอร์

เฉลย: ก.

24. หน้าที่ของ หัวอ่าน/เขียน (Read/Write Heads) ในหน่วยความจำจานแม่เหล็กคืออะไร?

ก. ควบคุมความเร็วรอบในการหมุนของจาน

ข. แปลงข้อมูลจากเลขฐานสองเป็นรหัส ASCII

ค. จัดการการเชื่อมต่อกับระบบบัส

ง. เปลี่ยนข้อมูลดิจิทัลให้เป็นสัญญาณแม่เหล็กเพื่อบันทึก
และเปลี่ยนสัญญาณแม่เหล็กกลับเป็นข้อมูลดิจิทัลเพื่ออ่าน

เฉลย: ง.

25. การพัฒนาที่ทำให้ SSD (Solid State Drive) เข้ามาแทนที่ Magnetic Disk ในคอมพิวเตอร์ยุคใหม่คืออะไร?

ก. มีราคาถูกกว่ามาก

ข. ไม่มีส่วนที่เคลื่อนไหว (Mechanical Parts) ทำให้เข้าถึงข้อมูลได้เร็วกว่าและทนทานกว่า

ค. ใช้การเข้าถึงข้อมูลแบบ Sequential Access เท่านั้น

ง. มีความจุสูงสุดที่ทำได้สูงกว่า Magnetic Disk

เฉลย: ข.

26. มาตรฐานการเชื่อมต่อในการส่งข้อมูลของหน่วยความจำจานแม่เหล็กหรือ SSD ที่ใช้กันอย่างแพร่หลายในปัจจุบันคืออะไร?

ก. PCI (Peripheral Component Interconnect)

ข. USB (Universal Serial Bus)

ค. SATA (Serial Advanced Technology Attachment)

ง. LPT (Line Printer Terminal)

เฉลย: ค.

27. สถานการณ์ใดที่เรียกว่า Cache Hit ในการทำงานของหน่วยความจำแคช?

ก. CPU พบข้อมูลที่ต้องการใน Cache Memory

ข. CPU ต้องไปค้นหาข้อมูลใน Main Memory

ค. เกิดข้อผิดพลาดในการถ่ายโอนข้อมูล

ง. Cache Memory ถูกเขียนทับด้วยข้อมูลใหม่

เฉลย: ก.

28. การโอนย้ายข้อมูลระหว่างหน่วยความจำแคชกับหน่วยความจำหลัก จะโอนย้ายข้อมูลในลักษณะใด?

ก. ทีละ 1 Word

ข. ทีละ 1 Byte

ค. ทีละ 1 Page

ง. เป็นก้อนข้อมูลที่เรียกว่า Block หรือ Line

เฉลย: ง.

29. การเชื่อมโยงแบบ Direct Mapping ใน Cache Memory มีหลักการวางข้อมูลอย่างไร?

ก. บล็อกใดๆ ของ Main Memory สามารถวางในตำแหน่งใดก็ได้ของ Cache

ข. บล็อกของ Main Memory จะถูกกำหนดให้วางในตำแหน่งเฉพาะเจาะจงของ Cache เท่านั้น

ค. บล็อกของ Main Memory ต้องวางในกลุ่ม (Set) ที่กำหนด

ง. ใช้สำหรับแทนที่ข้อมูลแบบ LRU เท่านั้น

เฉลย: ข.

30. การเชื่อมโยงแบบ Associative Mapping แตกต่างจากการเชื่อมโยงแบบ Direct Mapping อย่างไรในเรื่องของตำแหน่งการวางข้อมูล?

ก. การค้นหาข้อมูลทำได้รวดเร็วกว่า เพราะไม่ต้องตรวจสอบ Tag

ข. บล็อกใดๆ ของ Main Memory สามารถวางในช่องใดก็ได้ของ Cache

ค. บล็อกของ Main Memory จะต้องวางในกลุ่ม (Set) ที่กำหนดเท่านั้น

ง. มีความซับซ้อนในการออกแบบต่ำกว่า Direct Mapping

เฉลย: ค.

30. การเชื่อมโยงแบบ Associative Mapping มีความซับซ้อนในการค้นหาข้อมูล (Search) สูงกว่า Direct Mapping เพราะเหตุใด?

ก. บล็อกของ Main Memory ถูกกำหนดให้วางในตำแหน่งเฉพาะเจาะจงของ Cache เท่านั้น

ข. การค้นหาไม่ต้องตรวจสอบ Tag เลย

ค. ต้องเปรียบเทียบ Tag ของบล็อกที่ต้องการกับ Tag ทุกช่องใน Cache พร้อมกัน (Parallel Search)

ง. ใช้การจัดเก็บข้อมูลแบบ Write-through เท่านั้น

เฉลย: ค.

31. การเชื่อมโยงแบบ Set-Associative Mapping เป็นการรวมคุณสมบัติใดไว้ด้วยกัน?

ก. การวางตำแหน่งแบบกลุ่ม (Set) และการค้นหาแบบอิสระภายในกลุ่ม (Associative)

ข. การวางตำแหน่งแบบตายตัว (Direct) และการค้นหาตามค่า (Associative)

ค. การแทนที่แบบ LRU และ FIFO

ง. การเขียนข้อมูลแบบ Write-through และ Write-back

เฉลย: ก.

32. นโยบายการแทนที่ข้อมูล LRU (Least Recently Used) ใน Cache Replacement Policy มีหลักการเลือกบล็อกเพื่อแทนที่อย่างไร?

ก. เลือกบล็อกที่ถูกนำเข้ามาในแคชเป็นอันดับแรก

ข. เลือกบล็อกที่มีการเรียกใช้งานน้อยที่สุดตั้งแต่เริ่มต้น

ค. เลือกบล็อกแบบสุ่ม

ง. เลือกบล็อกที่ไม่ได้ถูกเรียกใช้งานเป็นเวลานานที่สุด

เฉลย: ง.

33. นโยบายการแทนที่ข้อมูล FIFO (First-in First-out) มีหลักการเลือกบล็อกเพื่อแทนที่อย่างไร?

ก. เลือกบล็อกที่ไม่ได้ถูกเรียกใช้งานเป็นเวลานานที่สุด

ข. เลือกบล็อกที่ถูกนำเข้ามาในแคชเป็นอันดับแรก (นานที่สุด)

ค. เลือกบล็อกที่มีการเรียกใช้งานน้อยที่สุดตั้งแต่เริ่มต้น

ง. เลือกบล็อกที่มี Tag bit เป็น 0

เฉลย: ข.

34. นโยบายการแทนที่ข้อมูล LFU (Least Frequently Used) มีหลักการเลือกบล็อกเพื่อแทนที่อย่างไร?

ก. เลือกบล็อกที่มีการเรียกใช้งานน้อยที่สุด

ข. เลือกบล็อกที่ถูกนำเข้ามาในแคชเป็นอันดับแรก

ค. เลือกบล็อกที่ไม่ได้ถูกเรียกใช้งานเป็นเวลานานที่สุด

ง. เลือกบล็อกแบบสุ่ม

เฉลย: ก.

35. การเลือกใช้การแทนที่ข้อมูลแบบ Random ใน Cache Replacement Policy มีข้อดีหลักอย่างไร?

ก. มีประสิทธิภาพสูงสุด เพราะคาดการณ์การใช้งานในอนาคตได้แม่นยำ

ข. ไม่จำเป็นต้องมีวงจรในการเก็บข้อมูลการใช้งาน (Usage Counter หรือ Time Stamp)

ค. มีความซับซ้อนของวงจรควบคุมต่ำที่สุด

ง. เหมาะสำหรับการใช้งานแบบ Write-back เท่านั้น

เฉลย: ค.

36. นโยบายการเขียนข้อมูลแบบ Write-back แตกต่างจาก Write-through อย่างไร?

ก. Write-through ต้องใช้ Dirty Bit เพื่อตรวจสอบการเปลี่ยนแปลงข้อมูล

ข. Write-back มีความเร็วช้ากว่าเพราะต้องเขียนข้อมูลลง Main Memory ทันที

ค. Write-through จะบันทึกข้อมูลที่เปลี่ยนเฉพาะใน Cache เท่านั้น

ง. Write-back จะปรับปรุงข้อมูลใน Cache ก่อน
และเขียนลง Main Memory ก็ต่อเมื่อบล็อกนั้นถูกแทนที่

เฉลย: ง.

37. หลักการทำงานของ หน่วยความจำเสมือน (Virtual Memory) คืออะไร?

ก. เพิ่มความเร็วในการเข้าถึงข้อมูลโดยใช้ Cache

ข. ใช้พื้นที่ของหน่วยความจำสำรอง (เช่น Hard Disk) เสมือนเป็นส่วนขยายของหน่วยความจำหลัก

ค. จัดการการสื่อสารข้อมูลแบบอนุกรม ง. ดำเนินการคำสั่งแบบ Pipelining

เฉลย: ข.

38. ข้อดีหลักของหน่วยความจำเสมือน (Virtual Memory) คืออะไร?

ก. มีความเร็วในการเข้าถึงข้อมูลสูงมาก

ข. ไม่ต้องใช้พื้นที่ในหน่วยความจำหลักเลย

ค. ช่วยให้สามารถรันโปรแกรมที่มีขนาดใหญ่กว่าหน่วยความจำหลักจริงได้

ง. ลดจำนวนครั้งของการเกิด Page Fault

เฉลย: ค.

39. หน่วยความจำเสมือนระบบ Paging System ทำการแบ่งหน่วยความจำอย่างไร?

ก. แบ่งหน่วยความจำเสมือนเป็นส่วนๆ ที่มีขนาดไม่เท่ากัน (Segmentation)

ข. แบ่งหน่วยความจำหลักเป็นส่วนๆ ที่มีขนาดไม่เท่ากัน

ค. แบ่งหน่วยความจำสำรองเป็น Block

ง. แบ่งหน่วยความจำเสมือน (Virtual) และหน่วยความจำจริง (Physical) เป็นส่วนย่อยที่มีขนาดเท่ากัน (Page/Frame)

เฉลย: ง.

39. หน่วยความจำเสมือนระบบ Paging System ทำการแบ่งหน่วยความจำอย่างไร?

ก. แบ่งหน่วยความจำเสมือน (Virtual) และหน่วยความจำจริง (Physical) เป็นส่วนย่อยที่มีขนาดเท่ากัน (Page/Frame)

ข. แบ่งหน่วยความจำเสมือนเป็นส่วนๆ ที่มีขนาดไม่เท่ากัน (Segmentation)

ค. แบ่งหน่วยความจำหลักเป็นส่วนๆ ที่มีขนาดไม่เท่ากัน

ง. แบ่งหน่วยความจำสำรองเป็น Block

เฉลย: ก.

40. เหตุการณ์ Page Fault เกิดขึ้นเมื่อใดในระบบ Paging System?

ก. เมื่อโปรแกรมสามารถเข้าถึงเพจที่ต้องการในหน่วยความจำจริงได้ทันที

ข. เมื่อ CPU ทำการเขียนข้อมูลลงในเพจ

ค. เมื่อทำการแปลงที่อยู่เสมือนเป็นที่อยู่จริงได้สำเร็จ

ง. เมื่อระบบปฏิบัติการตรวจสอบ Page Table แล้วพบว่าเพจที่ต้องการไม่ได้อยู่ในหน่วยความจำจริง

เฉลย: ง.

41. หากหน่วยความจำเสมือน (Virtual Address) มีขนาด 40 บิต และขนาดของหน่วยความจำจริง (Physical Address) มีขนาด 32 บิต จำนวนที่อยู่เสมือนจะมีค่าเท่ากับเท่าใด?

ก. 2^{32} ข. 2^{40} ค. $2^{40} - 2^{32}$ ง. $2^{32} \times 2^{40}$

เฉลย: ข.

42. ในระบบ Paging หากขนาดของเพจ (Page Size) เท่ากับ 2^{12} ไบต์ และที่อยู่เสมือน (Virtual Address) มีขนาด 32 บิต จำนวนบิตที่ใช้สำหรับระบุ Page Offset คือกี่บิต?

ก. 12 บิต ข. 20 บิต

ค. 32 บิต ง. 44 บิต

เฉลย: ก.

43. การแปลงที่อยู่เสมือน (Virtual Address) ไปเป็นที่อยู่จริง (Physical Address) ในระบบ Paging ต้องอาศัยการค้นหาข้อมูลจากส่วนใด?

ก. Cache Memory ข. Control Unit

ค. Page Table (ตารางเพจ) ง. Registers

เฉลย: ค.

44. DAT (Dynamic Address Translation) เป็นกระบวนการที่ใช้ในการทำอะไร?

ก. คำนวณค่า Bias ในเลขทศนิยม

ข. คำนวณผลคูณใน Booth's Algorithm

ค. การจัดการกับ Cache Miss

ง. การแปลงที่อยู่เสมือน (Virtual) ให้เป็นที่อยู่จริง (Physical) ในขณะรันโปรแกรม

เฉลย: ง.

45. PAGE REPLACEMENT ALGORITHMS ถูกนำมาใช้เพื่อแก้ปัญหาหลักใดในการบริหารการจัดการหน่วยความจำเสมือน?

ก. การเกิด Cache Miss

ข. การเลือกเพจที่จะถูกนำออกจากหน่วยความจำจริงเมื่อเกิด Page Fault

ค. การกำหนดขนาดของ Page Table

ง. การแปลงที่อยู่เสมือนให้เป็นที่อยู่จริง

เฉลย: ข.

46. ในการจัดการหน่วยความจำเสมือนด้วยวิธีสับเปลี่ยนแบบ FIFO (First-in First-out) จะพิจารณาเพจที่จะถูกนำออกอย่างไร?

ก. เพจที่ถูกใช้งานถี่น้อยที่สุด

ข. เพจที่ไม่ได้ถูกใช้งานนานที่สุด

ค. เพจที่ถูกนำเข้ามาในหน่วยความจำจริงเป็นลำดับแรก (นานที่สุด)

ง. เพจที่มีค่า Page Table Entry เป็น 0

เฉลย: ค.

47. วิธีสับเปลี่ยนแบบ Optimal Page Replacement (OPT) มีหลักการเลือกเพจที่จะถูกนำออกอย่างไร?

ก. เลือกเพจที่จะไม่ถูกเรียกใช้งานเป็นเวลานานที่สุดในอนาคต

ข. เลือกเพจที่ถูกนำเข้ามาในหน่วยความจำจริงเป็นลำดับแรก

ค. เลือกเพจที่ถูกเรียกใช้งานล่าสุด

ง. เลือกเพจแบบสุ่ม

เฉลย: ก.

48. วิธีสับเปลี่ยนแบบ Least Recently Used (LRU) มีหลักการเลือกเพจที่จะถูกนำออกอย่างไร?

ก. เลือกเพจที่ถูกนำเข้ามาในหน่วยความจำจริงเป็นลำดับแรก

ข. เลือกเพจที่ถูกใช้งานถี่น้อยที่สุด

ค. เลือกเพจแบบสุ่ม

ง. เลือกเพจที่ไม่ได้ถูกเรียกใช้งานมาเป็นเวลานานที่สุดในอดีต

เฉลย: ง.